Neural networks and CNN

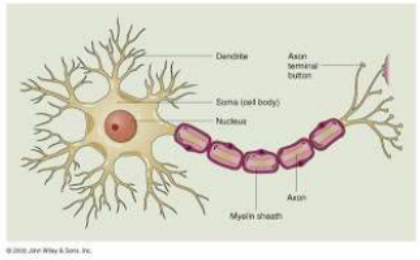
Biplab Banerjee

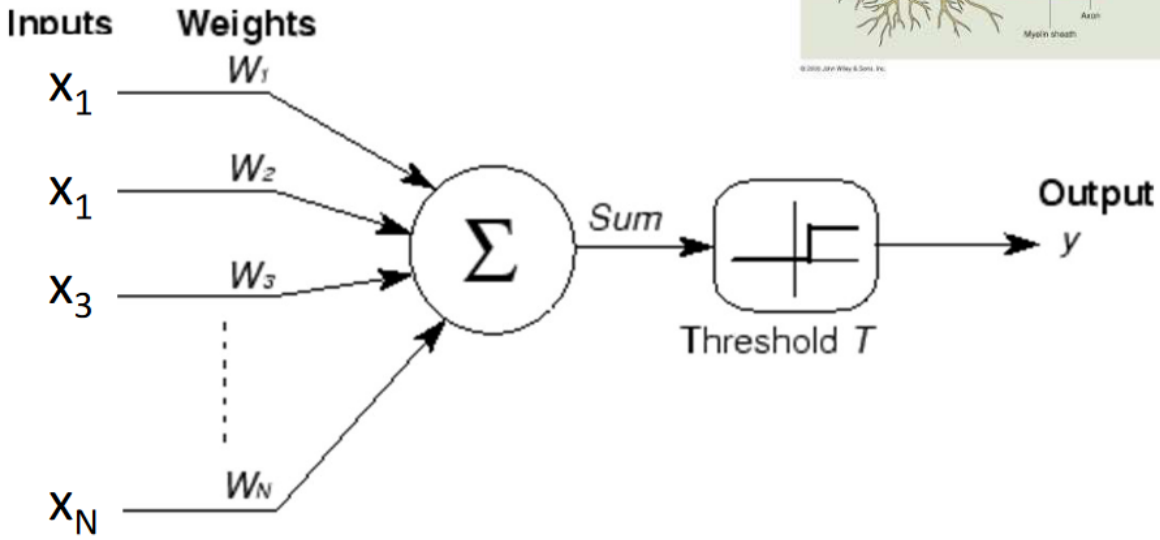
Thanks to towardsdatascience, Vincente Ordonez

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Perceptron Model

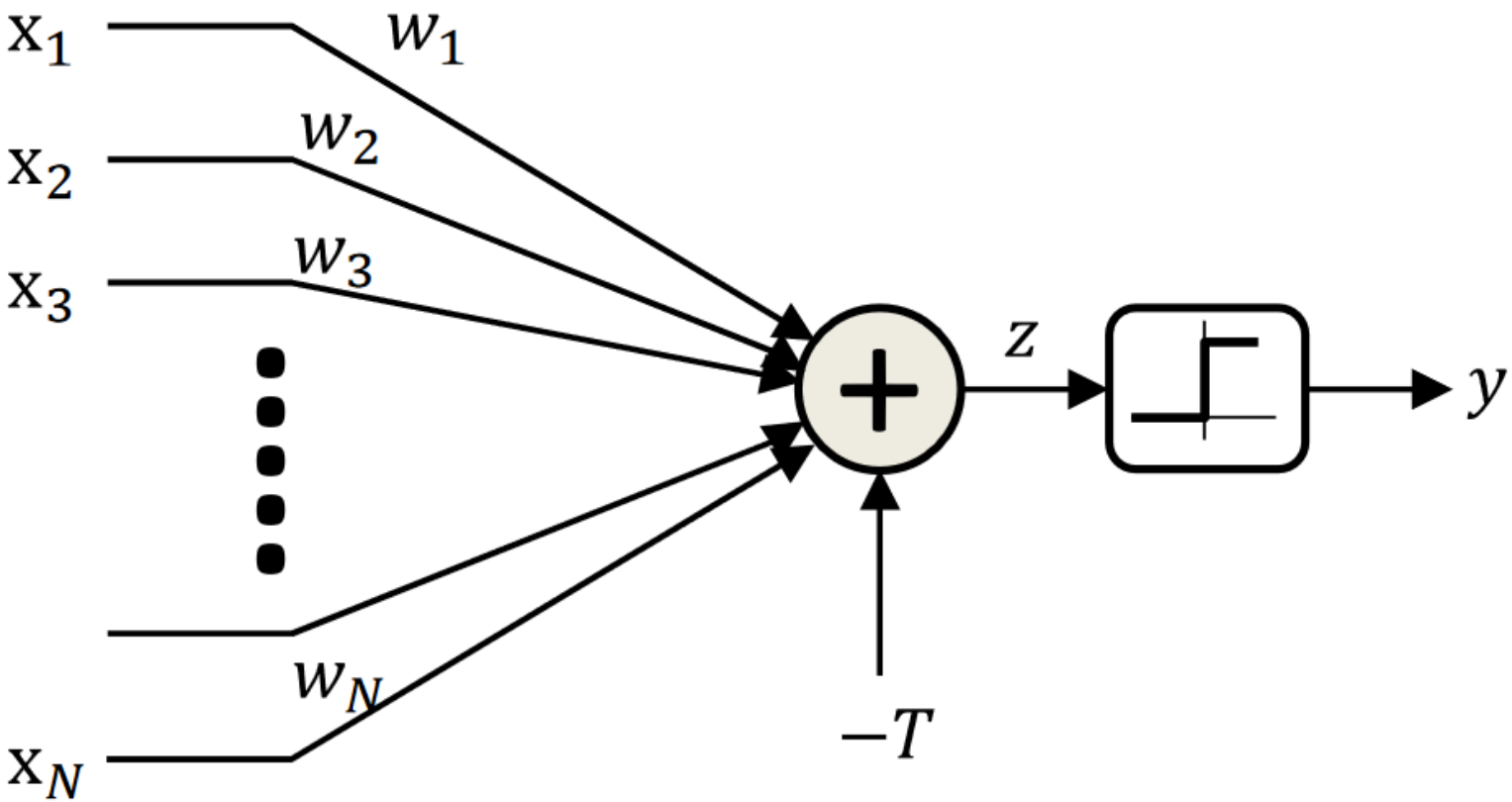
Frank Rosenblatt (1957) - Cornell University





$$y = \begin{cases} 1 \; if \sum_i w_i x_i \geq T \\ 0 \; else \end{cases}$$

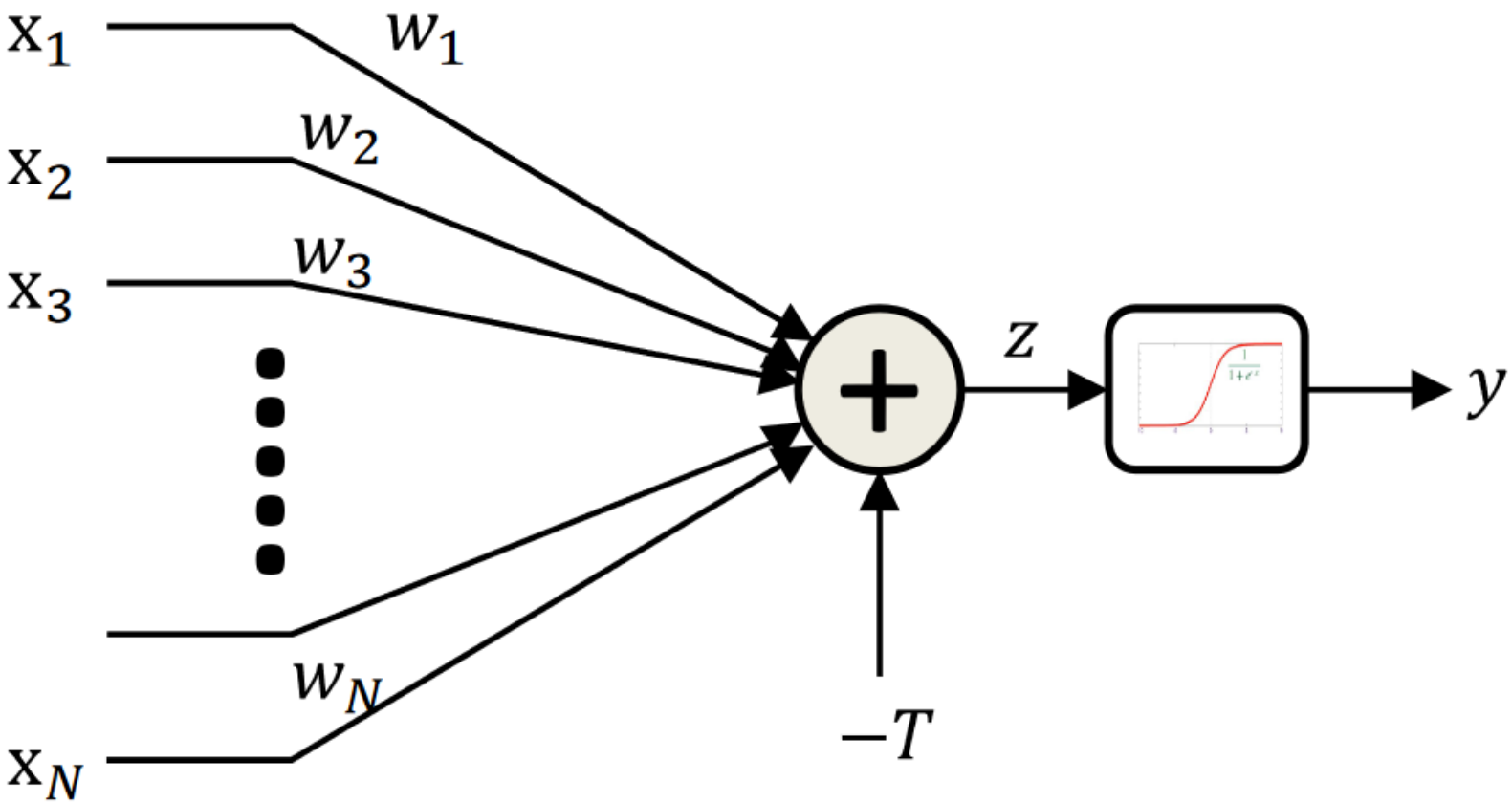
This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.



$$z = \sum_i w_i x_i - T$$

$$y = \begin{cases} 1 \; if \; z \geq 0 \\ 0 \; else \end{cases}$$

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.



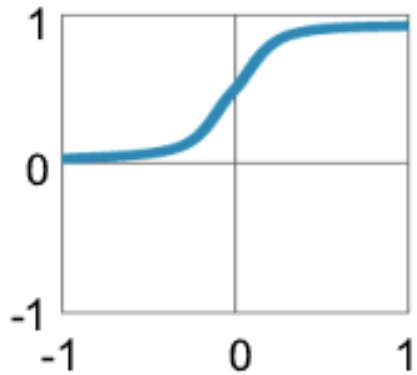
$$z = \sum_i w_i x_i - T$$

$$y = \frac{1}{1 + exp(-z)}$$

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

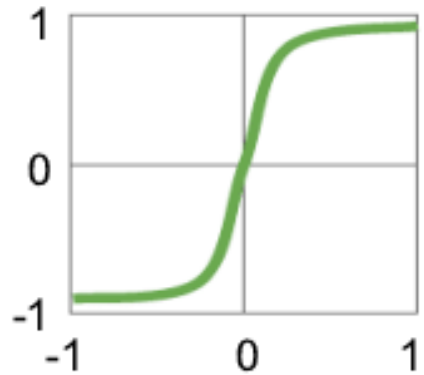
Traditional Non-Linear Activation Functions

Sigmoid



$y=1/(1+e^{-x})$

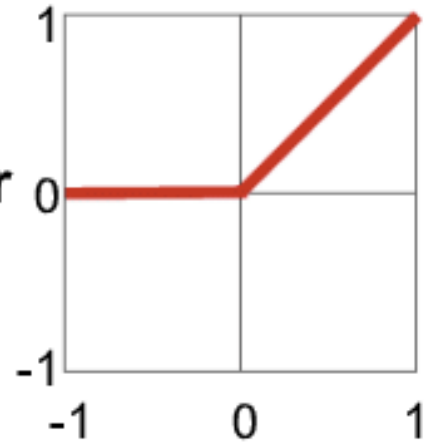
Hyperbolic Tangent



$y=(e^{x}-e^{-x})/(e^{x}+e^{-x})$

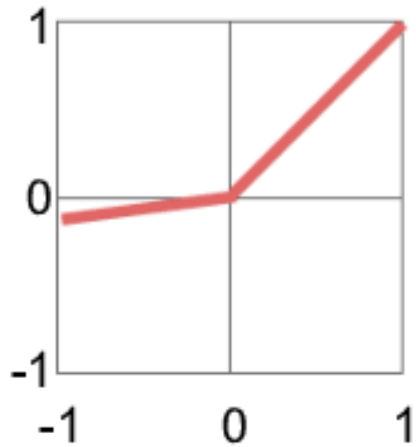
Modern Non-Linear Activation Functions

Rectified Linear Unit (ReLU)



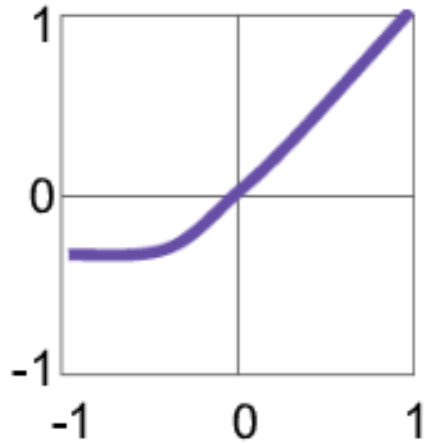
$y=\max(0,x)$

Leaky ReLU



$y=\max(\alpha x,x)$

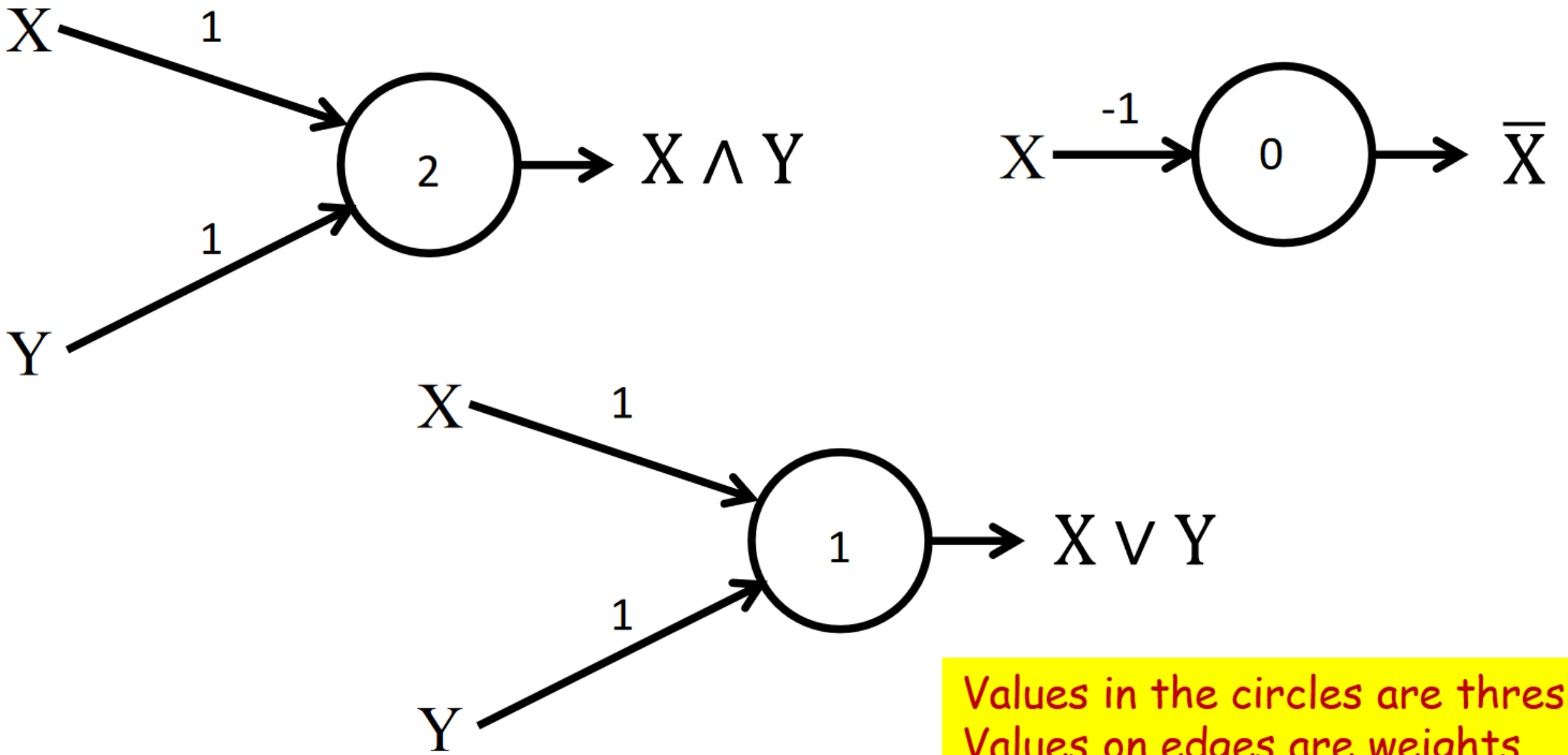
Exponential LU



$$y=\begin{cases} x, & x\geq 0 \\ \alpha(e^{x}-1), & x<0 \end{cases}$$

α = small const. (e.g. 0.1)

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

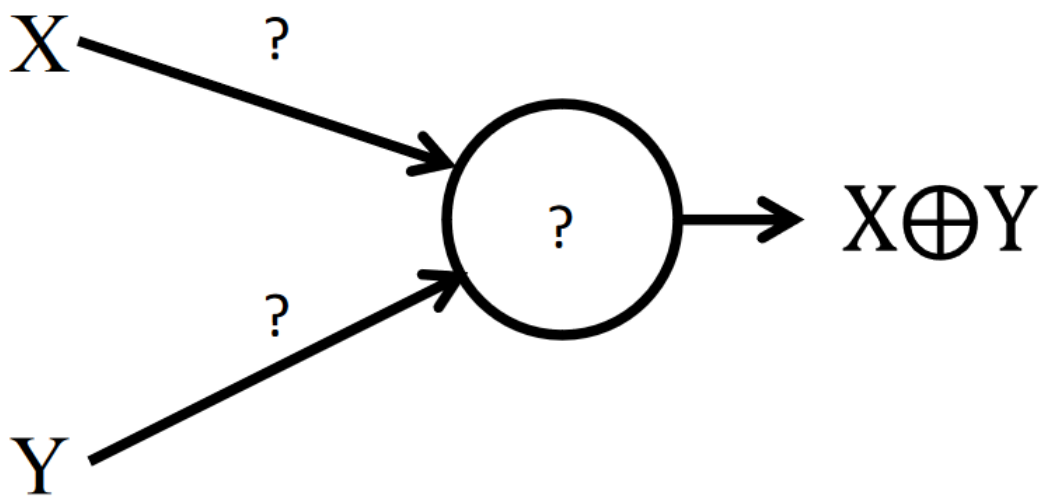


Values in the circles are thresholds
Values on edges are weights

- A perceptron can model any simple binary Boolean gate

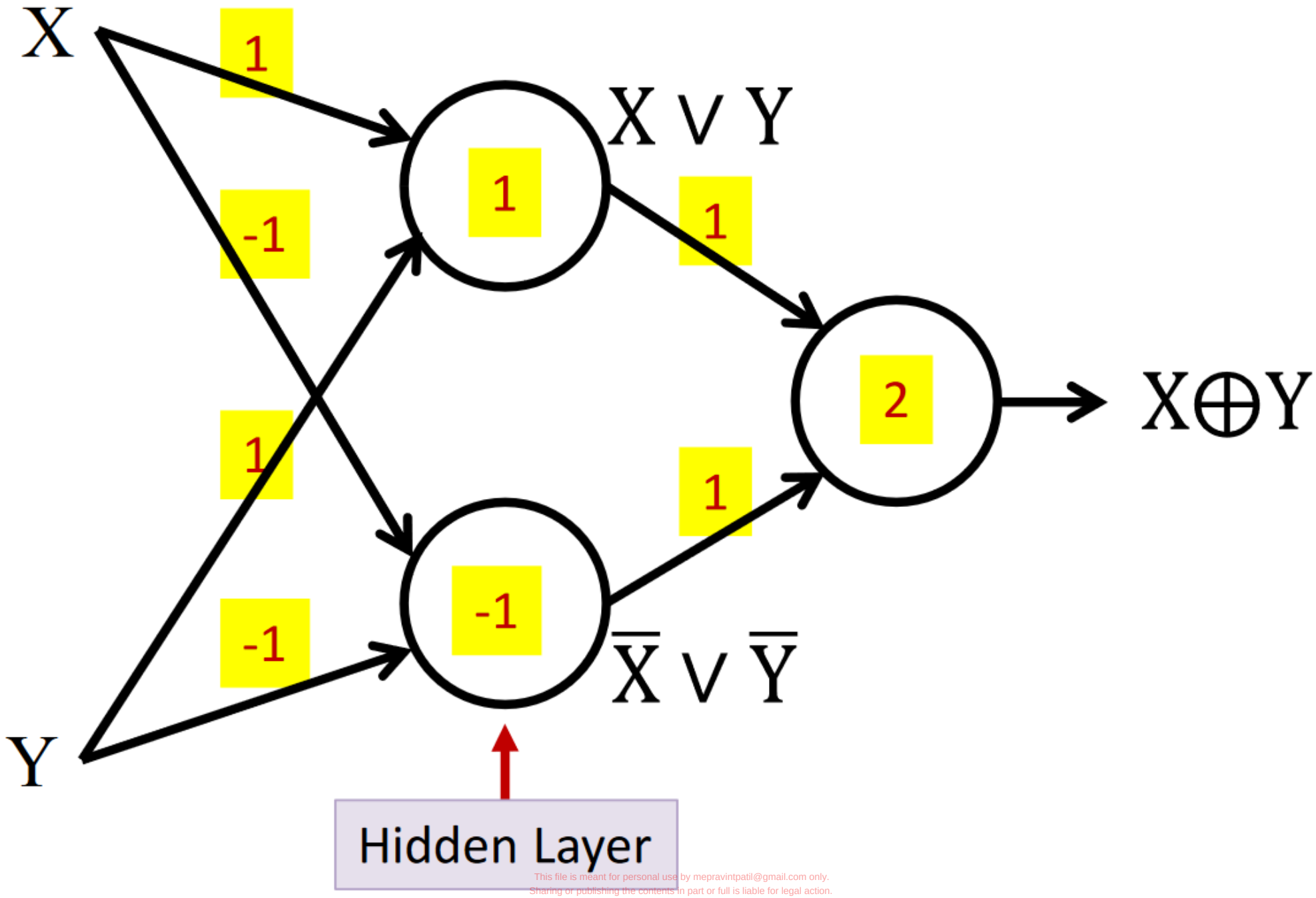
This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

The perceptron is not enough



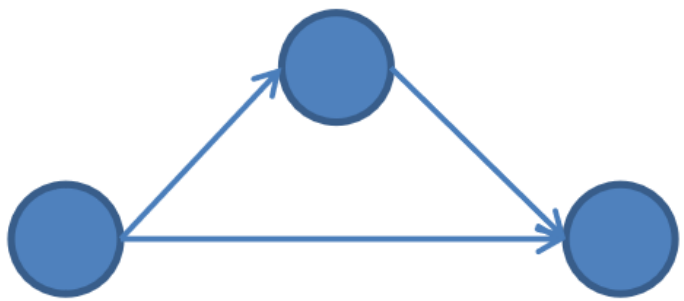
Cannot compute an XOR

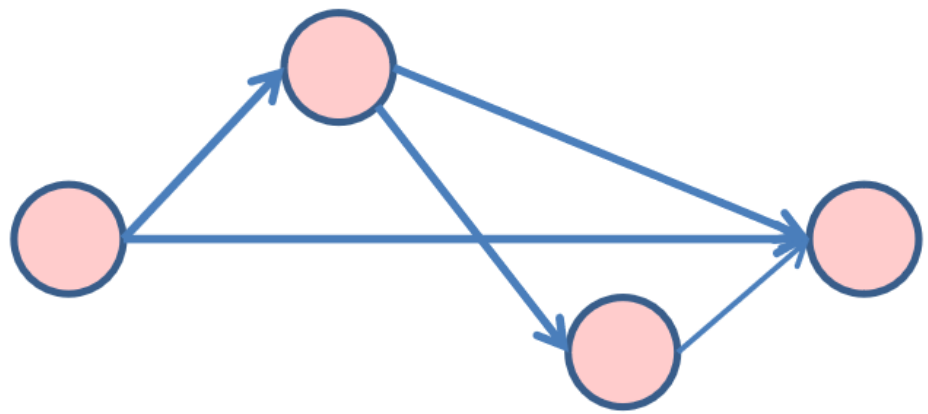
This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

- In any directed network of computational elements with input source nodes and output sink nodes, “depth” is the length of the longest path from a source to a sink
  - A “source” node in a directed graph is a node that has only outgoing edges
  - A “sink” node is a node that has only incoming edges

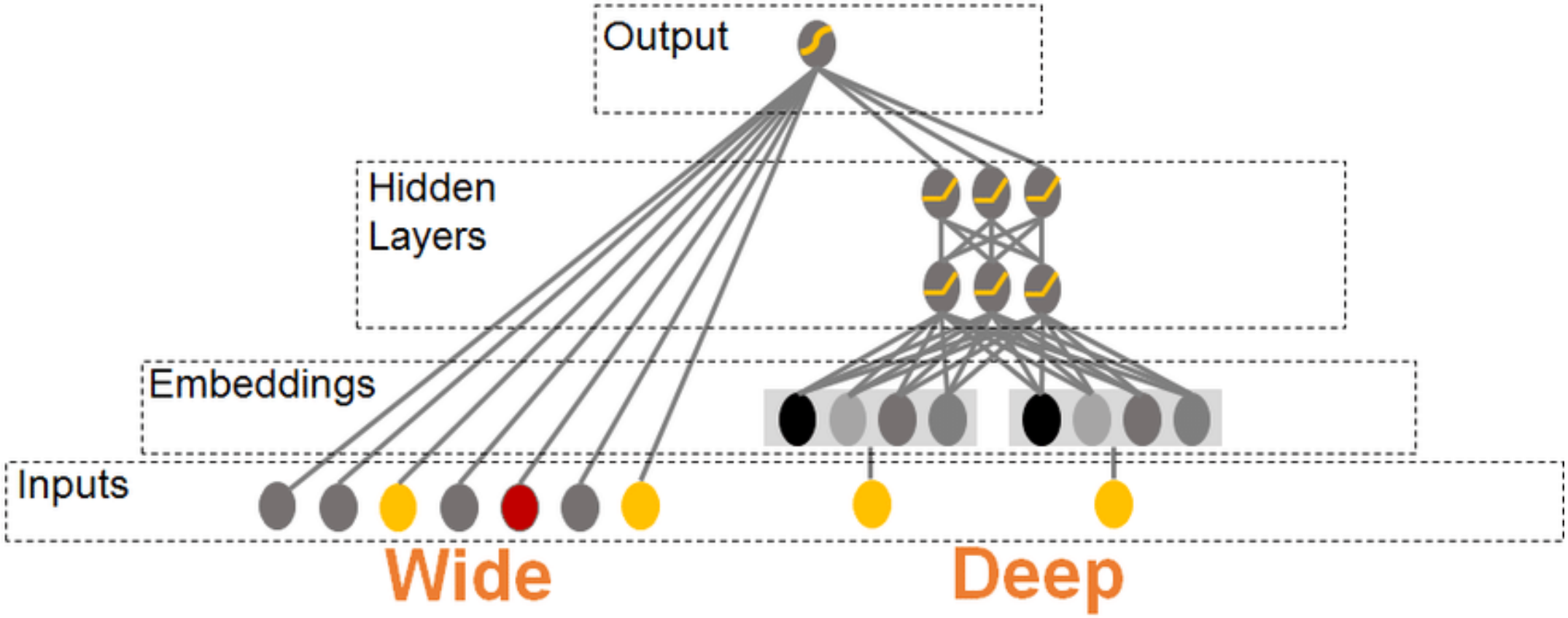




- Left: Depth = 2. Right: Depth = 3

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Wide vs deep networks



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Wide neural networks

- A sufficiently wide single-hidden-layer neural network with a nonlinear activation can approximate a large class of continuous functions.
- Compared to very deep architectures, wide networks rely more on ample neurons in one or two layers rather than intricate multi-layer designs.

✓Wide (shallow) networks, unlike deep architectures, do not learn hierarchically abstract features as effectively.

✓**With enough width, the network can memorize training examples (especially in small or moderate datasets).**

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Memorization in wide networks

Think of a simple linear model first:

$$y = Xw + \varepsilon, \quad X \in \mathbb{R}^{n \times p}.$$

If $p \geq n$ and X has rank n, there exist infinitely many w such that

$$Xw = y$$

(i.e., *zero* training error), including ones that fit the noise ε.

A wide neural net often behaves similarly because the training problem becomes **underdetermined**: many parameter settings achieve near-zero training loss, and without regularization/implicit bias, you may pick a solution that also fits noise (poor generalization).

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Deep neural networks

- In deep networks, lower layers tend to learn simple features (edges, corners, color blobs), while higher layers learn more complex patterns (object parts, semantic concepts). This hierarchical decomposition of features can lead to robust and generalizable representations.
- Deeper layers can gradually transform the input space into more compact, task-relevant feature spaces. This transformation helps remove irrelevant variations in the data (e.g., noise), focusing on features that matter for the target task.

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

- A **2-layer (shallow) ReLU net** with width m,

$$f(x) = \sum_{j=1}^{m} a_j \, [w_j x + b_j]_+ + c$$

  can create at most m kink locations (roughly: each ReLU can introduce one breakpoint), so it has $O(m)$ linear pieces.
- A **deep** ReLU net composes such operations:

$$f(x) = f_L \circ f_{L-1} \circ \cdots \circ f_1(x)$$

  and composition can **multiply** the number of pieces.

A simple rule of thumb:

- if each layer can create ~m "folds/pieces", then after L layers you can get on the order of \text{#pieces} \approx m^L

  (i.e., **exponential in depth**).

Why this matters

Suppose the target function needs K pieces (kinks) to model well.

- Shallow needs width $m \approx K$.
- Deep can get K with $m^L \approx K \Rightarrow m \approx K^{1/L}$.

So for the **same complexity** K, depth can reduce width (and parameters) drastically.

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

How does it matter?

For input dim d:

- Shallow params: $p_{\text{shallow}} \sim O(md) \sim O(Kd)$
- Deep params (width m, depth L): $p_{\text{deep}} \sim O(Lmd) \sim O(LK^{1/L}d)$

If K is large and $L > 1$, then $LK^{1/L} \ll K$.

So **deep represents the same function with many fewer parameters.**

Why that helps: with fewer degrees of freedom, it's harder to "wiggle" just to match random label noise. In learning theory language, many generalization bounds scale like

$$\text{gen gap} \lesssim O\left(\sqrt{\frac{\text{capacity}}{n}}\right),$$

and capacity often increases with parameter count or norms. Smaller effective capacity ⇒ smaller gap.

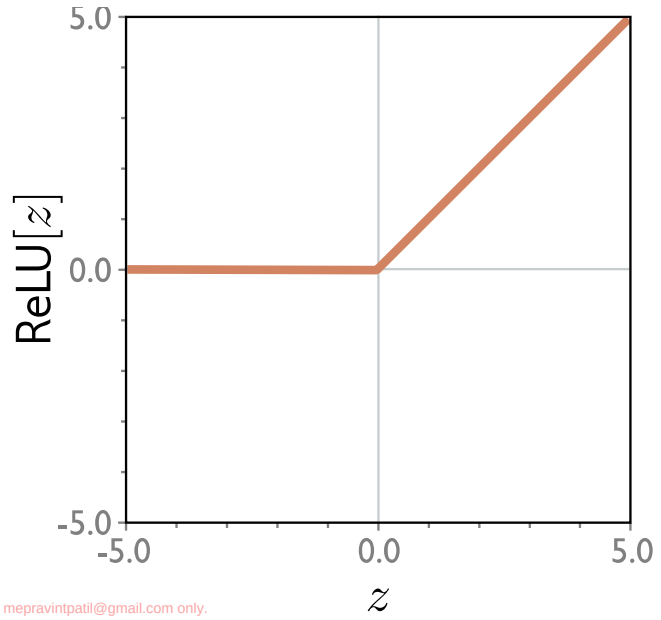
This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Example shallow network

Activation function

$$y = \mathrm{f}[x, \boldsymbol{\phi}]$$
$$= \phi_0 + \phi_1 \mathrm{a}[\theta_{10} + \theta_{11}x] + \phi_2 \mathrm{a}[\theta_{20} + \theta_{21}x] + \phi_3 \mathrm{a}[\theta_{30} + \theta_{31}x]$$

$$\mathrm{a}[z] = \mathrm{ReLU}[z] = \begin{cases} 0 & z < 0 \\ z & z \geq 0 \end{cases}.$$



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Example shallow network

$$y = \mathrm{f}[x, \boldsymbol{\phi}]$$
$$= \phi_0 + \phi_1 \mathrm{a}[\theta_{10} + \theta_{11} x] + \phi_2 \mathrm{a}[\theta_{20} + \theta_{21} x] + \phi_3 \mathrm{a}[\theta_{30} + \theta_{31} x]$$

$$\boldsymbol{\phi} = \{\phi_0, \phi_1, \phi_2, \phi_3, \theta_{10}, \theta_{11}, \theta_{20}, \theta_{21}, \theta_{30}, \theta_{31}\}$$

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Hidden units

$$y = \phi_0 + \phi_1 \mathrm{a}[\theta_{10} + \theta_{11}x] + \phi_2 \mathrm{a}[\theta_{20} + \theta_{21}x] + \phi_3 \mathrm{a}[\theta_{30} + \theta_{31}x].$$

Break down into two parts:

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

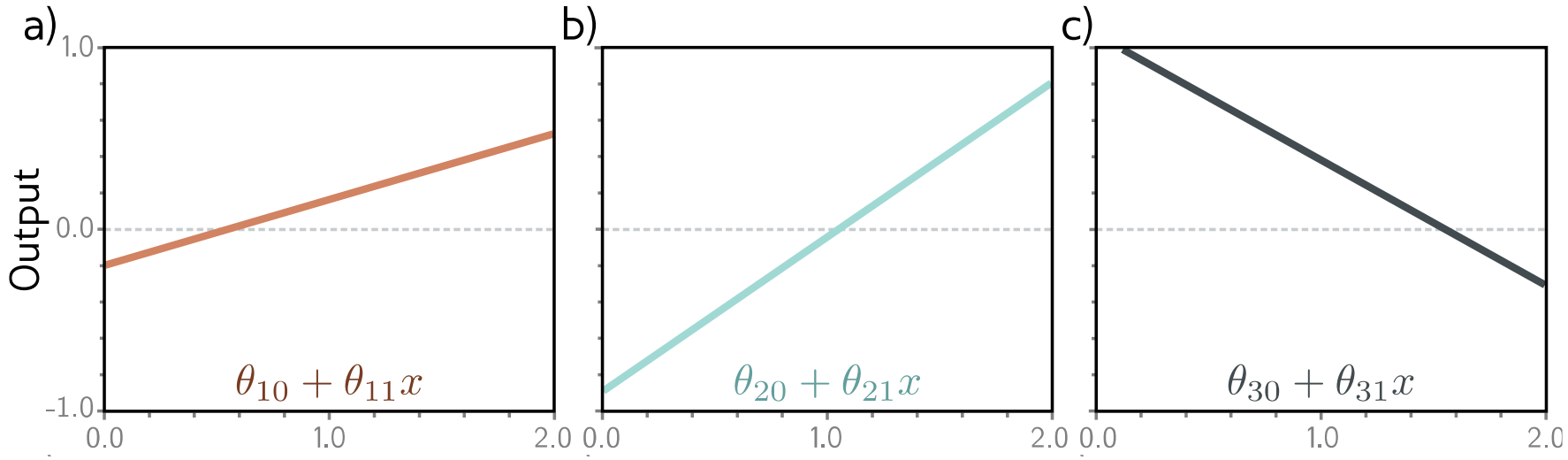
where:

Hidden units

$$\begin{aligned} h_1 &= \mathrm{a}[\theta_{10} + \theta_{11}x] \\ h_2 &= \mathrm{a}[\theta_{20} + \theta_{21}x] \\ h_3 &= \mathrm{a}[\theta_{30} + \theta_{31}x] \end{aligned}$$

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

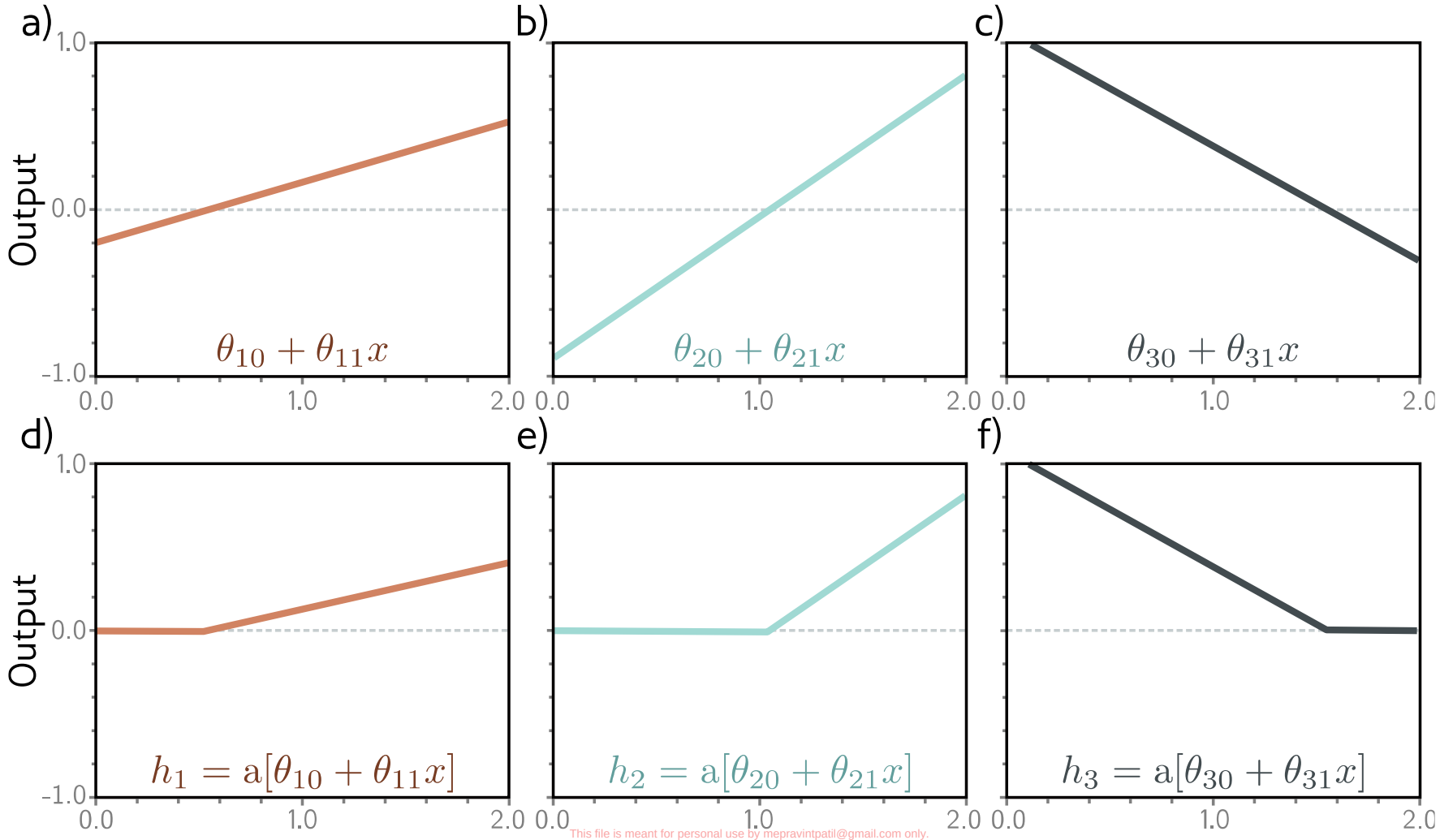
1. compute three linear functions



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

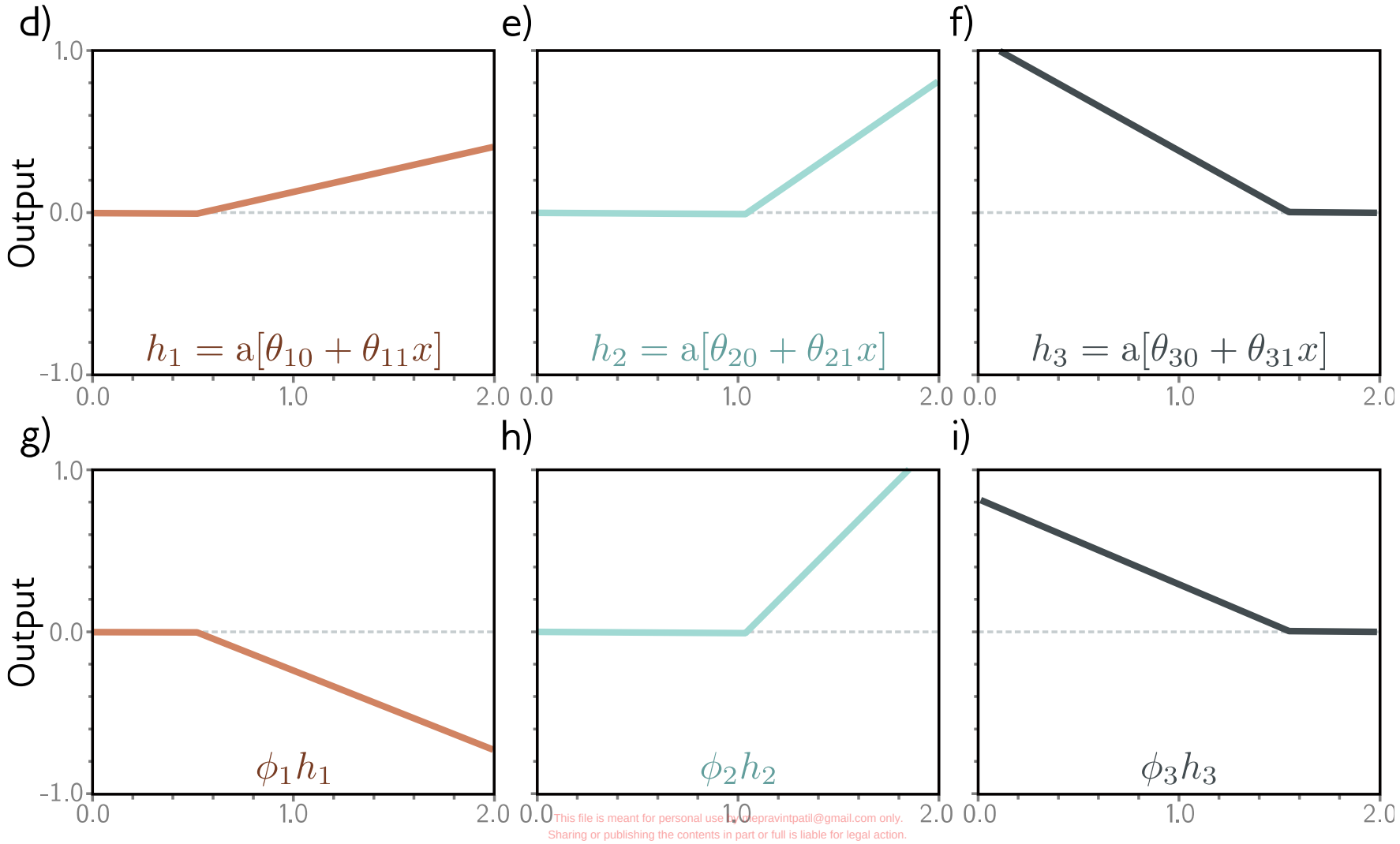
2. Pass through ReLU functions (creates hidden units)

$$h_1 = \mathrm{a}[\theta_{10} + \theta_{11}x]$$
$$h_2 = \mathrm{a}[\theta_{20} + \theta_{21}x]$$
$$h_3 = \mathrm{a}[\theta_{30} + \theta_{31}x],$$



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

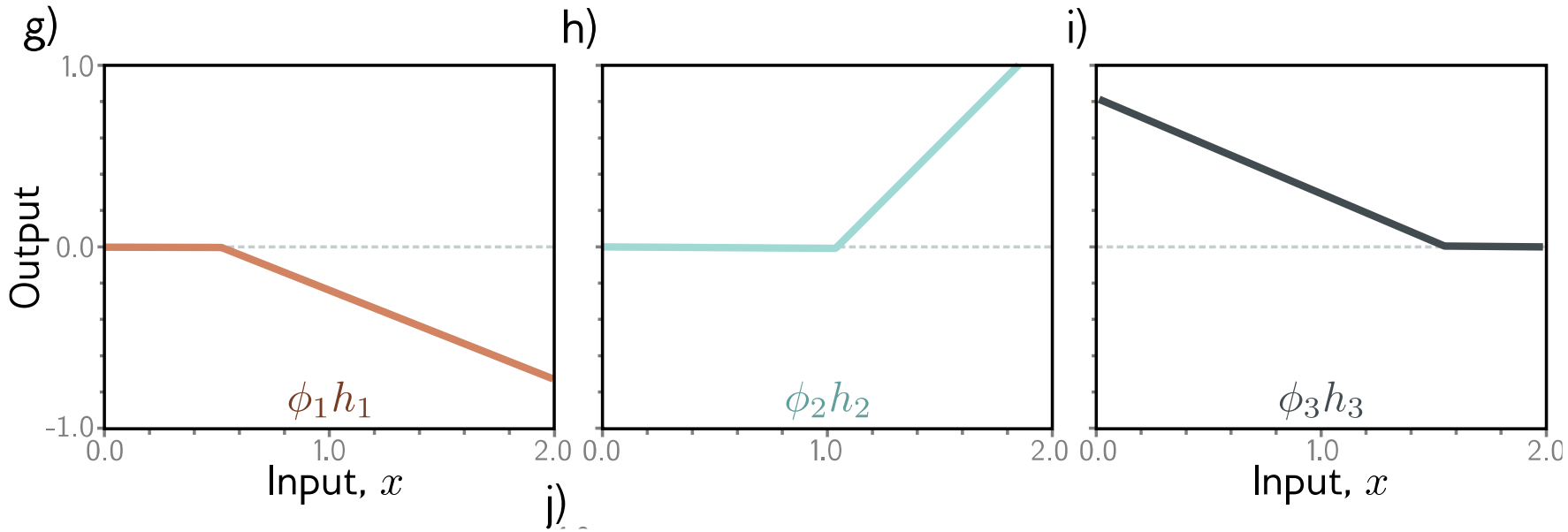
2. Weight the hidden units



This file is meant for personal use by deepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

4. Sum the weighted hidden units to create output

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

With 3 hidden units:

$$h_1 = \mathrm{a}[\theta_{10} + \theta_{11}x]$$
$$h_2 = \mathrm{a}[\theta_{20} + \theta_{21}x]$$
$$h_3 = \mathrm{a}[\theta_{30} + \theta_{31}x]$$

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

With D hidden units:

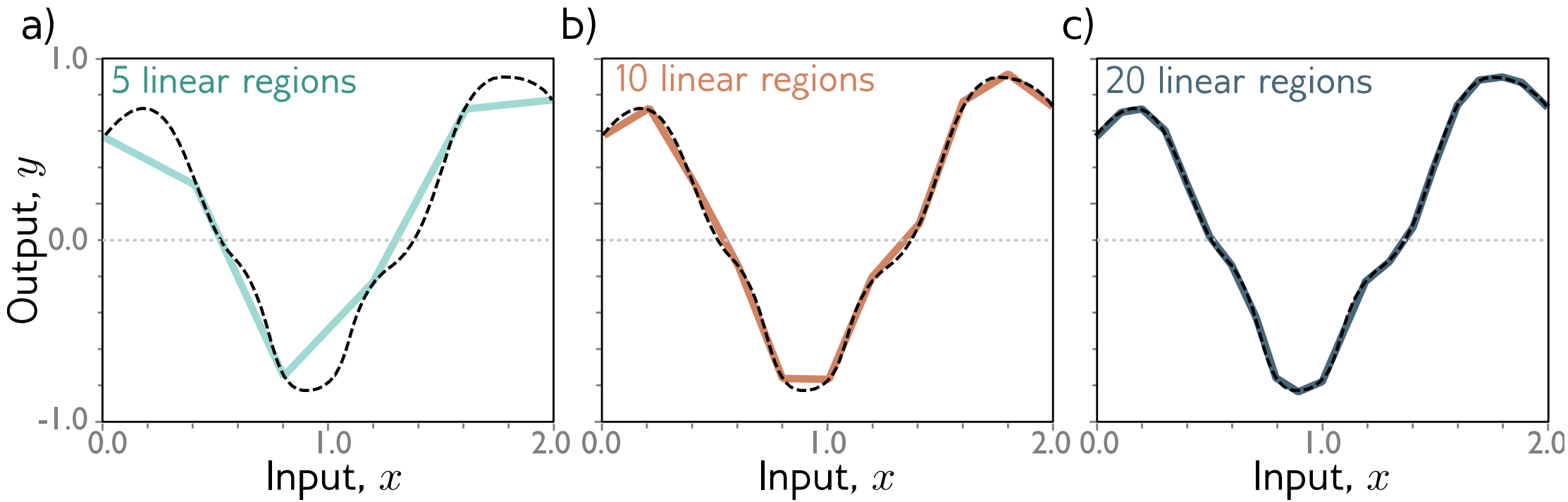
$$h_d = \mathrm{a}[\theta_{d0} + \theta_{d1}x]$$

$$y = \phi_0 + \sum_{d=1}^{D} \phi_d h_d$$

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

With enough hidden units…

… we can describe any 1D function to arbitrary accuracy



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Now what happens with depth

In **1D** (input is a line), each ReLU introduces at most **one kink** (one point where it changes slope). With m ReLUs in a single hidden layer, you can create at most:

$$m + 1 \text{ intervals (pieces)}$$

That's the classic "m kinks $\Rightarrow m + 1$ linear segments" result.

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

So when you enter one region (say an interval in 1D), the signal $h(x)$ going into layer 2 is a linear function of x on that interval.

Now layer 2 computes for each of its neurons:

$$z_j^{(2)}(x) = w_j^\top h(x) + b_j$$

But inside that interval, $h(x)$ is linear in x, so $z_j^{(2)}(x)$ is also linear in x.
A linear function crosses 0 at **at most one point** inside that interval.

So: **each ReLU neuron in layer 2 can add (at most) one new split inside each existing interval.**

That means:

- If layer 2 has m ReLUs, it can split **each** existing region into up to $(m+1)$ subregions.

So the total regions after 2 layers is roughly:

$$R_2 \approx R_1 \cdot (m+1)$$

This is the "multiplication" effect.

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Universal approximation theorem

Let $K \subset \mathbb{R}^n$ be **compact** (think: a closed and bounded input region, like $[0,1]^n$).

Let $f : K \to \mathbb{R}$ be **continuous**.

Then for any $\varepsilon > 0$, there exists a **2-layer (one hidden layer) ReLU network** of the form

$$g(x) = \sum_{j=1}^{m} a_j \, \sigma(w_j^\top x + b_j) + c, \qquad \sigma(t) = \max(0, t)$$

such that

$$\sup_{x \in K} |f(x) - g(x)| < \varepsilon.$$

So with enough hidden neurons m, the ReLU network can get arbitrarily close to f everywhere on K.

A feed-forward neural network with at least one hidden layer, a **sufficient number** of hidden units, and a **nonlinear** activation function can approximate **any** continuous function on a compact set to an arbitrary degree of accuracy.

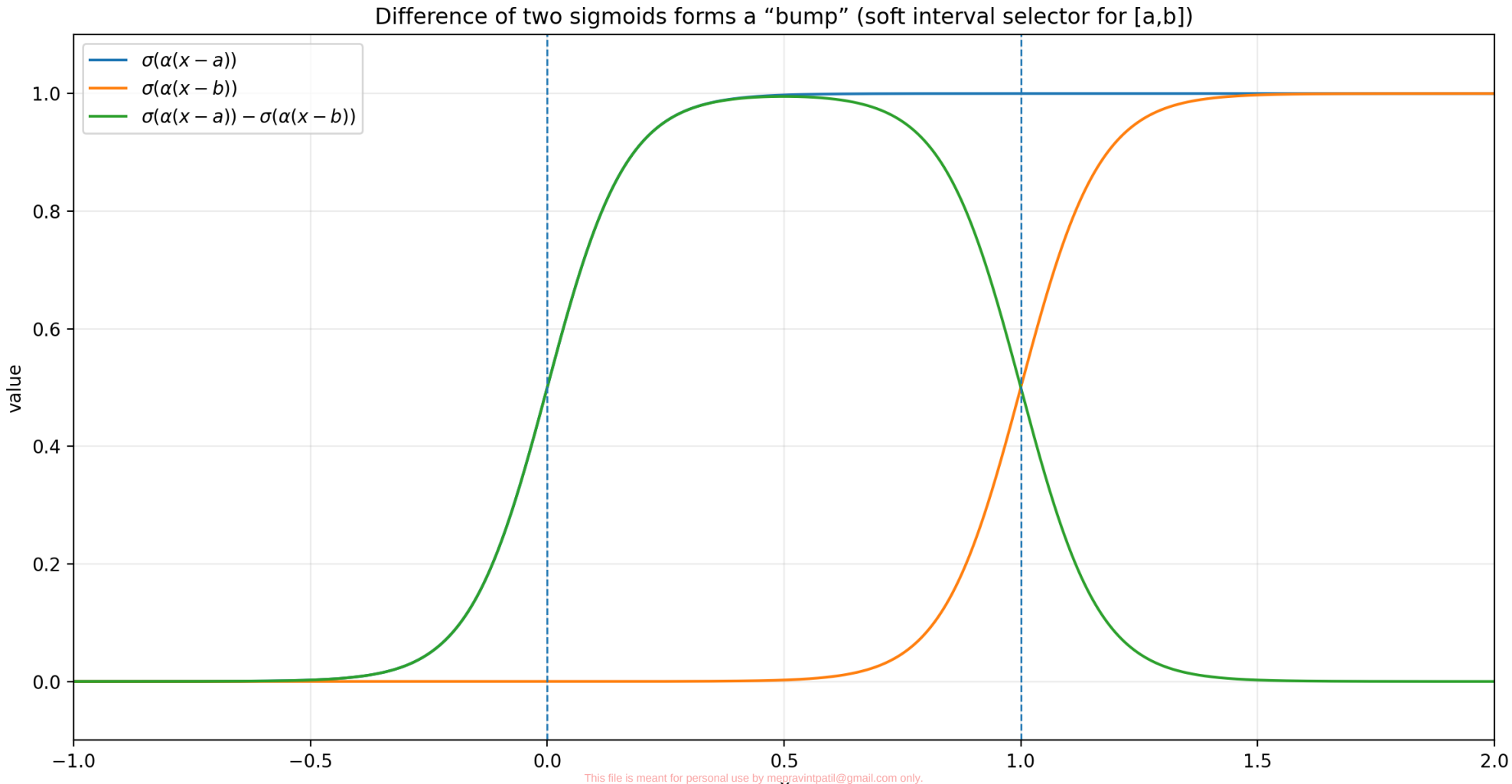
This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Universal approximator theorem

- It does **not** say a small network can do it.
- It does **not** tell you **how many** neurons you need.
- It does **not** guarantee training will find those weights.
- It applies to **bounded** domains; outside the domain, no guarantee.
- In practical machine learning, you have finite (often noisy) training data, so the model's ability to generalize beyond that data is **not** addressed by the universal approximation theorem.
-
- A model could "approximate" a function well in theory but still *overfit* or fail to generalize.

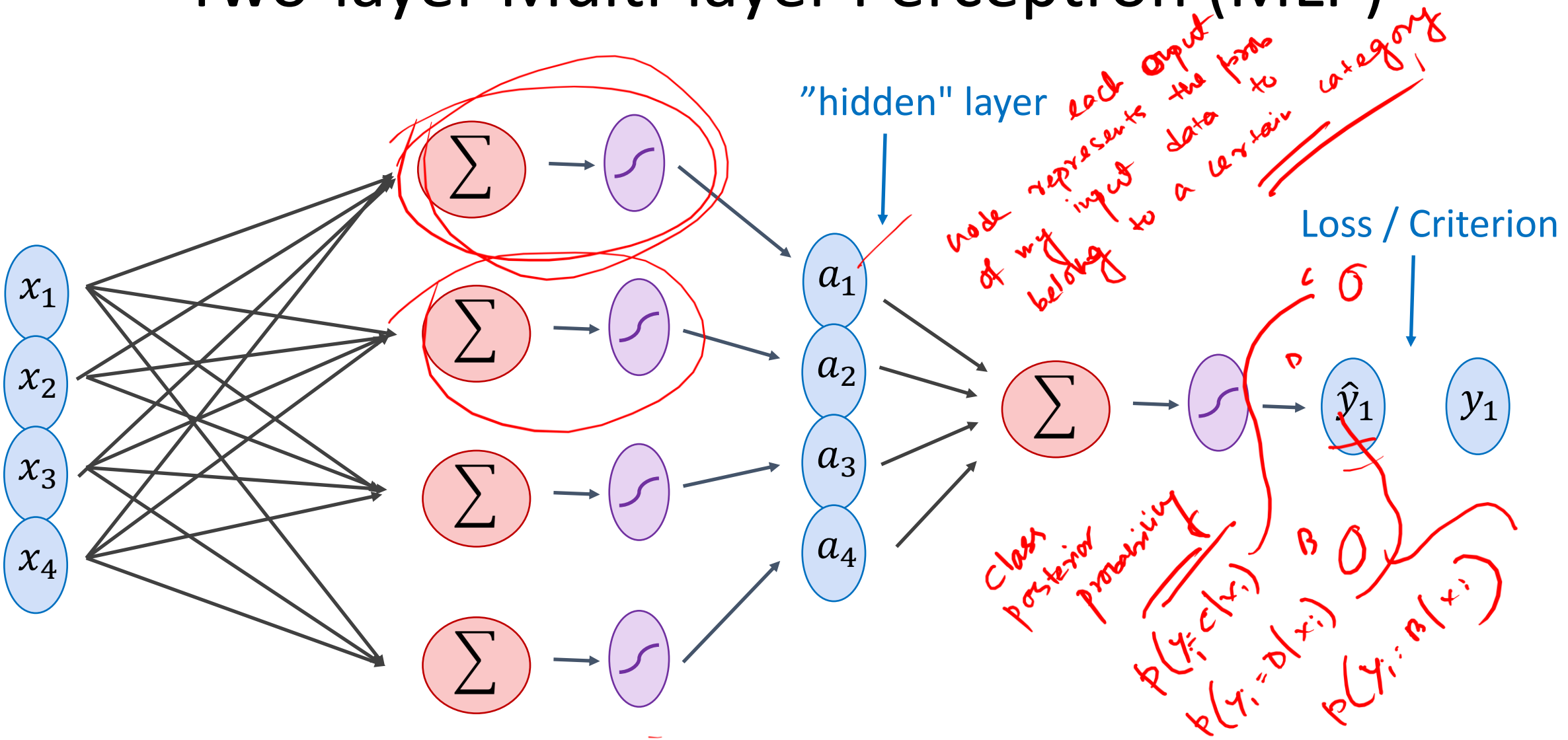
This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Can you have your arguments for Sigmoid network?



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Two-layer Multi-layer Perceptron (MLP)



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

① $P(y_i = C|x_i) + P(y_i = D|x_i) + P(y_i = B|x_i) = 1$

② $0 \leq P(y_i = C|x_i) \leq 1$. $0 \leq P(y_i = D|x_i) \leq 1$. $0 \leq P(y_i = B|x_i) \leq 1$

Correct class prob → 1

for incorrect classes ← 1

→ 0

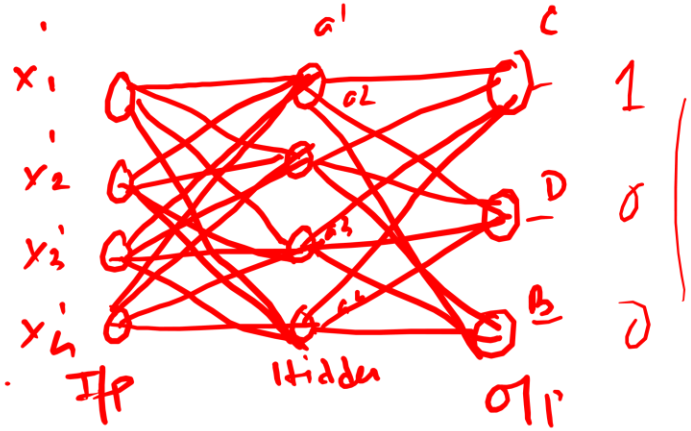
.8

.1 = 1

.1

$x' = [x'_1, x'_2, x'_3, x'_4]$

$[1, 0, 0]$



$\sigma(wx + b)$

Softmax $\left(\begin{array}{l} \sigma(w^C a + b^C) = \\ \sigma(w^D a + b^D) = \\ \sigma(w^B a + b^B) = \end{array}\right)$

X What are the optimum values of my weight & bias variables so that the model outputs, which are class posterior probabilities, should match the one-hot ground truths

→ For that I need a measure of loss < MSE / cross entropy

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Linear Softmax

$x_i = [x_{i1} \quad x_{i2} \quad x_{i3} \quad x_{i4}]$

$y_i = [1 \quad 0 \quad 0]$

$\hat{y}_i = [f_c \quad f_d \quad f_b]$

$$g_c = w_{c1}x_{i1} + w_{c2}x_{i2} + w_{c3}x_{i3} + w_{c4}x_{i4} + b_c$$

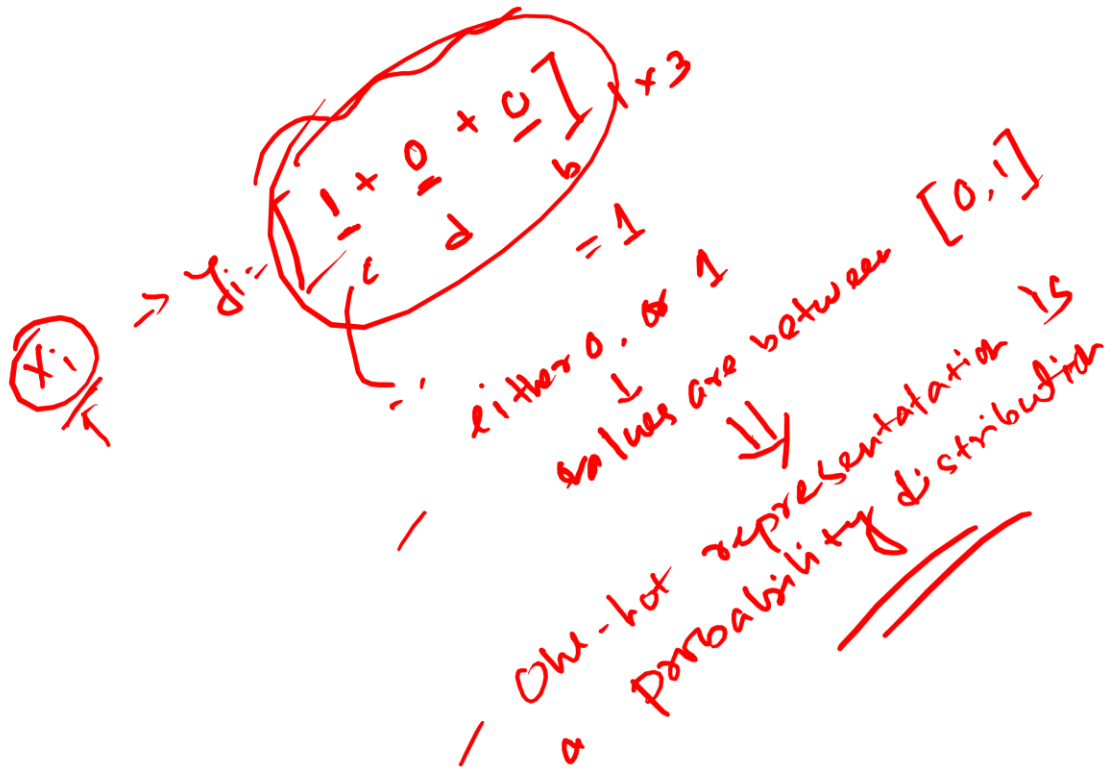
$$g_d = w_{d1}x_{i1} + w_{d2}x_{i2} + w_{d3}x_{i3} + w_{d4}x_{i4} + b_d$$

$$g_b = w_{b1}x_{i1} + w_{b2}x_{i2} + w_{b3}x_{i3} + w_{b4}x_{i4} + b_b$$

$$f_c = e^{g_c}/(e^{g_c}+e^{g_d} + e^{g_b})$$

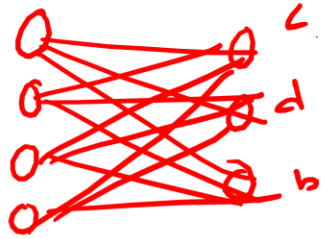
$$f_d = e^{g_d}/(e^{g_c}+e^{g_d} + e^{g_b})$$

$$f_b = e^{g_b}/(e^{g_c}+e^{g_d} + e^{g_b})$$



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Linear Softmax



$x_i = [x_{i1} \quad x_{i2} \quad x_{i3} \quad x_{i4}]$

$y_i = [1 \quad 0 \quad 0]$

$\hat{y}_i = [f_c \quad f_d \quad f_b]$

$$g_c = w_{c1}x_{i1} + w_{c2}x_{i2} + w_{c3}x_{i3} + w_{c4}x_{i4} + b_c$$
$$g_d = w_{d1}x_{i1} + w_{d2}x_{i2} + w_{d3}x_{i3} + w_{d4}x_{i4} + b_d$$
$$g_b = w_{b1}x_{i1} + w_{b2}x_{i2} + w_{b3}x_{i3} + w_{b4}x_{i4} + b_b$$

$$f_c = e^{g_c}/(e^{g_c}+e^{g_d}+e^{g_b})$$
$$f_d = e^{g_d}/(e^{g_c}+e^{g_d}+e^{g_b})$$
$$f_b = e^{g_b}/(e^{g_c}+e^{g_d}+e^{g_b})$$

$$W = \begin{bmatrix} w_{c1} & w_{c2} & w_{c3} & w_{c4} \\ w_{d1} & w_{d2} & w_{d3} & w_{d4} \\ w_{b1} & w_{b2} & w_{b3} & w_{b4} \end{bmatrix}$$

$$b = [b_c \quad b_d \quad b_b]$$

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Linear Softmax

$x_i = [x_{i1} \quad x_{i2} \quad x_{i3} \quad x_{i4}]$

$y_i = [1 \quad 0 \quad 0]$

$\hat{y}_i = [f_c \quad f_d \quad f_b]$

$g = wx^T + b^T$

$$w = \begin{bmatrix} w_{c1} & w_{c2} & w_{c3} & w_{c4} \\ w_{d1} & w_{d2} & w_{d3} & w_{d4} \\ w_{b1} & w_{b2} & w_{b3} & w_{b4} \end{bmatrix}$$

$b = [b_c \quad b_d \quad b_b]$

$f_c = e^{g_c}/(e^{g_c}+e^{g_d}+e^{g_b})$

$f_d = e^{g_d}/(e^{g_c}+e^{g_d}+e^{g_b})$

$f_b = e^{g_b}/(e^{g_c}+e^{g_d}+e^{g_b})$

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Linear Softmax

$$x_i = [x_{i1} \quad x_{i2} \quad x_{i3} \quad x_{i4}]$$

$$y_i = [1 \quad 0 \quad 0]$$

$$\hat{y}_i = [f_c \quad f_d \quad f_b]$$

$$w = \begin{bmatrix} w_{c1} & w_{c2} & w_{c3} & w_{c4} \\ w_{d1} & w_{d2} & w_{d3} & w_{d4} \\ w_{b1} & w_{b2} & w_{b3} & w_{b4} \end{bmatrix}$$

$$b = [b_c \quad b_d \quad b_b]$$

learnable weight & bias

$$g = wx^T + b^T$$

$$f = softmax(g)$$

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Linear Softmax

$x_i = [x_{i1} \quad x_{i2} \quad x_{i3} \quad x_{i4}]$

$y_i = [1 \quad 0 \quad 0]$

$\hat{y}_i = [f_c \quad f_d \quad f_b]$

$$f = softmax(wx^T + b^T)$$ → one ip & one op layer

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Two-layer MLP + Softmax

$x_i = [x_{i1} \quad x_{i2} \quad x_{i3} \quad x_{i4}]$ $\quad y_i = [1 \quad 0 \quad 0]$ $\quad \hat{y}_i = [f_c \quad f_d \quad f_b]$

$$a_1 = sigmoid(w_{[1]}x^T + b^T_{[1]})$$
$$f = softmax(w_{[2]}a[1]^T + b^T_{[2]})$$

one i/p. one hidden. one o/p

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

N-layer MLP + Softmax

$x_i = [x_{i1} \quad x_{i2} \quad x_{i3} \quad x_{i4}]$ $\quad y_i = [1 \quad 0 \quad 0]$ $\quad \hat{y}_i = [f_c \quad f_d \quad f_b]$

$$a_1 = sigmoid(w_{[1]}x^T + b^T_{[1]})$$

$$a_2 = sigmoid(w_{[2]}a^T_1 + b^T_{[2]})$$

...

$$a_k = sigmoid(w_{[k]}a^T_{k-1} + b^T_{[k]})$$

...

$$f = softmax(w_{[n]}a^T_{n-1} + b^T_{[n]})$$

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

How to train the parameters?

$x_i = [x_{i1} \quad x_{i2} \quad x_{i3} \quad x_{i4}]$ $\quad y_i = [1 \quad 0 \quad 0]$ $\quad \hat{y}_i = [f_c \quad f_d \quad f_b]$

$$a_1 = sigmoid(w_{[1]}x^T + b^T_{[1]})$$

$$a_2 = sigmoid(w_{[2]}a^T_1 + b^T_{[2]})$$

...

$$a_k = sigmoid(w_{[k]}a^T_{k-1} + b^T_{[k]})$$

...

$$f = softmax(w_{[n]}a^T_{n-1} + b^T_{[n]})$$

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

How to train the parameters?

$x_i = [x_{i1} \quad x_{i2} \quad x_{i3} \quad x_{i4}]$

$y_i = [1 \quad 0 \quad 0]$

$\hat{y}_i = [f_c \quad f_d \quad f_b]$

$$a_1 = sigmoid(w_{[1]}x^T + b^T_{[1]})$$

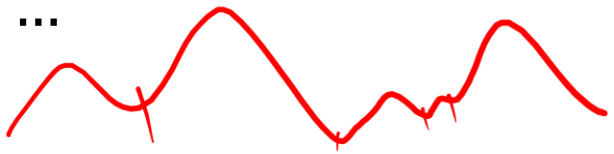
$$a_2 = sigmoid(w_{[2]}a^T_1 + b^T_{[2]})$$

...

$$a_k = sigmoid(w_{[k]}a^T_{k-1} + b^T_{[i]})$$

...

$$f = softmax(w_{[n]}a^T_{n-1} + b^T_{[n]})$$



what are the optimum w.b so that my loss function over all the training samples is minimized

$l = loss(f, y)$

$\min \sum_{i=1}^{N} l$

w, b

We can still use SGD

cross entropy

-> convex function

We need!

$\frac{\partial l}{\partial w_{[k]ij}}$ $\frac{\partial l}{\partial b_{[k]i}}$

back propagation

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

f_c f_b f_b

y_i: [1 0 0]

[y_c y_b y_D]

x_i: [x_1^i, x_2^i, x_3^i, x_4^i]

$$L_{CE} = \frac{1}{N}\sum_{i=1}^{N} - \underbrace{y_c \log f_c}_{0} - \underbrace{y_b \log f_b}_{0} - \underbrace{y_D \log f_D}_{0}$$

$$\sum_{i=1}^{3} - y_i \log f_i$$

→ class where the data is actually present

→ minimum value possible is zero

→ which means $f_c = 1$. $\log f_c = 0$.

$f_c = 1$. $f_b = 0$ $f_D = 0$

[1 0 0]

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

- Regression:
  - Use the same objective as Linear Regression
  - Quadratic loss (i.e. mean squared error)
- Classification:
  - Use the same objective as Logistic Regression
  - Cross-entropy (i.e. negative log likelihood)
  - This requires probabilities, so we add an additional "softmax" layer at the end of our network

	Forward	Backward
Quadratic	$J = \frac{1}{2}(y - y^*)^2$	$\frac{dJ}{dy} = y - y^*$
Cross Entropy	$J = y^* \log(y) + (1 - y^*) \log(1 - y)$	$\frac{dJ}{dy} = y^* \frac{1}{y} + (1 - y^*) \frac{1}{y - 1}$

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

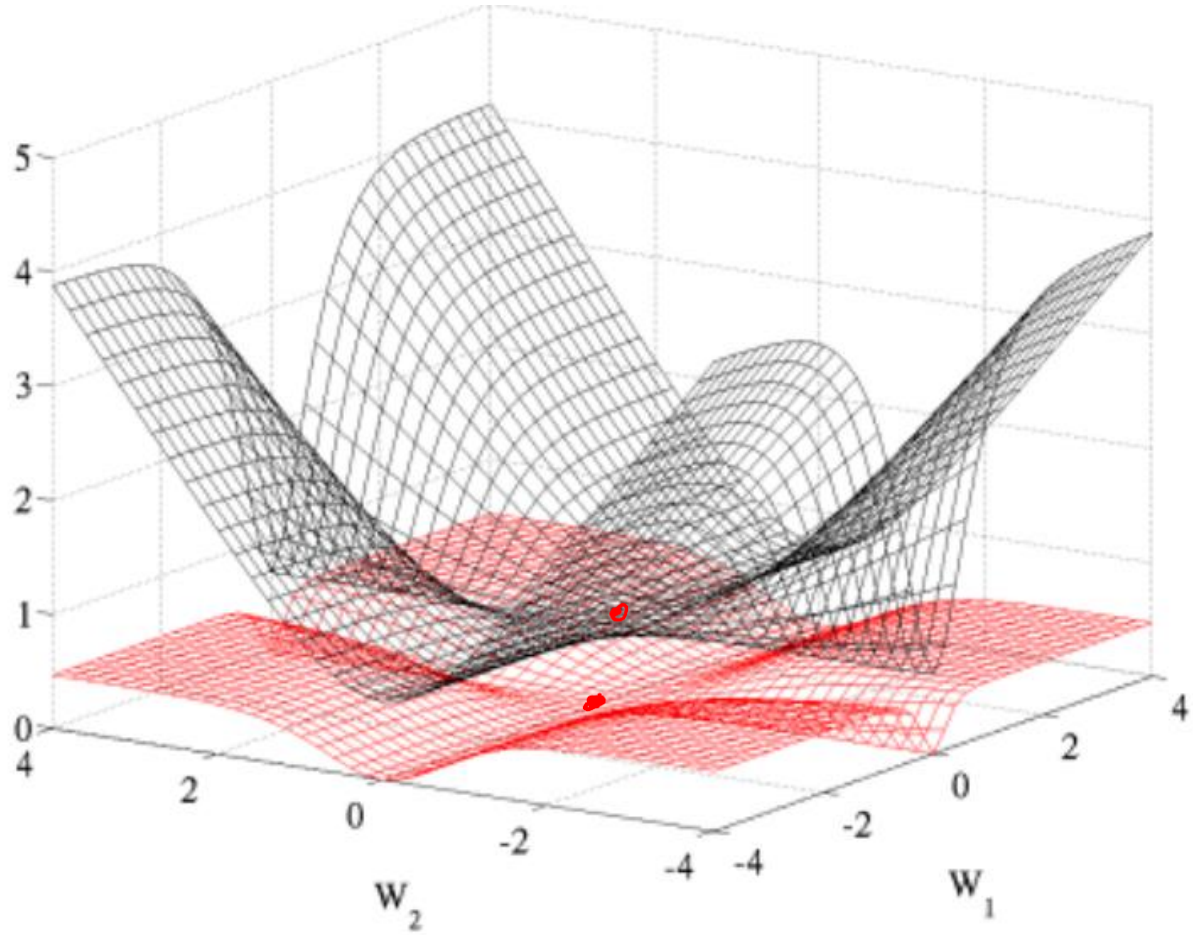


Figure 5: *Cross entropy (black, surface on top) and quadratic (red, bottom surface) cost as a function of two weights (one at each layer) of a network with two layers, W_1 respectively on the first layer and W_2 on the second, output layer.*

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Derivation of MLE to cross-entropy

Here $y_i \in \{0, 1\}$. Model gives

$$p_\theta(y = 1 \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

z=wx+b

It follows a Bernoulli distribution

Then $p_\theta(y = 0 \mid x) = 1 - \sigma(z)$.

Likelihood for one point:

$$p_\theta(y \mid x) = \sigma(z)^y \left(1 - \sigma(z)\right)^{1-y}.$$

Log-likelihood:

$$\log p_\theta(y \mid x) = y \log \sigma(z) + (1 - y) \log(1 - \sigma(z)).$$

Negative log-likelihood:

$$-\log p_\theta(y \mid x) = -\left[y \log \sigma(z) + (1 - y) \log(1 - \sigma(z))\right],$$

which is the **binary cross-entropy**.

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

For regression – MLE to MSE

Assume:

$$y_i = f_\theta(x_i) + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2)$$

(i.i.d. Gaussian noise).

That means the conditional likelihood is:

$$p_\theta(y_i \mid x_i) = \mathcal{N}\big(y_i;\, f_\theta(x_i),\, \sigma^2\big) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - f_\theta(x_i))^2}{2\sigma^2}\right).$$

Likelihood of whole dataset:

$$\mathcal{L}(\theta) = \prod_{i=1}^{N} p_\theta(y_i \mid x_i).$$

Log-likelihood:

$$\log \mathcal{L}(\theta) = \sum_{i=1}^{N} \left[-\frac{1}{2}\log(2\pi\sigma^2) - \frac{(y_i - f_\theta(x_i))^2}{2\sigma^2}\right].$$

Negative log-likelihood (NLL):

$$-\log \mathcal{L}(\theta) = \sum_{i=1}^{N} \left[\frac{1}{2}\log(2\pi\sigma^2) + \frac{(y_i - f_\theta(x_i))^2}{2\sigma^2}\right].$$

If σ^2 is treated as a constant, the first term is a constant w.r.t. θ, so minimizing NLL is equivalent to minimizing:

$$\sum_{i=1}^{N} (y_i - f_\theta(x_i))^2.$$

That is exactly the **sum of squared errors**, and dividing by N gives **MSE**:

$$\boxed{\text{MSE} = \frac{1}{N}\sum_{i=1}^{N} (y_i - f_\theta(x_i))^2.}$$

This file is meant for personal use by meravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Background

A Recipe for Machine Learning

1. Given training data:

$$\{\boldsymbol{x}_i, \boldsymbol{y}_i\}_{i=1}^{N}$$

2. Choose each of these:

- Decision function

$$\hat{\boldsymbol{y}} = f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)$$

- Loss function

$$\ell(\hat{\boldsymbol{y}}, \boldsymbol{y}_i) \in \mathbb{R}$$

3. Define goal:

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{N} \ell(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i), \boldsymbol{y}_i)$$

4. Train with SGD:

(take small steps opposite the gradient)

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta_t \nabla \ell(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i), \boldsymbol{y}_i)$$

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

L^0_{CE} at w^0

$$w^1 = w^0 - \eta \frac{\partial L_{CE}}{\partial w_0}$$

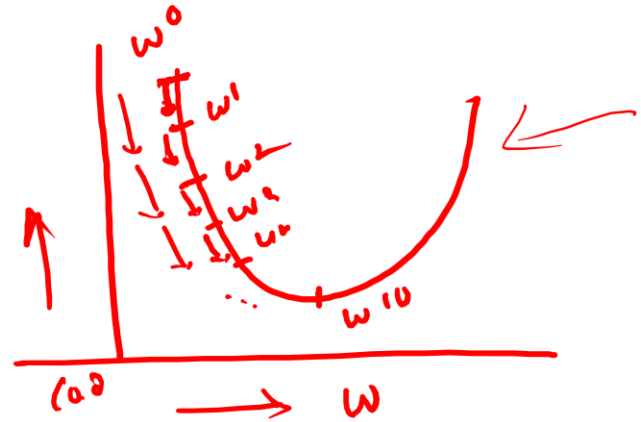
learning rate

$$w^2 = w^1 - \eta \frac{\partial L_{CE}}{\partial w_1}$$

w^{10}

Convergence criteria

1. $L^t_{CE} \approx 0$ STOP
2. if t is large $t = 1000$ STOP
3. $|L^t_{CE} - L^{t-1}_{CE}| < \epsilon$ STOP
4. early stopping



L^1_{CE} at w^1

L^2_{CE} at w^2

L^{10}_{CE} at $w^{10} \approx 0$

w^{10} → optimal weight value

trained model

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

$X = \{(x_i, y_i)\}_{i=1}^{N}$ $\quad x_i \in \mathbb{R}^D$. $y \rightarrow$ class labels C

Input layer – D nodes

Hidden layer → D_1 number of nodes

$h_1 = \sigma(W_1 x + b_1)$ (W_1, b_1) are weight & biases betw i/p & hidden layers

output layer → C number of nodes

$O_1 = s/m(h_1 W_2 + b_2)$ (W_2, b_2) weight & bias between hidden & o/p.

f_Θ

$\Theta = (W_1, W_2, b_1, b_2)$

Loss fn : $L = \frac{1}{N} \sum_{i=1}^{N} CE(f(x_i), y_i)$

Our goal

$\min_{\Theta} L$ → backpropagation

BP

Loop → ① Initialize Θ (may be randomly) Θ^0

② Forward propagate x to get $f(x)$

③ Calculate $\frac{\partial L}{\partial \Theta^t}$ $\left(\frac{\partial L}{\partial W_1^t}, \frac{\partial L}{\partial W_2^t}, \frac{\partial L}{\partial b_1^t}, \frac{\partial L}{\partial b_2^t}\right)$

④ update

$$W_1^{t+1} = W_1^t - \eta_t \frac{\partial L}{\partial W_1^t}$$

learning rate

Iterative process

G.D. Batch SGD MB GD

This file is meant for personal use by deepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Computation graph representation of MLPs

- A computation graph is the “unrolled” representation of a neural network’s forward computation as a sequence of simple operations. Backpropagation is exactly the procedure for computing gradients on this graph efficiently.
- Because gradients can be computed by the chain rule, and the chain rule naturally follows the graph’s dependencies.

What a computation graph contains

Think of a neural net forward pass as a program. The computation graph is the same program, but written as a graph:

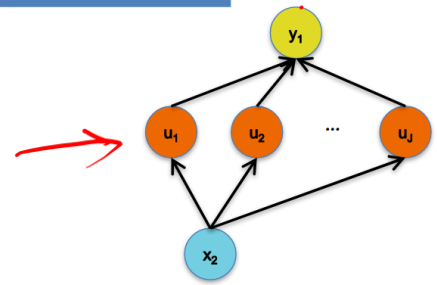
- **Value nodes (tensors):** inputs x, parameters W, b, intermediate activations $z, h, \hat{y}$, loss L.
- **Operation nodes:** matmul, add, ReLU, sigmoid, softmax, log, sum, etc.
- **Directed edges:** show dependencies (which values are used to compute which other values).

It’s usually a **DAG** (directed acyclic graph) for feedforward nets, because data flows forward without loops.

Given: $\boldsymbol{y} = g(\boldsymbol{u})$ and $\boldsymbol{u} = h(\boldsymbol{x})$.

Chain Rule:

$$\frac{dy_i}{dx_k} = \sum_{j=1}^{J} \frac{dy_i}{du_j} \frac{du_j}{dx_k}, \quad \forall i, k$$



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

A 2-layer MLP graph

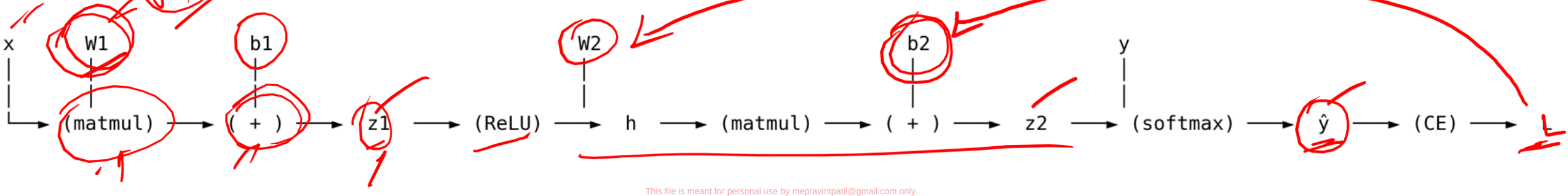
Forward definitions:

$$z_1 = W_1 x + b_1, \quad h = \phi(z_1), \quad z_2 = W_2 h + b_2, \quad \hat{y} = \mathrm{softmax}(z_2), \quad L = -\log \hat{y}_y$$

Graph dependencies (text form):

- $x, W_1, b_1 \to z_1$
- $z_1 \to h$
- $h, W_2, b_2 \to z_2$
- $z_2 \to \hat{y}$
- $\hat{y}, y \to L$

During the **forward pass**, you compute these nodes in order and **store** intermediates like $z_1, h, z_2, \hat{y}$.



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

The backward propagation

```
Start:
δ2 = dL/dz2 = ŷ - y_onehot

Linear2: z2 = W2 h + b2
dW2 = δ2 h^T
db2 = δ2
dh  = W2^T δ2

Activation: h = ϕ(z1)
δ1 = dz1 = dh ⊙ ϕ'(z1)

Linear1: z1 = W1 x + b1
dW1 = δ1 x^T
db1 = δ1
dx  = W1^T δ1
```

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Static vs Dynamic CG

Aspect	Static graph	Dynamic graph
When graph is built	Before execution	During execution
Debugging	Harder (graph is separate)	Easier (print/step through)
Python control flow	Needs graph-control ops	Works naturally
Optimization potential	Strong (whole graph known)	Needs tracing/compiling for best speed
Export/deploy	Often straightforward	Usually needs tracing/script/compile step

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Given $f : \mathbb{R}^A \rightarrow \mathbb{R}^B, f(\mathbf{x})$

Compute $\frac{\partial f(\mathbf{x})_i}{\partial x_j} \forall i, j$

1. Finite Difference Method
   - Pro: Great for testing implementations of backpropagation
   - Con: Slow for high dimensional inputs / outputs
   - Required: Ability to call the function f(**x**) on any input **x**
2. Symbolic Differentiation
   - Note: The method you learned in high-school
   - Note: Used by Mathematica / Wolfram Alpha / Maple
   - Pro: Yields easily interpretable derivatives
   - Con: Leads to exponential computation time if not carefully implemented
   - Required: Mathematical expression that defines f(**x**)
3. Automatic Differentiation - Reverse Mode
   - Note: Called *Backpropagation* when applied to Neural Nets
   - Pro: Computes partial derivatives of one output $f(\mathbf{x})_i$ with respect to all inputs x_j in time proportional to computation of f(**x**)
   - Con: Slow for high dimensional outputs (e.g. vector-valued functions)
   - Required: Algorithm for computing f(**x**)
4. Automatic Differentiation - Forward Mode
   - Note: Easy to implement. Uses dual numbers.
   - Pro: Computes partial derivatives of all outputs $f(\mathbf{x})_i$ with respect to one input x_j in time proportional to computation of f(**x**)
   - Con: Slow for high dimensional inputs (e.g. vector-valued **x**)
   - Required: Algorithm for computing f(**x**)

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Finite and symbolic differentiation

For each parameter θ_j:

1. run the graph with $\theta \rightarrow$ get $L(\theta)$
2. slightly perturb that one parameter: $\theta_j \leftarrow \theta_j + \varepsilon$
3. run graph again $\rightarrow$ get $L(\theta + \varepsilon e_j)$
4. approximate slope:

$$\frac{\partial L}{\partial \theta_j} \approx \frac{L(\theta + \varepsilon e_j) - L(\theta)}{\varepsilon}$$

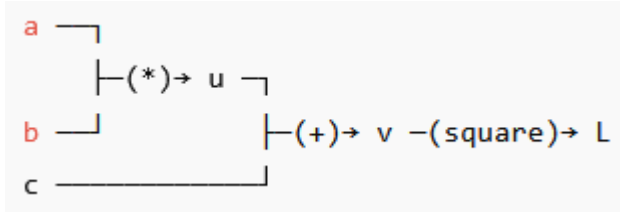
You take the computation graph and try to convert it into a **single algebraic expression**, then differentiate it *symbolically*.

- You repeatedly apply algebraic rules (product rule, chain rule, etc.)
- You generate a new symbolic expression for $\frac{\partial L}{\partial \theta_j}$

Graph-wise, this means: "expand" the whole graph into formulas.

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Automatic differentiation – forward mode



Step 1: Decide which input you care about

Say we want $\frac{\partial L}{\partial a}$.

So we start with:

$$a_a = \frac{\partial a}{\partial a} = 1, \quad b_a = \frac{\partial b}{\partial a} = 0, \quad c_a = \frac{\partial c}{\partial a} = 0$$

Because b and c do not change when a changes.

"How input affects things" forward

Step 2: Move forward and compute derivatives at each node

Node 1: $u = ab$

$$u_a = \frac{\partial(ab)}{\partial a} = b$$

Node 2: $v = u + c$

$$v_a = \frac{\partial(u+c)}{\partial a} = u_a + 0 = b$$

Node 3: $L = v^2$

$$L_a = \frac{\partial(v^2)}{\partial a} = 2v \cdot v_a = 2(ab+c) \cdot b$$

So:

$$\boxed{\frac{\partial L}{\partial a} = 2(ab+c)b}$$

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Automatic differentiation – reverse mode

Step 1: Start at the end

$$\frac{\partial L}{\partial L} = 1$$

From $L = v^2$:

$$L_v = \frac{\partial L}{\partial v} = 2v$$

Step 2: Move backward using chain rule

From $v = u + c$:

$$L_u = L_v \cdot \frac{\partial v}{\partial u} = (2v) \cdot 1 = 2v$$

$$L_c = L_v \cdot \frac{\partial v}{\partial c} = (2v) \cdot 1 = 2v$$

From $u = ab$:

$$L_a = L_u \cdot \frac{\partial u}{\partial a} = (2v) \cdot b$$

$$L_b = L_u \cdot \frac{\partial u}{\partial b} = (2v) \cdot a$$

So:

$$\boxed{\frac{\partial L}{\partial a} = 2vb, \quad \frac{\partial L}{\partial b} = 2va, \quad \frac{\partial L}{\partial c} = 2v}$$

and since $v = ab + c$, this matches.

“How does the loss depend on each intermediate variable?”

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Reverse mode is better for deep nets

Neural nets have:

- **one loss L** (a scalar)
- **millions of parameters θ**

Reverse-mode computes $\nabla_\theta L$ efficiently in one backward pass.

Forward-mode would need (roughly) one pass **per parameter** → impossible.

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Forward Computation

1. Write an **algorithm** for evaluating the function y = f(**x**). The algorithm defines a **directed acyclic graph,** where each variable is a node (i.e. the "**computation graph**")
2. Visit each node in **topological order**.
   For variable u_i with inputs $v_1,\ldots, v_N$
   a. Compute $u_i = g_i(v_1,\ldots, v_N)$
   b. Store the result at the node

Backward Computation

1. **Initialize** all partial derivatives dy/du_j to 0 and $dy/dy = 1$.
2. Visit each node in **reverse topological order**.
   For variable $u_i = g_i(v_1,\ldots, v_N)$
   a. We already know dy/du_i
   b. Increment dy/dv_j by $(dy/du_i)(du_i/dv_j)$
      (Choice of algorithm ensures computing (du_i/dv_j) is easy)

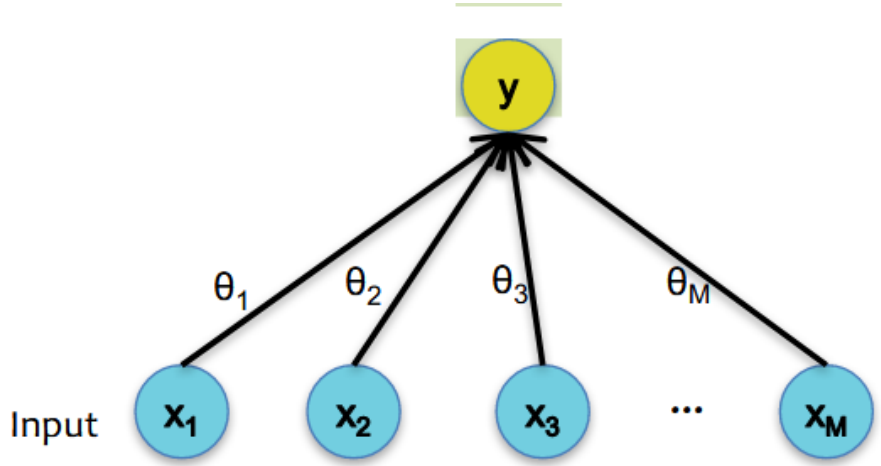
This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Simple Example: The goal is to compute $J = \cos(\sin(x^2) + 3x^2)$ on the forward pass and the derivative $\frac{dJ}{dx}$ on the backward pass.

Forward	Backward
$J = cos(u)$	$\frac{dJ}{du}$ += $-sin(u)$
$u = u_1 + u_2$	$\frac{dJ}{du_1}$ += $\frac{dJ}{du}\frac{du}{du_1}$, $\frac{du}{du_1} = 1$ $\quad\quad$ $\frac{dJ}{du_2}$ += $\frac{dJ}{du}\frac{du}{du_2}$, $\frac{du}{du_2} = 1$
$u_1 = sin(t)$	$\frac{dJ}{dt}$ += $\frac{dJ}{du_1}\frac{du_1}{dt}$, $\frac{du_1}{dt} = \cos(t)$
$u_2 = 3t$	$\frac{dJ}{dt}$ += $\frac{dJ}{du_2}\frac{du_2}{dt}$, $\frac{du_2}{dt} = 3$
$t = x^2$	$\frac{dJ}{dx}$ += $\frac{dJ}{dt}\frac{dt}{dx}$, $\frac{dt}{dx} = 2x$

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Logistic Regression



Forward

$$J = y^* \log y + (1 - y^*) \log(1 - y)$$

$$y = \frac{1}{1 + \exp(-a)}$$

$$a = \sum_{j=0}^{D} \theta_j x_j$$

Backward

$$\frac{dJ}{dy} = \frac{y^*}{y} + \frac{(1 - y^*)}{y - 1}$$

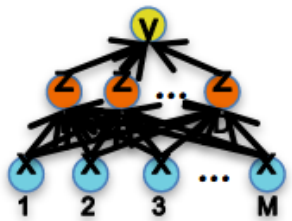
$$\frac{dJ}{da} = \frac{dJ}{dy}\frac{dy}{da}, \frac{dy}{da} = \frac{\exp(-a)}{(\exp(-a) + 1)^2}$$

$$\frac{dJ}{d\theta_j} = \frac{dJ}{da}\frac{da}{d\theta_j}, \frac{da}{d\theta_j} = x_j$$

$$\frac{dJ}{dx_j} = \frac{dJ}{da}\frac{da}{dx_j}, \frac{da}{dx_j} = \theta_j$$

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Neural Network



Forward

$$J = y^* \log y + (1 - y^*) \log(1 - y)$$

$$y = \frac{1}{1 + \exp(-b)}$$

$$b = \sum_{j=0}^{D} \beta_j z_j$$

$$z_j = \frac{1}{1 + \exp(-a_j)}$$

$$a_j = \sum_{i=0}^{M} \alpha_{ji} x_i$$

Backward

$$\frac{dJ}{dy} = \frac{y^*}{y} + \frac{(1 - y^*)}{y - 1}$$

$$\frac{dJ}{db} = \frac{dJ}{dy}\frac{dy}{db}, \frac{dy}{db} = \frac{\exp(-b)}{(\exp(-b) + 1)^2}$$

$$\frac{dJ}{d\beta_j} = \frac{dJ}{db}\frac{db}{d\beta_j}, \frac{db}{d\beta_j} = z_j$$

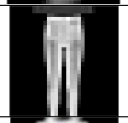
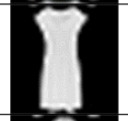
$$\frac{dJ}{dz_j} = \frac{dJ}{db}\frac{db}{dz_j}, \frac{db}{dz_j} = \beta_j$$

$$\frac{dJ}{da_j} = \frac{dJ}{dz_j}\frac{dz_j}{da_j}, \frac{dz_j}{da_j} = \frac{\exp(-a_j)}{(\exp(-a_j) + 1)^2}$$

$$\frac{dJ}{d\alpha_{ji}} = \frac{dJ}{da_j}\frac{da_j}{d\alpha_{ji}}, \frac{da_j}{d\alpha_{ji}} = x_i$$

$$\frac{dJ}{dx_i} = \frac{dJ}{da_j}\frac{da_j}{dx_i}, \frac{da_j}{dx_i} = \sum_{j=0}^{D} \alpha_{ji}$$

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Label	Description	Example
0	T-shirt/top	
1	Trouser	
2	Pullover	
3	Dress	
4	Coat	
5	Sandal	
6	Shirt	
7	Sneaker	
8	Bag	
9	Ankle boot	

Normalized CNN Confusion Matrix

	T-shirt/top	Trouser	Pullover	Dress	Coat	Sandal	Shirt	Sneaker	Bag	Ankle boot
T-shirt/top	0.83	0.00	0.01	0.03	0.00	0.00	0.12	0.00	0.00	0.00
Trouser	0.00	0.99	0.00	0.01	0.00	0.00	0.00	0.00	0.00	0.00
Pullover	0.01	0.00	0.85	0.01	0.07	0.00	0.07	0.00	0.00	0.00
Dress	0.01	0.00	0.00	0.93	0.03	0.00	0.02	0.00	0.00	0.00
Coat	0.00	0.00	0.05	0.03	0.86	0.00	0.06	0.00	0.00	0.00
Sandal	0.00	0.00	0.00	0.00	0.00	0.98	0.00	0.01	0.00	0.00
Shirt	0.08	0.00	0.05	0.02	0.07	0.00	0.78	0.00	0.00	0.00
Sneaker	0.00	0.00	0.00	0.00	0.00	0.01	0.00	0.97	0.00	0.02
Bag	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.99	0.00
Ankle boot	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.03	0.00	0.96

Normalized MLP Confusion Matrix

	T-shirt/top	Trouser	Pullover	Dress	Coat	Sandal	Shirt	Sneaker	Bag	Ankle boot
T-shirt/top	0.84	0.00	0.01	0.03	0.00	0.00	0.11	0.00	0.01	0.00
Trouser	0.00	0.98	0.00	0.02	0.00	0.00	0.00	0.00	0.00	0.00
Pullover	0.01	0.00	0.80	0.01	0.11	0.00	0.07	0.00	0.00	0.00
Dress	0.02	0.01	0.01	0.91	0.03	0.00	0.02	0.00	0.00	0.00
Coat	0.00	0.00	0.09	0.04	0.80	0.00	0.06	0.00	0.00	0.00
Sandal	0.00	0.00	0.00	0.00	0.00	0.97	0.00	0.02	0.00	0.01
Shirt	0.13	0.00	0.08	0.03	0.07	0.00	0.68	0.00	0.01	0.00
Sneaker	0.00	0.00	0.00	0.00	0.00	0.04	0.00	0.94	0.00	0.03
Bag	0.00	0.00	0.00	0.00	0.00	0.00	0.01	0.00	0.98	0.00
Ankle boot	0.00	0.00	0.00	0.00	0.00	0.03	0.00	0.04	0.00	0.92

	CNN	MLP
Accuracy	0.915	0.880
Cohen's Kappa	0.906	0.868
MCC	0.906	0.869
Cross-Entropy	0.227	0.324

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Invariance

- A function f[x] is invariant to a transformation t[] if:

$$\mathbf{f}\left[\mathbf{t}\left[\mathbf{x}\right]\right] = \mathbf{f}\left[\mathbf{x}\right]$$

i.e., the function output is the same even after the transformation is applied.

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Invariance example

e.g., Image classification

- Image has been translated, but we want our classifier to give the same result

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Equivariance

- A function f[x] is equivariant to a transformation t[] if:

$$\mathbf{f}\left[\mathbf{t}[\mathbf{x}]\right] = \mathbf{t}\left[\mathbf{f}[\mathbf{x}]\right]$$

i.e., the output is transformed in the same way as the input

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Equivariance example

e.g., Image segmentation

- Image has been translated and we want segmentation to translate with it




Sharing or publishing the contents in part or full is liable for legal action.

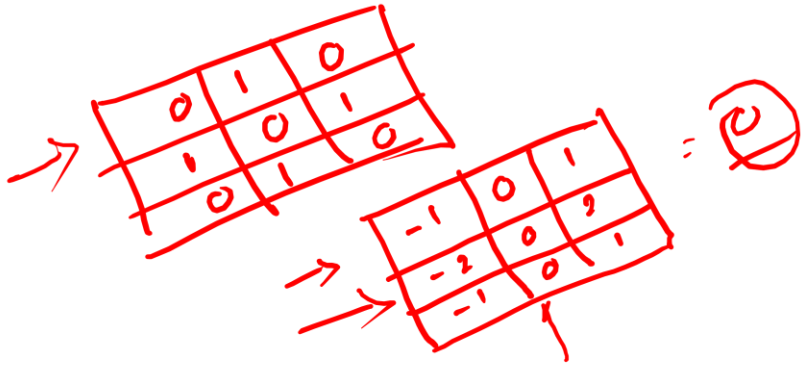
Basic building blocks of the CNN architecture

- Input layer
- Convolutional layer
- Fully connected layer
- Loss layer

- Convolutional layer
  - Convolutional kernel
  - Pooling layer
  - Non-linearity

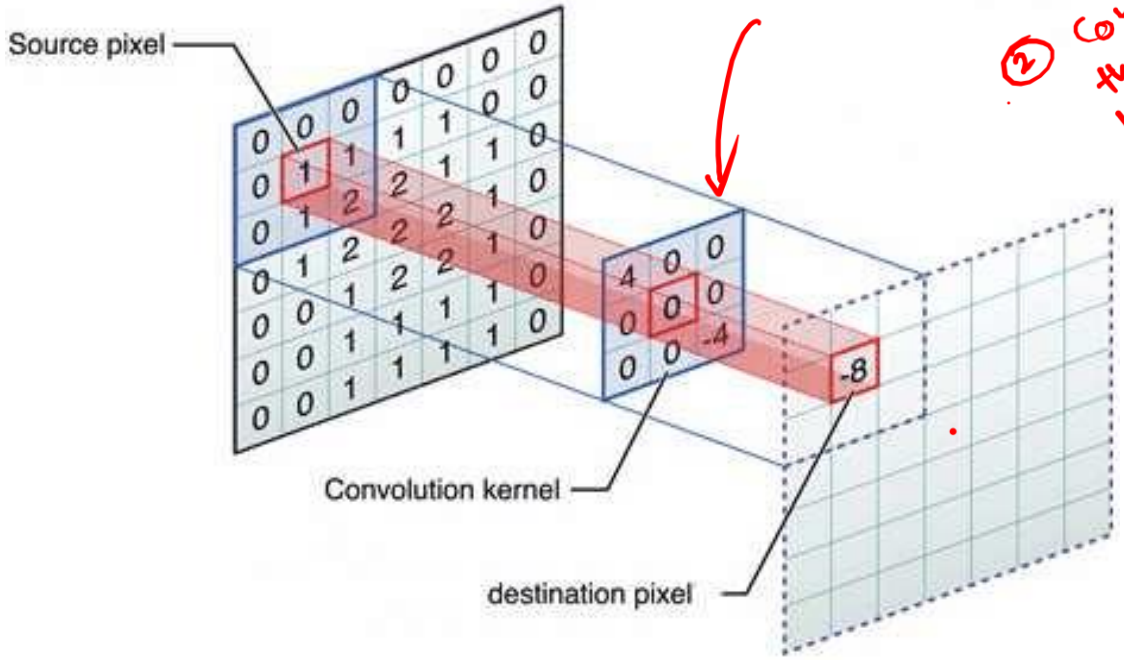
This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

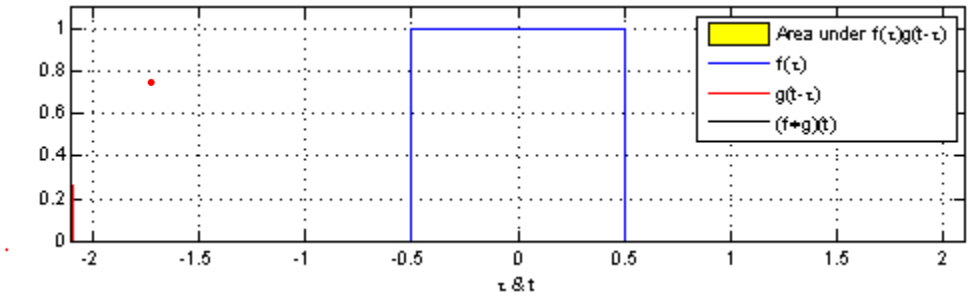
Convolution operation



① learn meaningful kernels

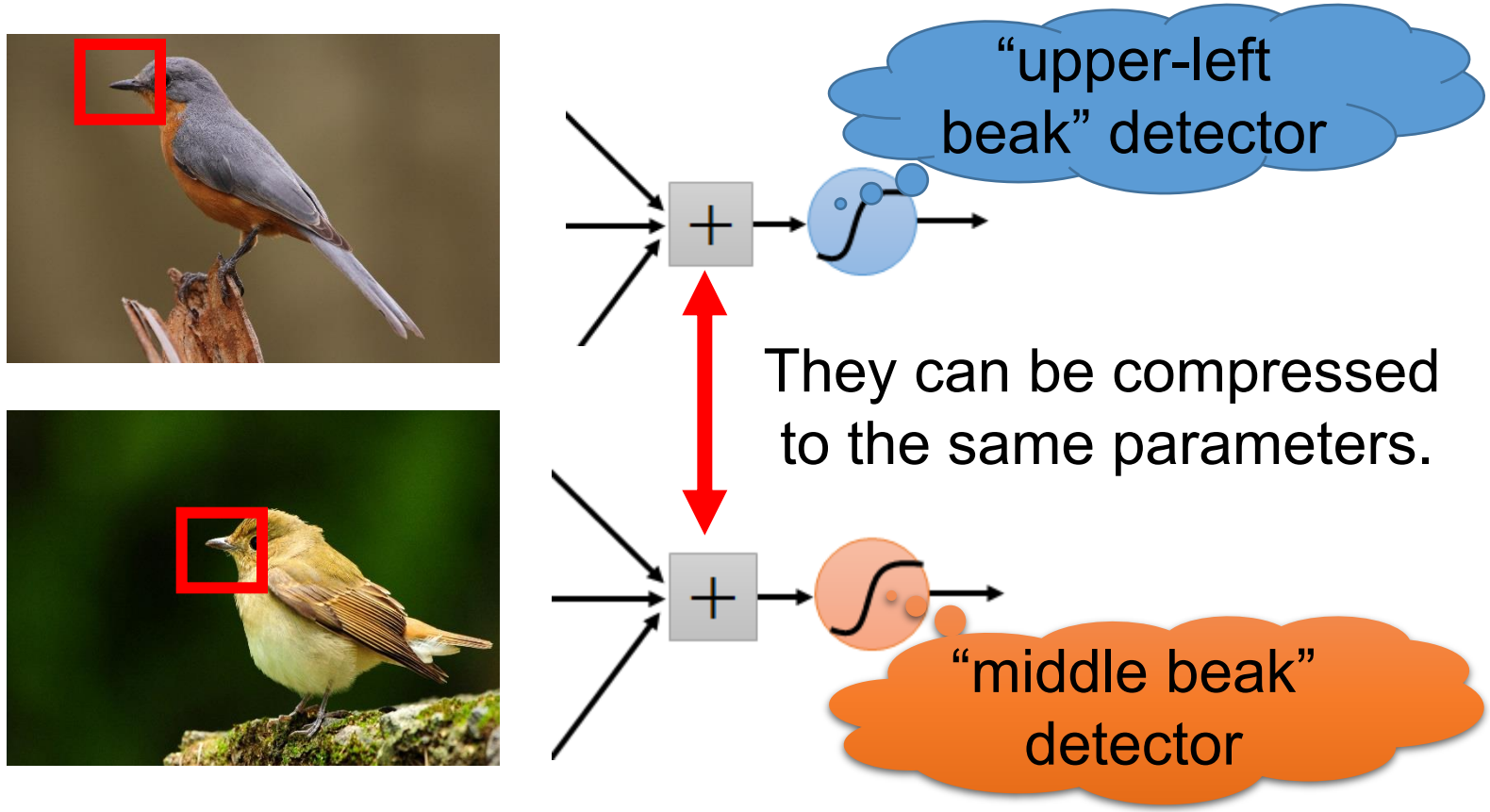
② Convolve with the image to highlight those features in the image





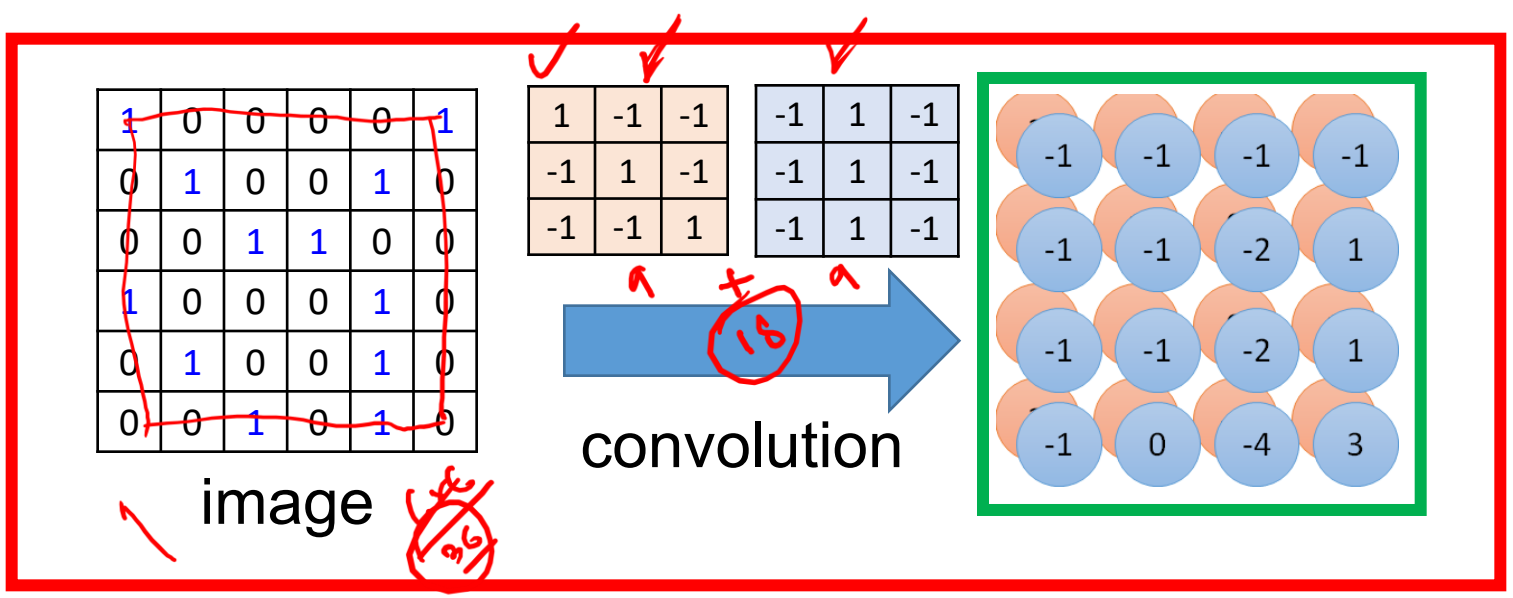
This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Same pattern appears in different places:
They can be compressed!
What about training a lot of such “small” detectors
and each detector must “move around”.

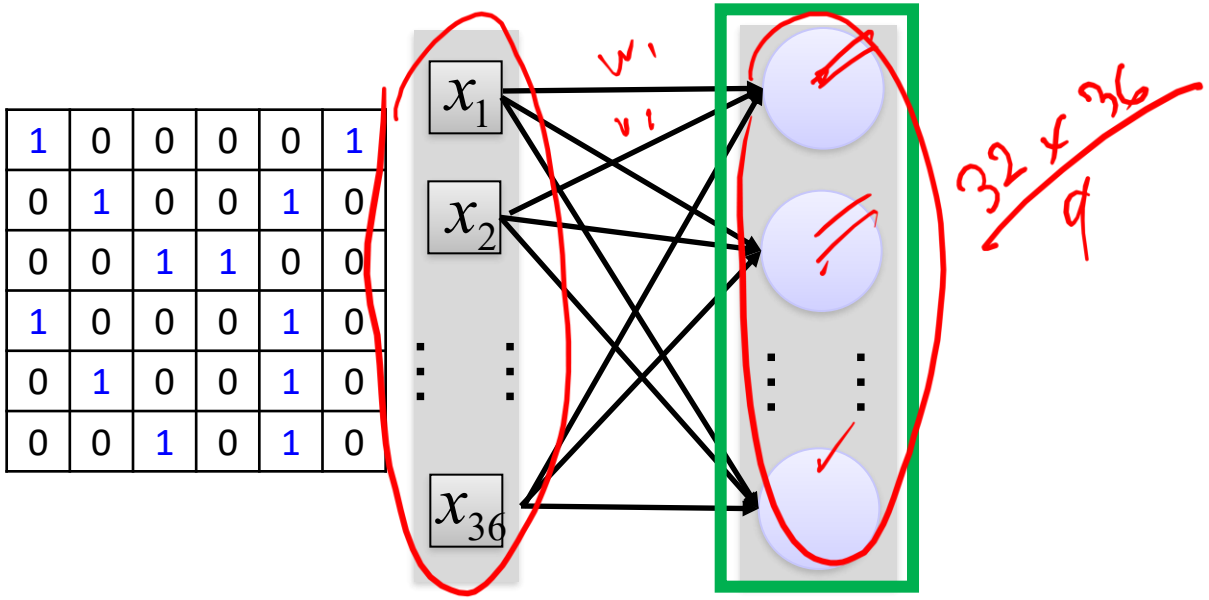


This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

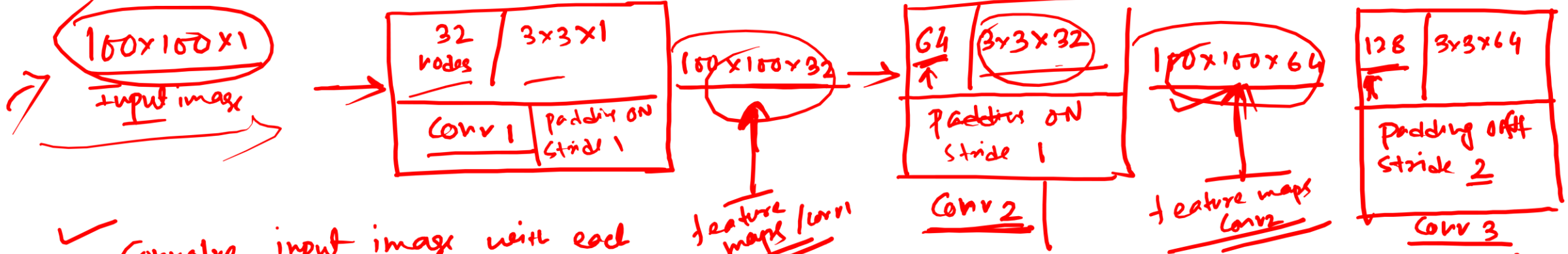
Convolution v.s. Fully Connected



Fully-connected



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.



Convolve input image with each 3x3 kernel (Stride = 1, padding ON)

output size = 100x100x1

each kernel is learning some specific feature for the image

Conv 1 :- 9 numbers/weights per kernel

32 x 9 → # weights + 32 biases

Conv 2 :- 64x9x32 → weights

64 biases

Conv 3 :- 128 x 3x3x64 weights

128 biases

Convolve (100x100x32) with (3x3x32)

the output size

= 100x100x1

100x100x32 ⊙ 3x3x32 → 100x100x1 ... 100x100x1

tell me the output shape of 'Conv3'

48x48x128

100x100x64 / 3x3x64 → 48x48x1

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Convolution* in 1D

- Input vector **x:**

$$\mathbf{x} = [x_1, x_2, \ldots, x_I]$$

- Output is weighted sum of neighbors:

$$z_i = \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1}$$

- Convolutional kernel or filter:

$$\boldsymbol{\omega} = [\omega_1, \omega_2, \omega_3]^T$$

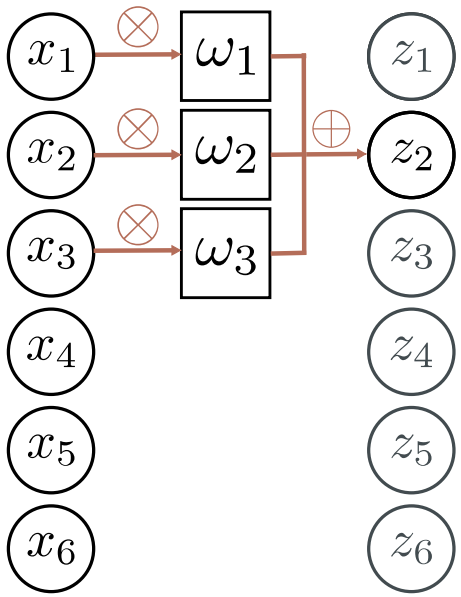
Kernel size = 3

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

* Not really technically convolution

Convolution with kernel size 3

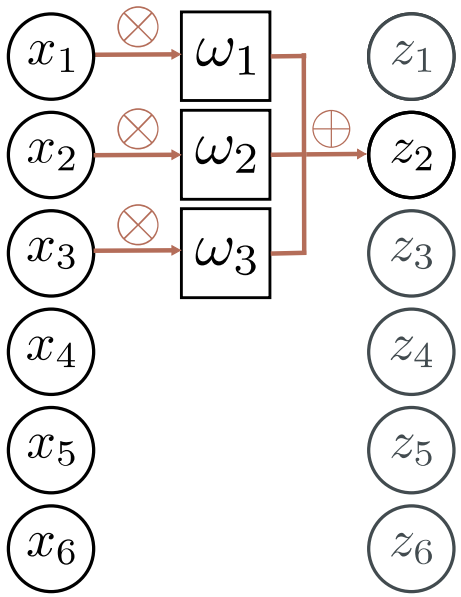
a)



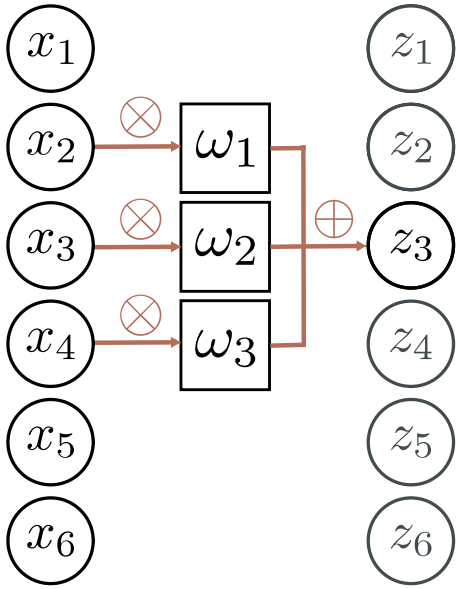
This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Convolution with kernel size 3

a)

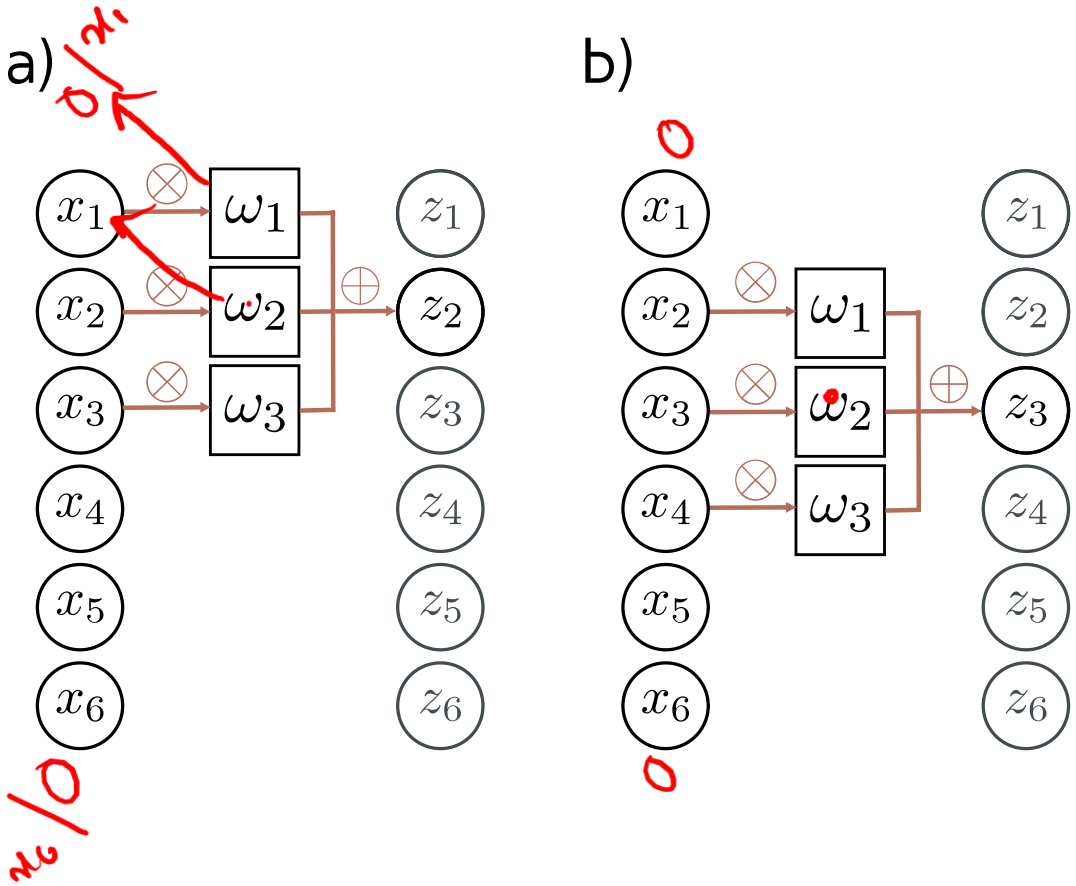


b)



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Convolution with kernel size 3

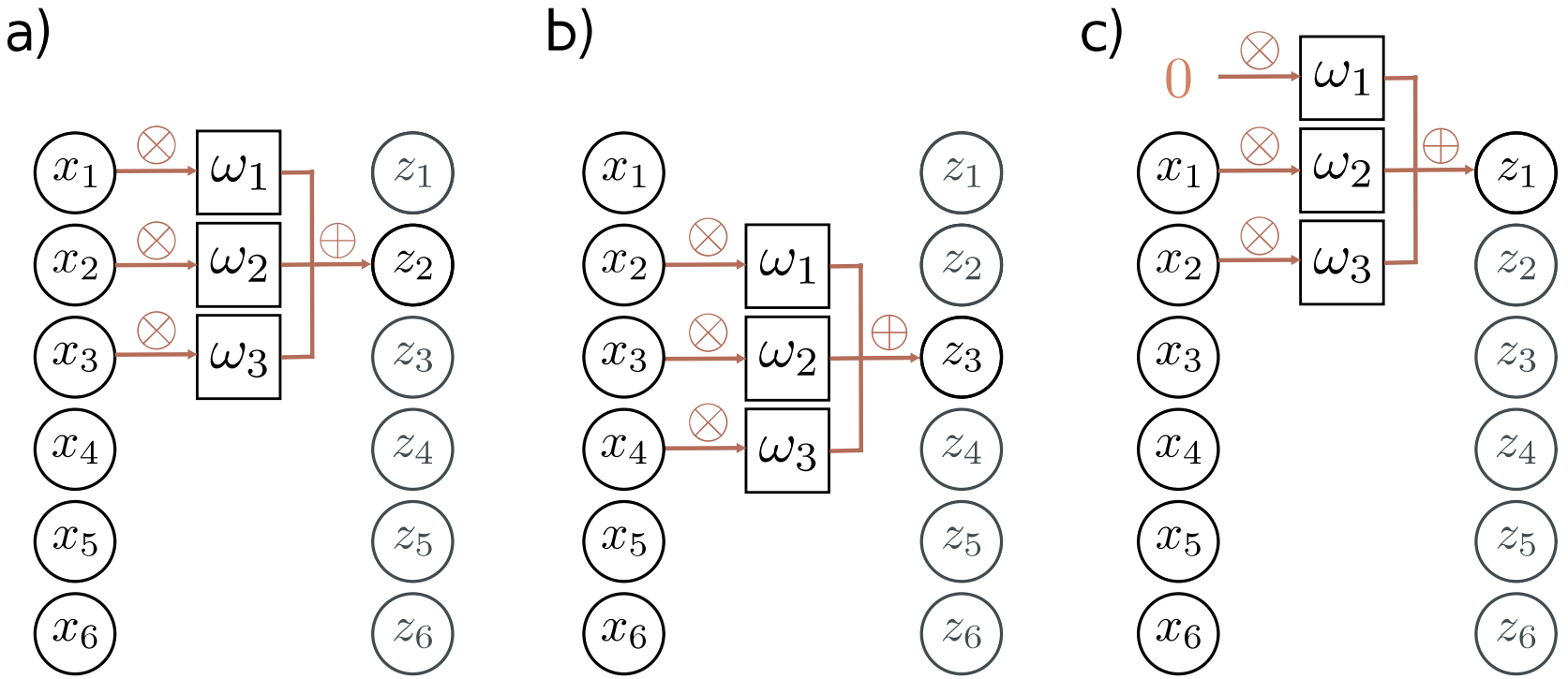


Equivariant to translation of input

$$\mathbf{f}[\mathbf{t}[\mathbf{x}]] = \mathbf{t}[\mathbf{f}[\mathbf{x}]]$$

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

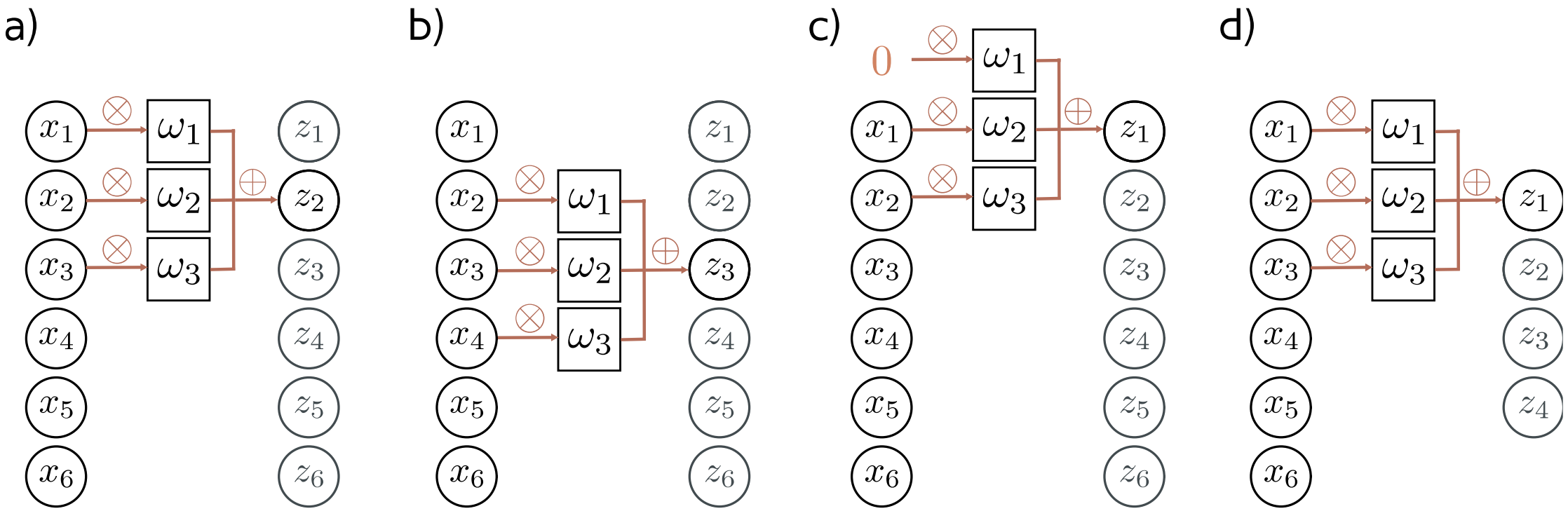
Zero padding



Treat positions that are beyond end of the input as zero.

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

“Valid” convolutions



Only process positions where kernel falls in image (smaller output).

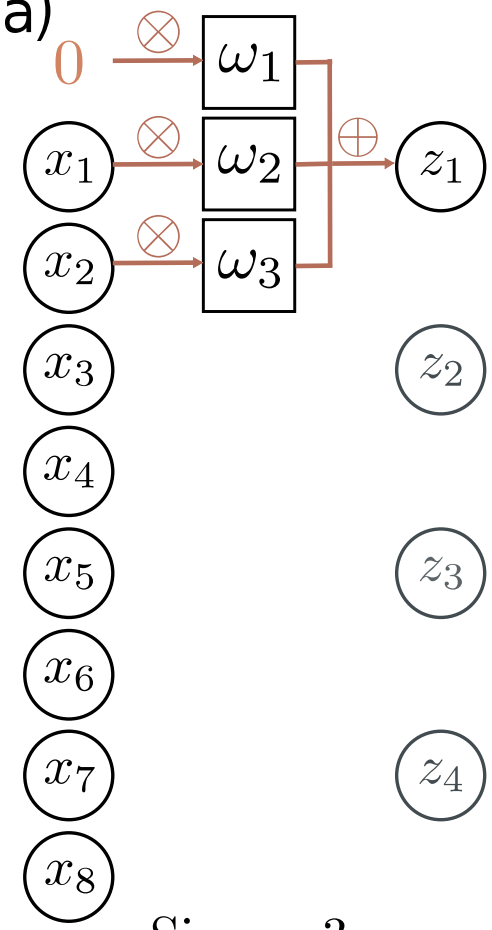
This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Stride, kernel size, and dilation

- Stride = shift by k positions for each output
  - Decreases size of output relative to input
- Kernel size = weight a different number of inputs for each output
  - Combine information from a larger area
  - But kernel size 5 uses 5 parameters
- Dilated or atrous convolutions = intersperse kernel values with zeros
  - Combine information from a larger area
  - Fewer parameters

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

a)



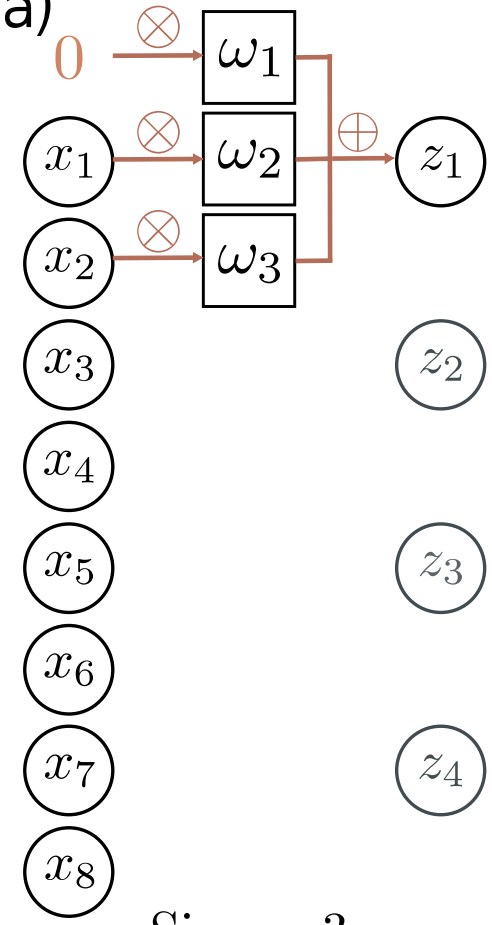
$\text{Size} = 3$

$\text{Stride} = 2$

$\text{Dilation} = 1$

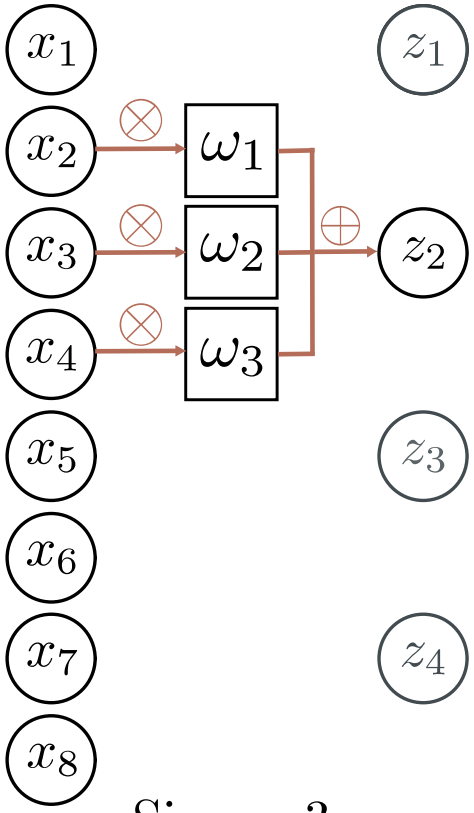
This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

a)



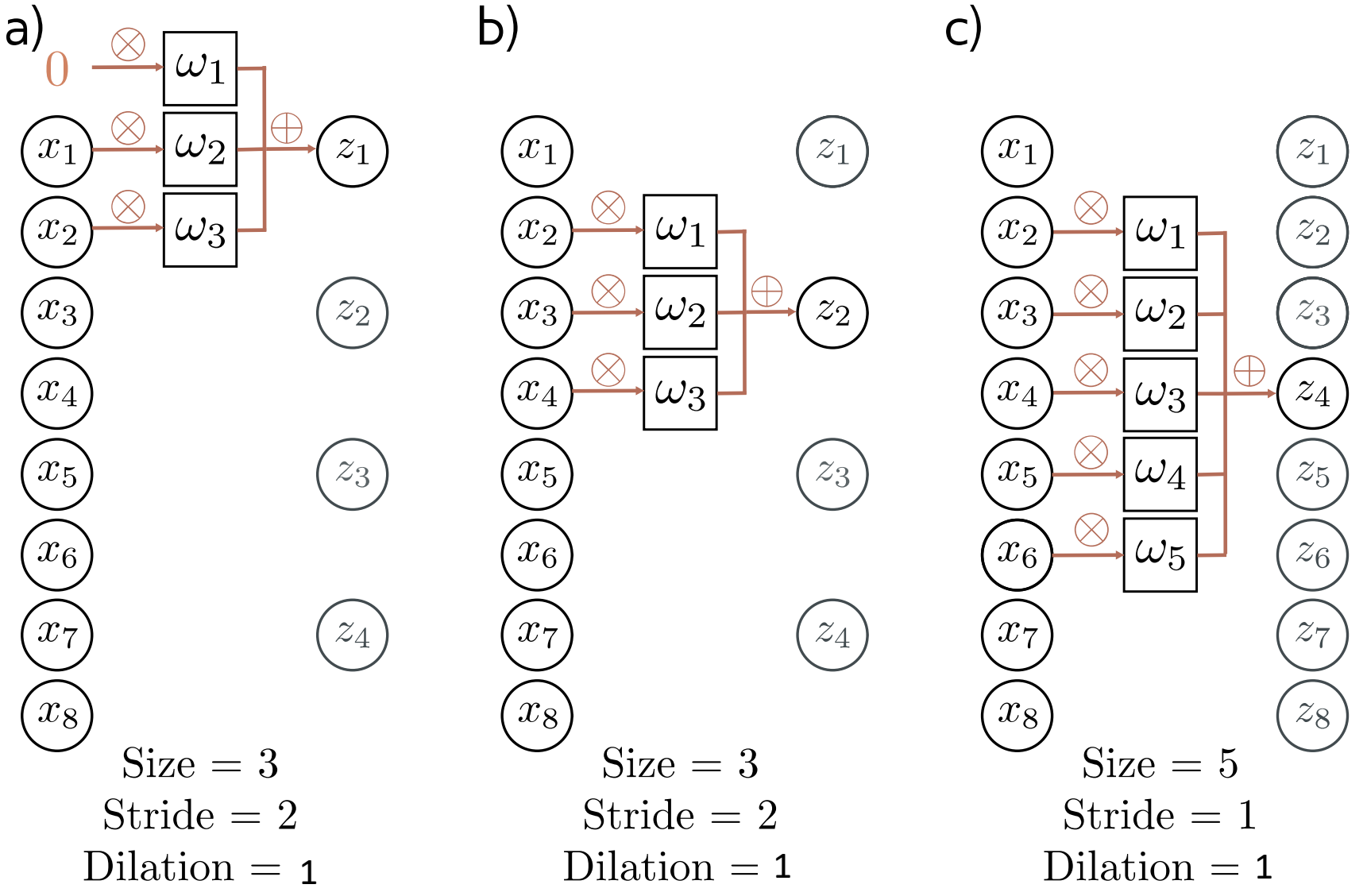
Size = 3
Stride = 2
Dilation = 1

b)

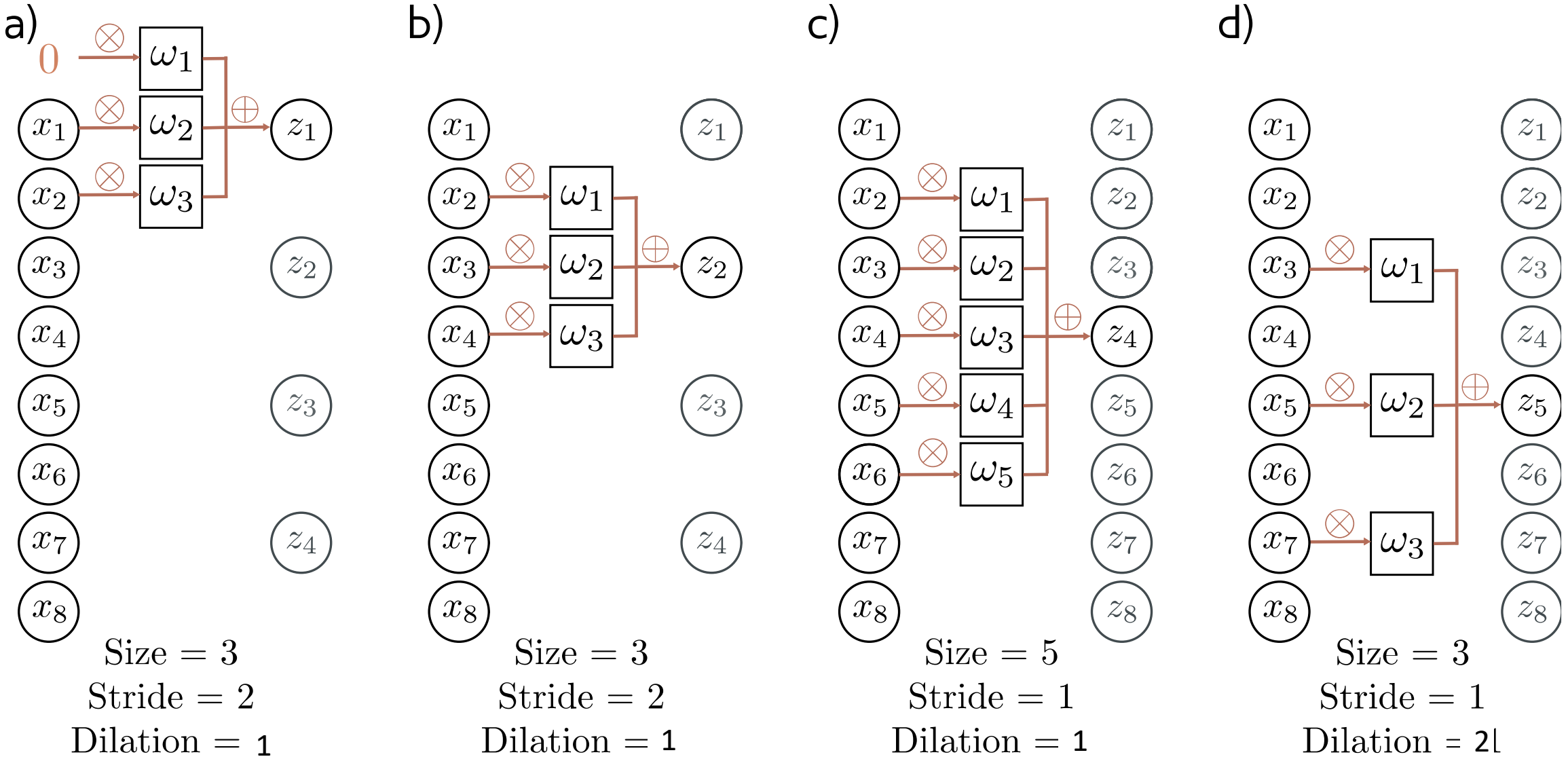


Size = 3
Stride = 2
Dilation = 1

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

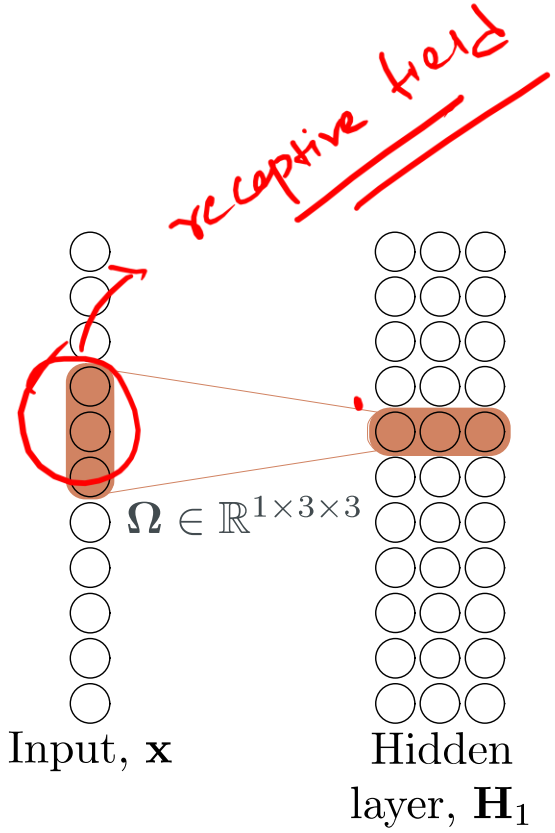


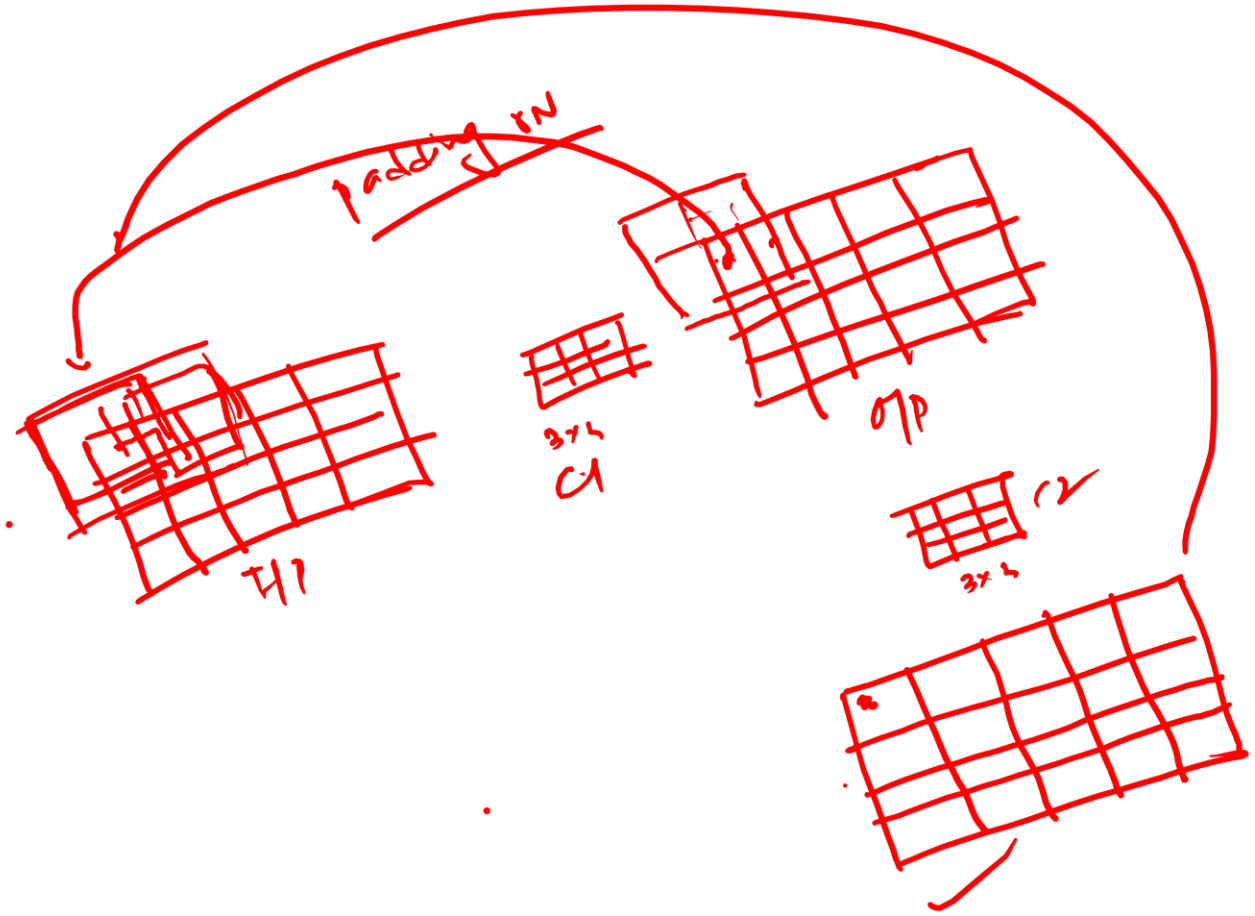
This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

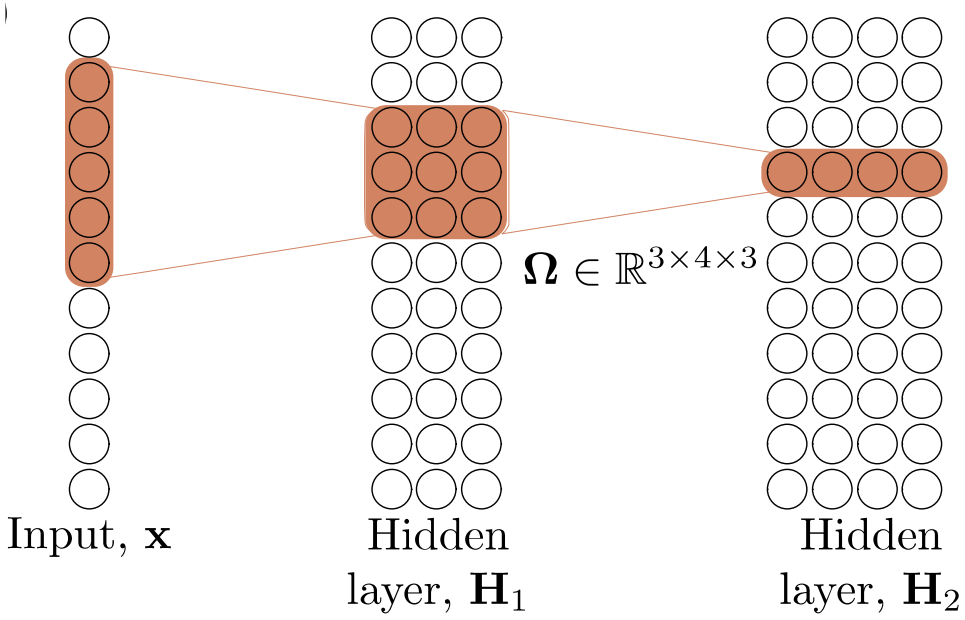
Receptive fields





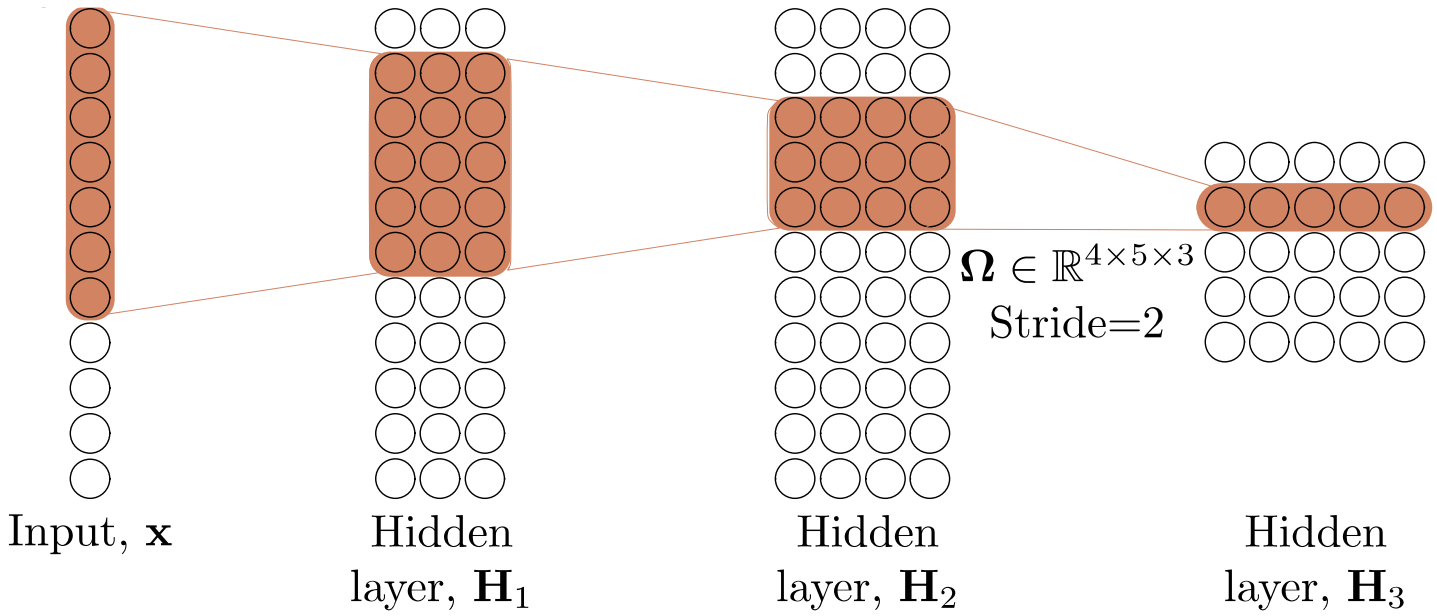
This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Receptive fields



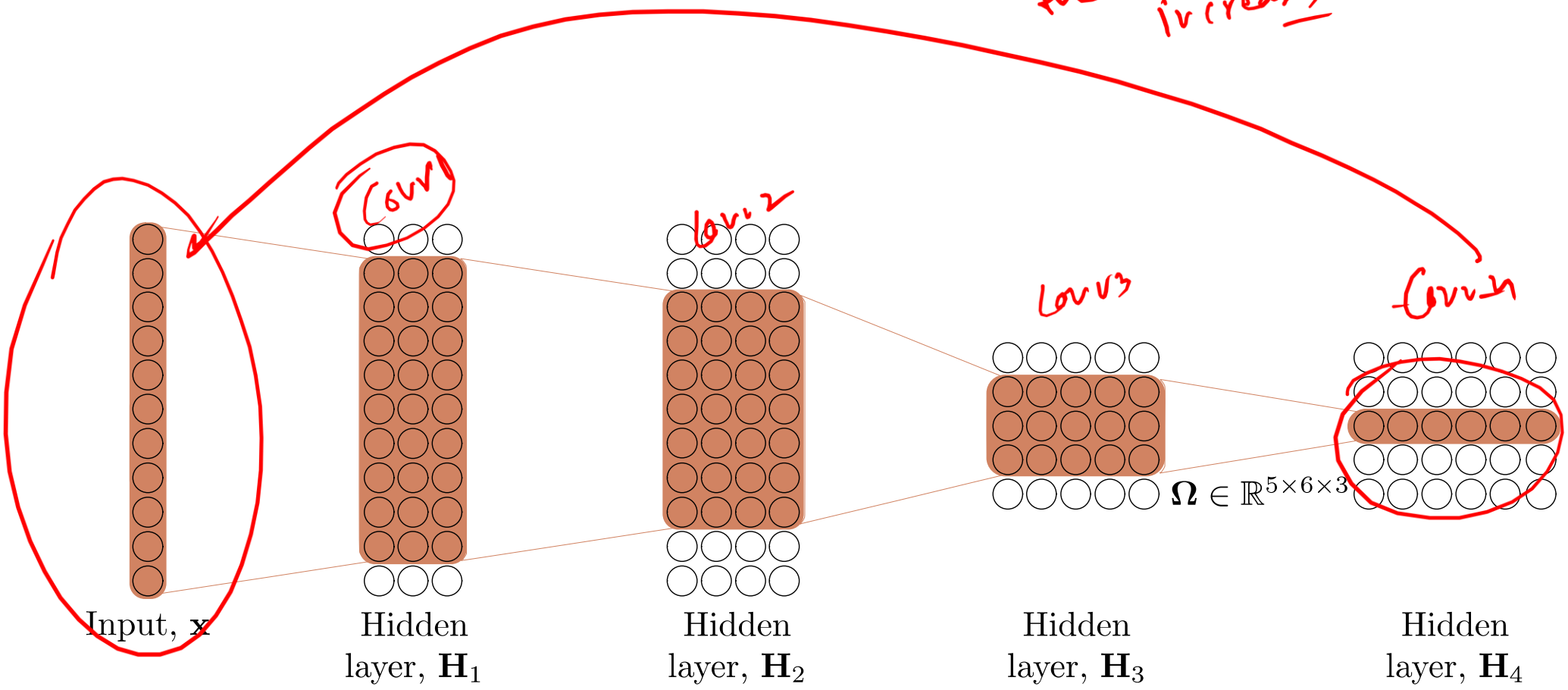
This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Receptive fields



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Receptive fields



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

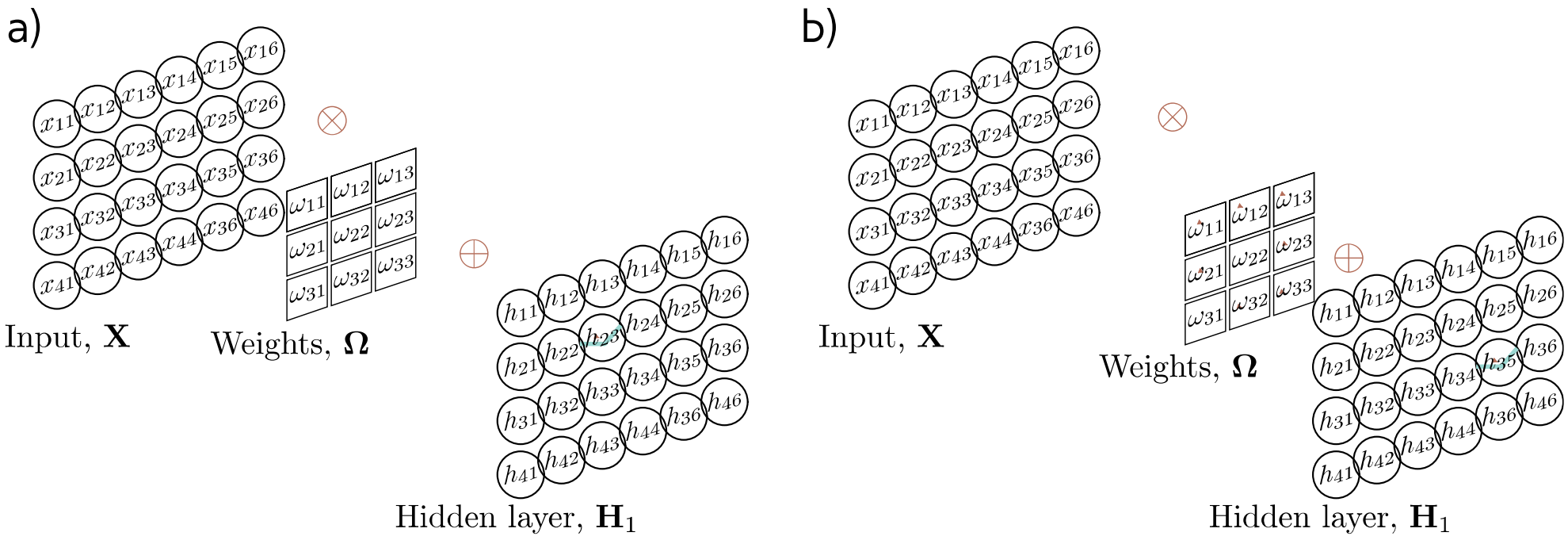
2D Convolution

- Convolution in 2D
  - Weighted sum over a K x K region
  - K x K weights
- Build into a convolutional layer by adding bias and passing through activation function

$$h_{i,j} = \mathrm{a}\left[\beta + \sum_{m=1}^{3}\sum_{n=1}^{3} \omega_{m,n} x_{i+m-2,j+n-2}\right]$$

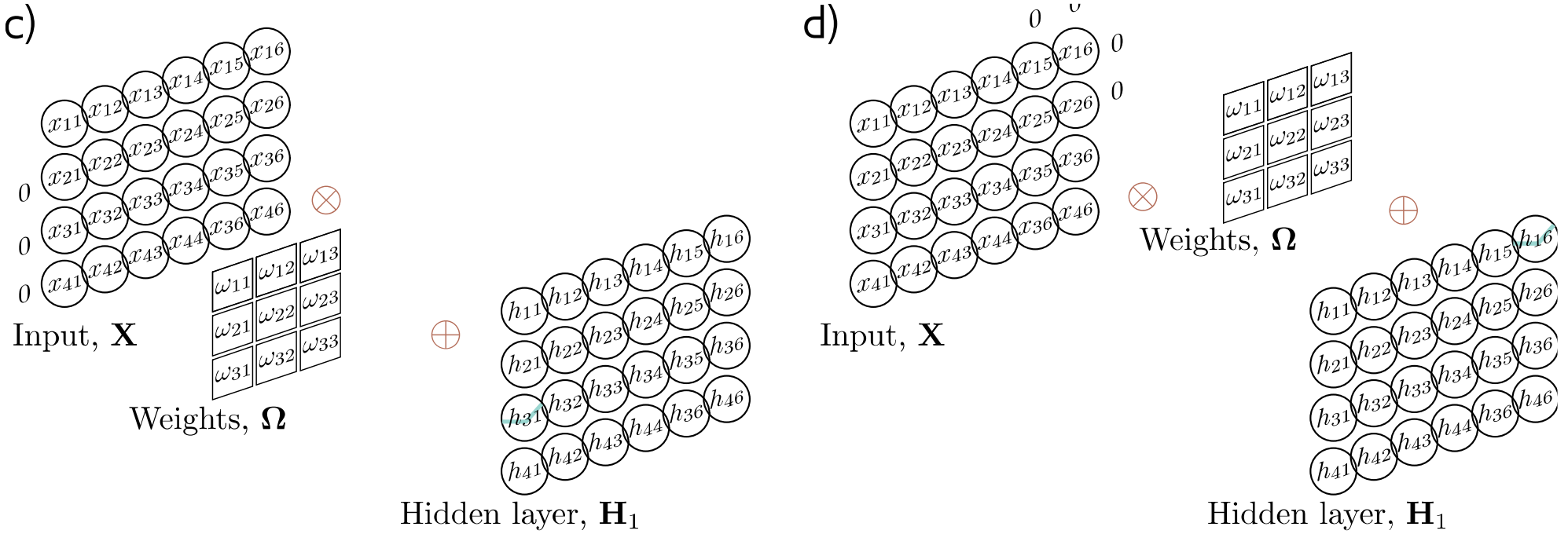
This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

2D Convolution



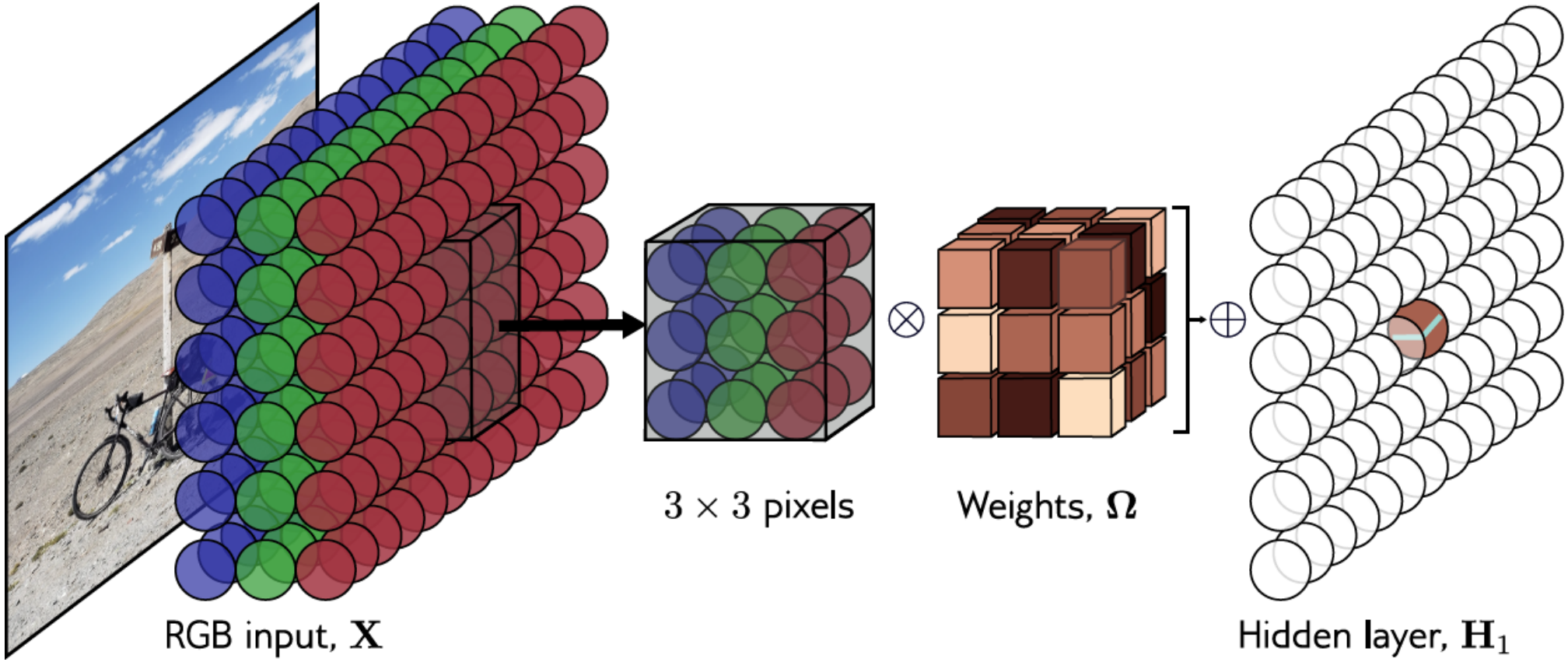
This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

2D Convolution



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Channels in 2D convolution



Kernel size, stride, dilation all work as you would expect

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

How many parameters?

- If there are C_i input channels and kernel size K x K

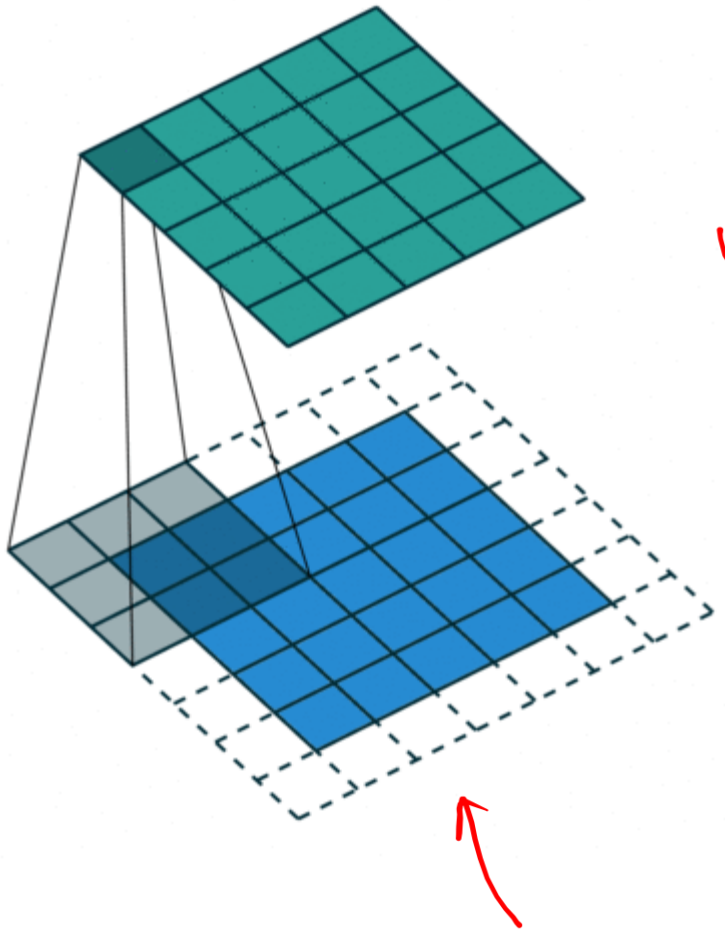
$$\boldsymbol{\omega} \in \mathbb{R}^{C_i \times K \times K} \qquad \boldsymbol{\beta} \in \mathbb{R}$$

- If there are C_i input channels and C_o output channels

$$\boldsymbol{\omega} \in \mathbb{R}^{C_i \times C_o \times K \times K} \qquad \boldsymbol{\beta} \in \mathbb{R}^{C_o}$$

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Different types of convolution



Dilation

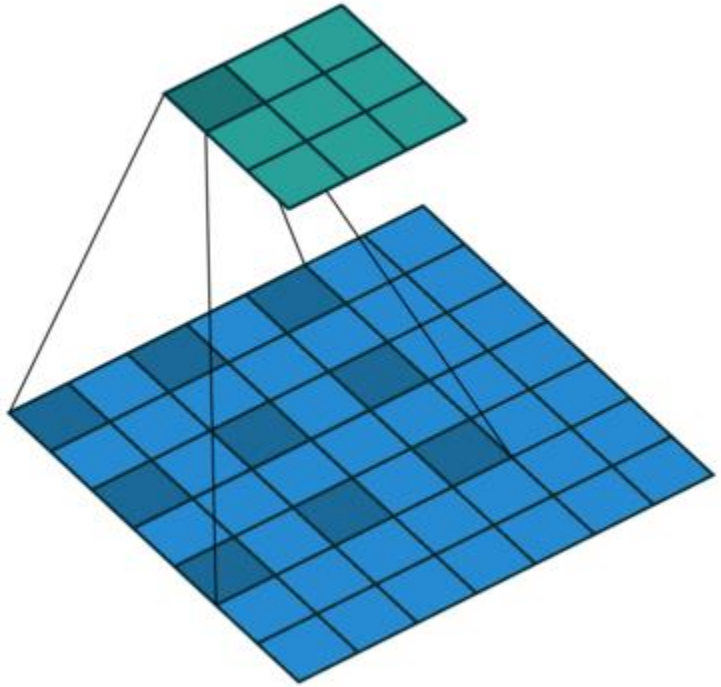
3×3 → 9

use ↓ best get the effect of

5×5

↓

25

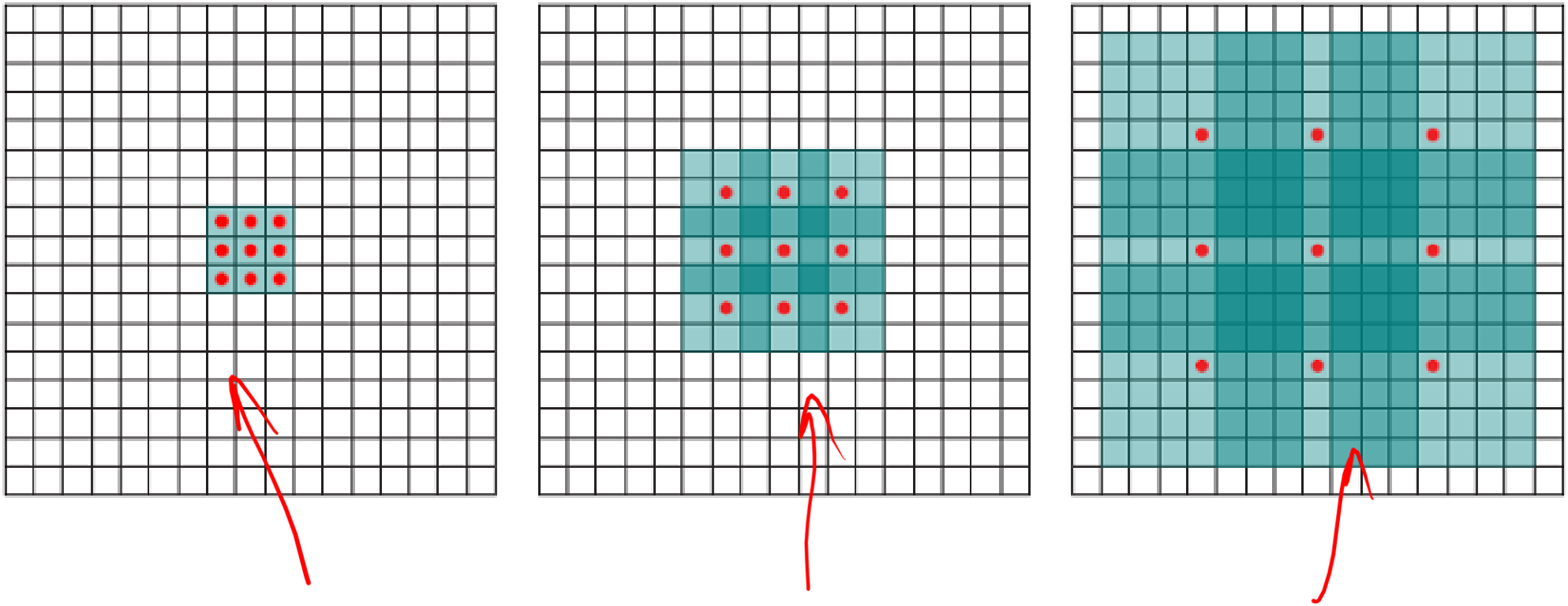


Normal vs dialated

Dilation width = 2

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Dilated convolution

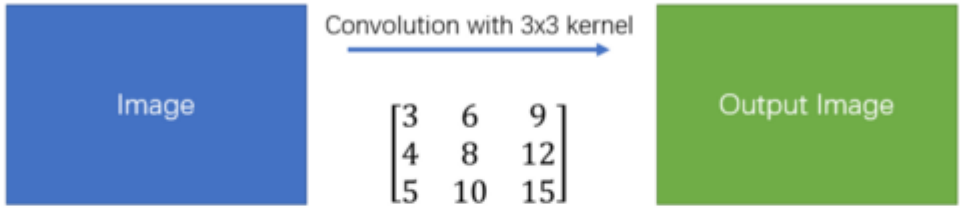


This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Spatially Separable convolution

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \times \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$$

Simple Convolution



Spatial Separable Convolution

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Assume we have a **10×10 input image** and we are using **no padding** and a **stride of 1**.

In a standard convolution, the 3×3 kernel must be centered over every possible pixel that allows the kernel to stay fully within the image boundaries.

- **Output Dimensions:** For a 10×10 input and 3×3 kernel, the output size is $(10 - 3 + 1) \times (10 - 3 + 1) = \mathbf{8} \times \mathbf{8}$.
- **Total Output Positions:** $8 \times 8 = \mathbf{64}$ positions.
- **Multiplications per Position:** At every one of those 64 positions, the 3×3 kernel performs $3 \times 3 = \mathbf{9}$ multiplications (one for each weight in the grid).

Calculation:

$$64 \text{ positions} \times 9 \text{ multiplications/position} = \mathbf{576} \text{ multiplications}$$

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Stage 1: Vertical Pass (3×1 Kernel)

We apply a 3×1 kernel to the original 10×10 image.

- **Vertical Shrinkage:** The height shrinks from 10 to $(10 - 3 + 1) = \mathbf{8}$.
- **Horizontal Persistence:** Since the kernel is only 1 pixel wide, we can slide it across all **10** columns.
- **Intermediate Output Size:** 8×10.
- **Multiplications:** For each of these 80 positions, we perform **3** multiplications.

Calculation:

$$(8 \times 10) \text{ positions} \times 3 \text{ multiplications/position} = \mathbf{240} \text{ multiplications}$$

Stage 2: Horizontal Pass (1×3 Kernel)

Now, we take the 8×10 intermediate result and apply a 1×3 horizontal kernel.

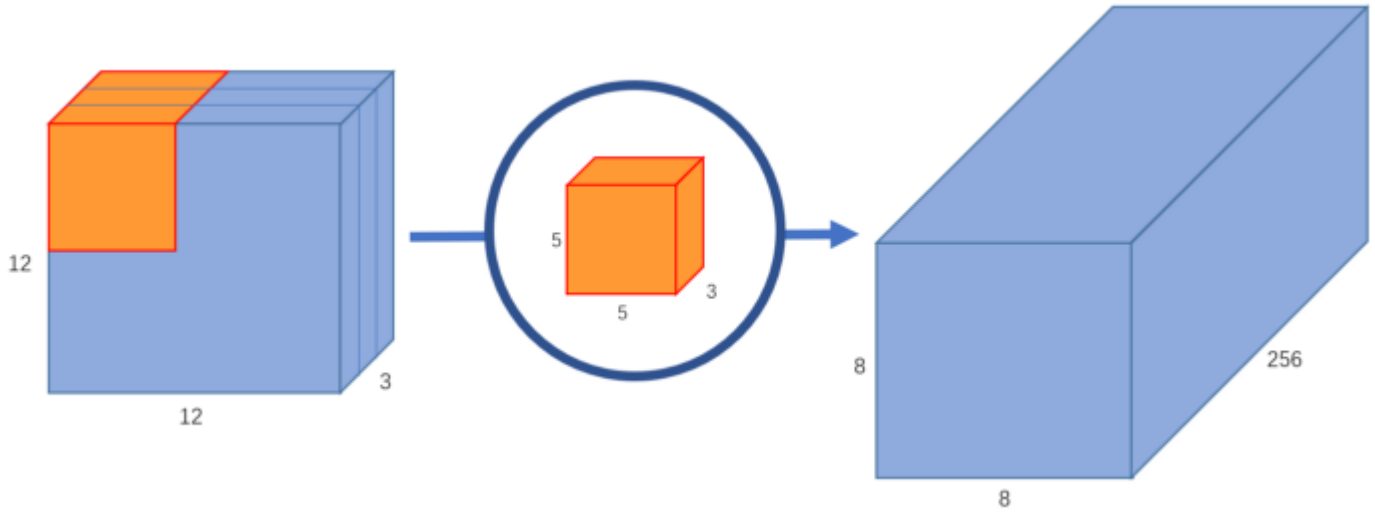
- **Vertical Persistence:** The height is already 8; the 1-pixel tall kernel doesn't change this.
- **Horizontal Shrinkage:** The width shrinks from 10 to $(10 - 3 + 1) = \mathbf{8}$.
- **Final Output Size:** 8×8 (matches the standard version).
- **Multiplications:** For each of these 64 positions, we perform **3** multiplications.

Calculation:

$$(8 \times 8) \text{ positions} \times 3 \text{ multiplications/position} = \mathbf{192} \text{ multiplications}$$

This file is meant for personal use. Copyright @ 2025
Sharing or publishing the contents in part or full is liable for legal action.

While **Spatially Separable Convolutions** decompose the width and height, **Depthwise Separable Convolutions** decompose the interaction between the **spatial dimensions** and the **channel dimensions**.

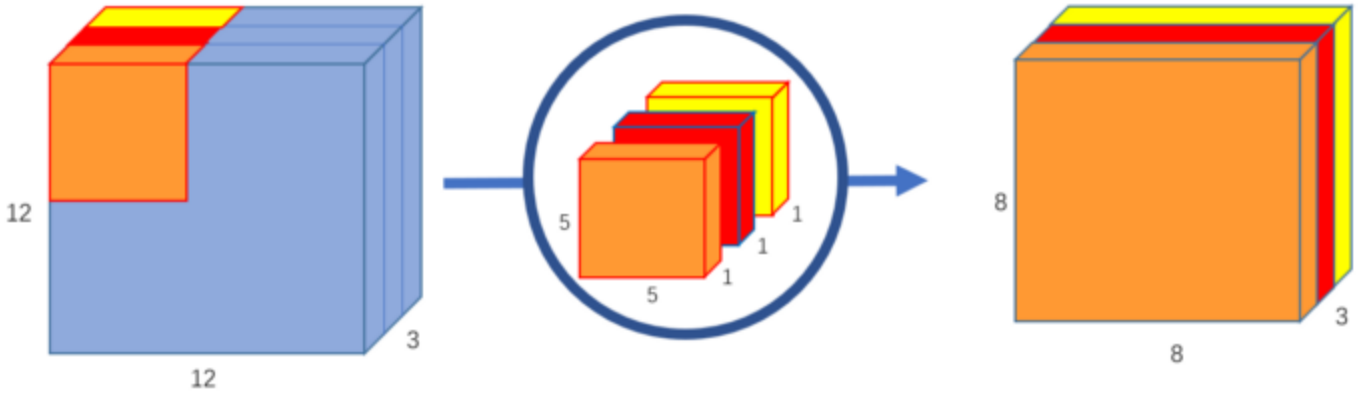


Convolving by 256 5x5 kernels over the input volume

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Depthwise separable convolution – step1

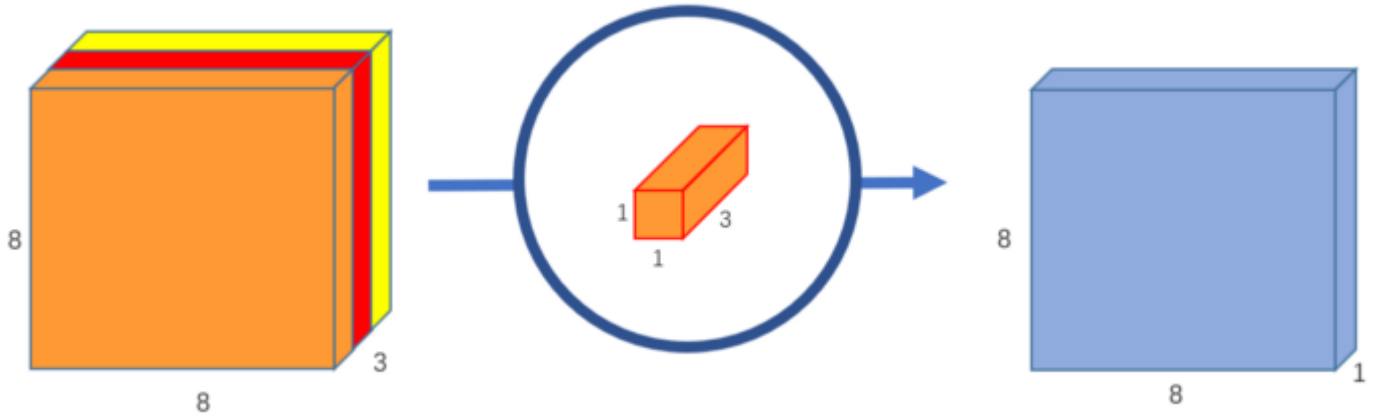
Along depth



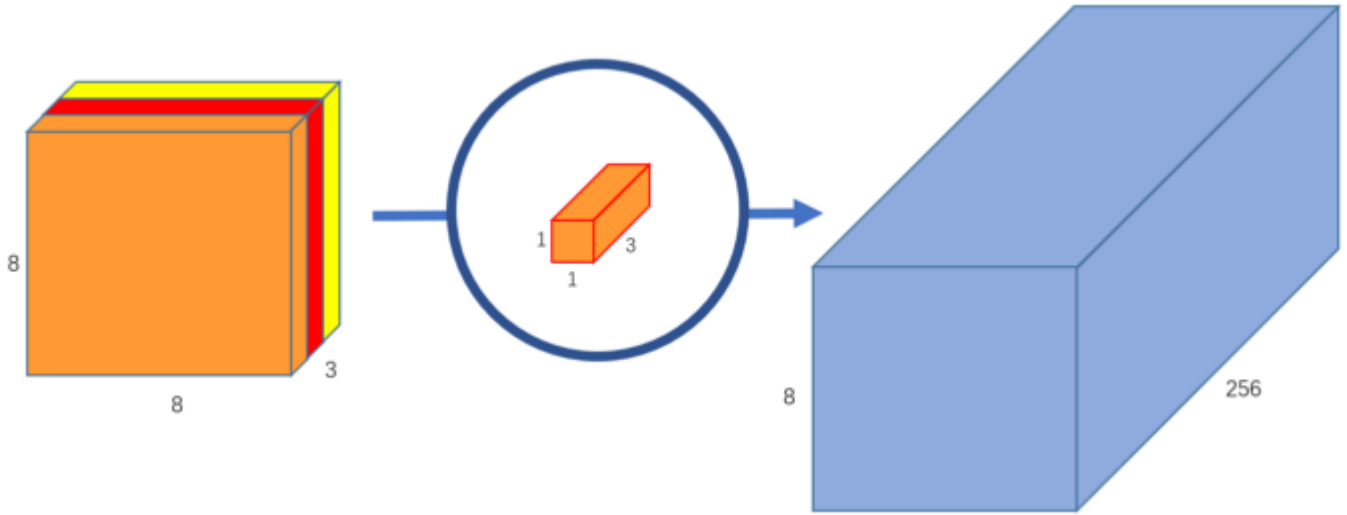
Each 5x5x1 kernel iterates 1 channel of the image (note: **1 channel**, not all channels), getting the scalar products of every 25 pixel group, giving out a 8x8x1 image. Stacking these images together creates a 8x8x3 image.

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Depthwise separable convolution – step2

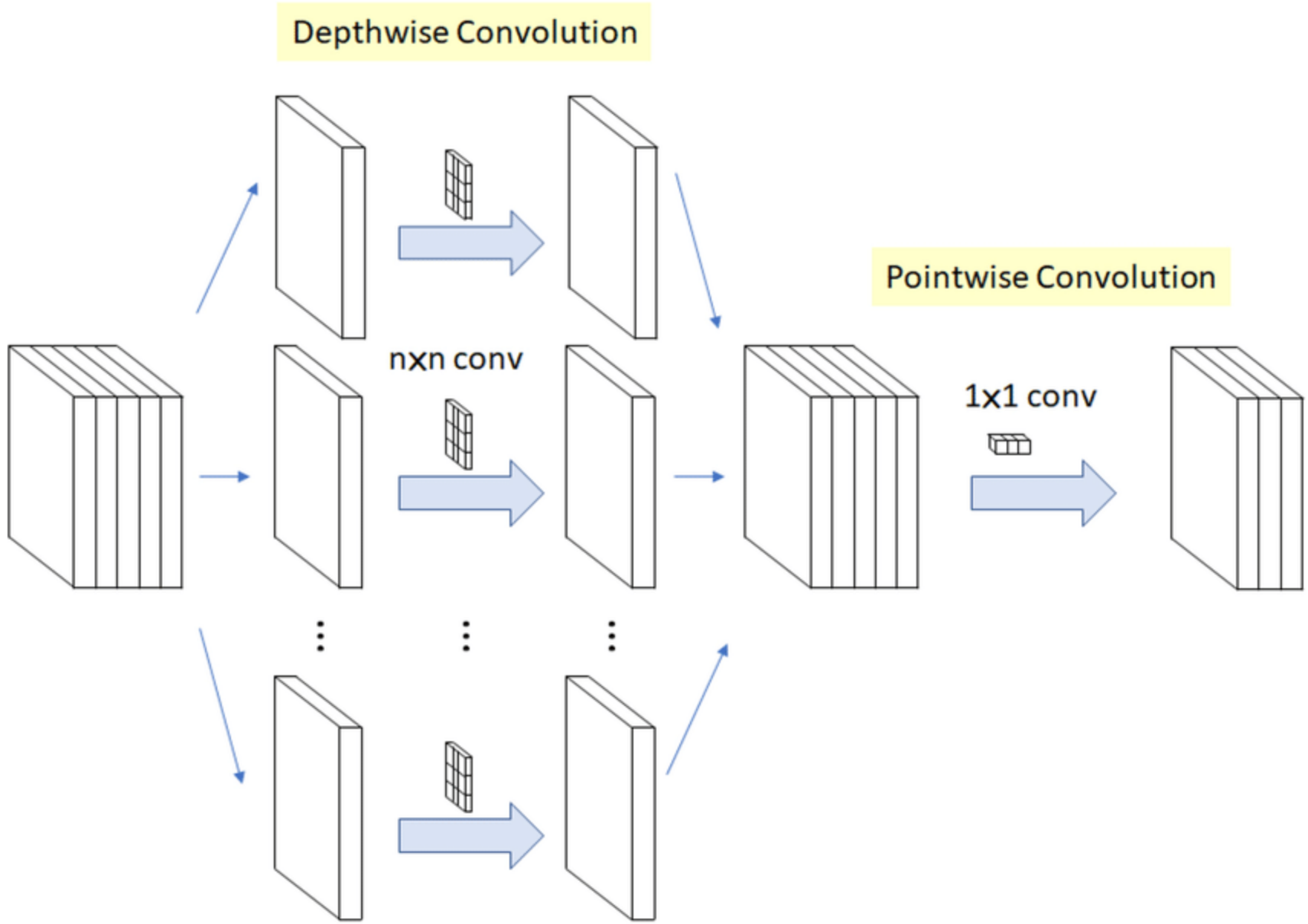


Pointwise



1x1 conv

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

```
model = tf.keras.Sequential([
        tf.keras.layers.Conv2D(8, 7, input_shape=(28,28,1), strides=2),
        tf.keras.layers.Conv2D(16, 5, strides=2),
        tf.keras.layers.Conv2D(32, 3),
        tf.keras.layers.GlobalAveragePooling2D(),
        tf.keras.layers.Dense(10, activation='softmax')
])
model.compile(loss='categorical_crossentropy', metrics=['acc'])
model.summary()
```

```
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d_43 (Conv2D)          (None, 11, 11, 8)         400

 conv2d_44 (Conv2D)          (None, 4, 4, 16)          3216

 conv2d_45 (Conv2D)          (None, 2, 2, 32)          4640

 global_average_pooling2d_17  (None, 32)               0
  (GlobalAveragePooling2D)

 dense_17 (Dense)            (None, 10)                330

=================================================================
Total params: 8,586
Trainable params: 8,586
Non-trainable params: 0
_________________________________________________________________
```

```
Test Accuracy = 0.9168999791145325
Time taken for evaluation of 10000 images = 1.43 seconds
```

```
model = tf.keras.Sequential([
        tf.keras.layers.SeparableConv2D(8, 7, input_shape=(28,28,1), strides=2),
        tf.keras.layers.SeparableConv2D(16, 5, strides=2),
        tf.keras.layers.SeparableConv2D(32, 3),
        tf.keras.layers.GlobalAveragePooling2D(),
        tf.keras.layers.Dense(10, activation='softmax')
])
model.compile(loss='categorical_crossentropy', metrics=['acc'])
model.summary()
```

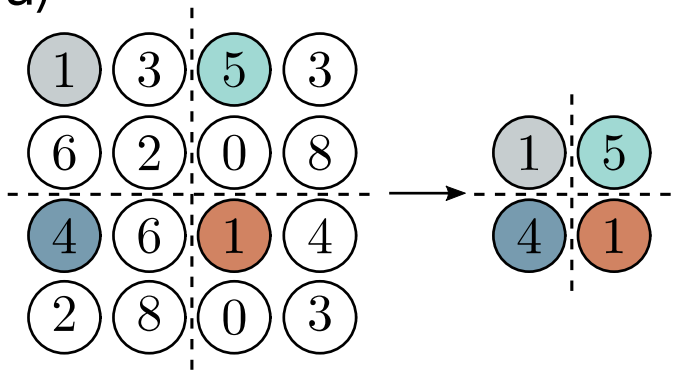
```
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 separable_conv2d_9 (Separab  (None, 11, 11, 8)        65
 leConv2D)

 separable_conv2d_10 (Separa  (None, 4, 4, 16)         344
 bleConv2D)

 separable_conv2d_11 (Separa  (None, 2, 2, 32)         688
 bleConv2D)

 global_average_pooling2d_18  (None, 32)               0
  (GlobalAveragePooling2D)

 dense_18 (Dense)            (None, 10)                330

=================================================================
Total params: 1,427
Trainable params: 1,427
Non-trainable params: 0
_________________________________________________________________
```

```
Test Accuracy = 0.9021000266075134
Time taken for evaluation of 10000 images = 1.11 seconds
```

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Downsampling

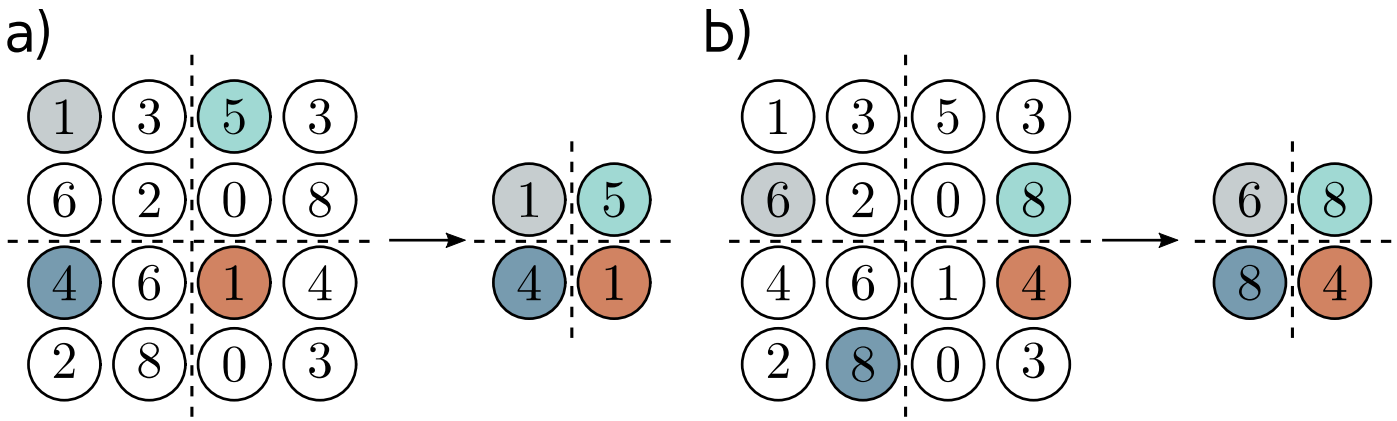
a)



Sample every other position (equivalent to stride two)

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Downsampling

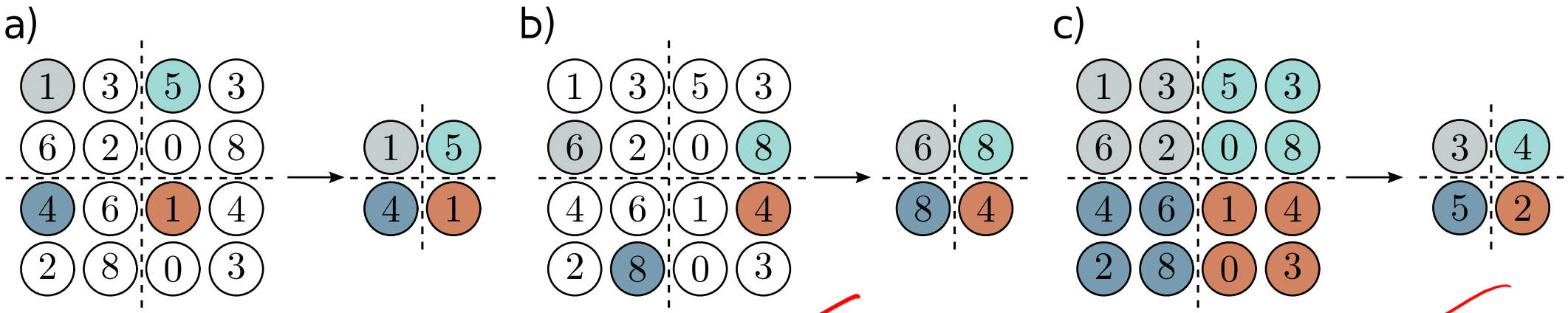


Sample every other position (equivalent to stride two)

Max pooling (partial invariance to translation)

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Downsampling



Sample every other position (equivalent to stride two)

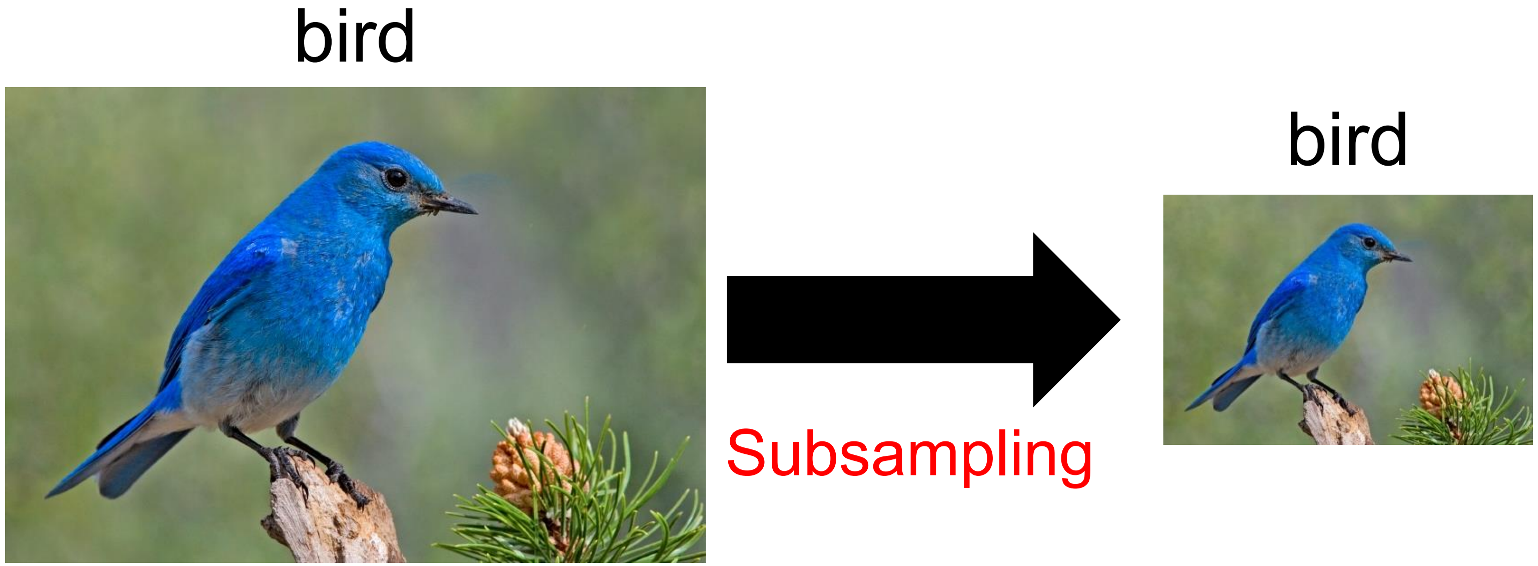
Max pooling (partial invariance to translation)

Mean pooling

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Why Pooling

- Subsampling pixels will not change the object



✓ We can subsample the pixels to make image smaller

✓ fewer parameters to characterize the image

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Pooling or strided convolution?

STRIVING FOR SIMPLICITY: THE ALL CONVOLUTIONAL NET

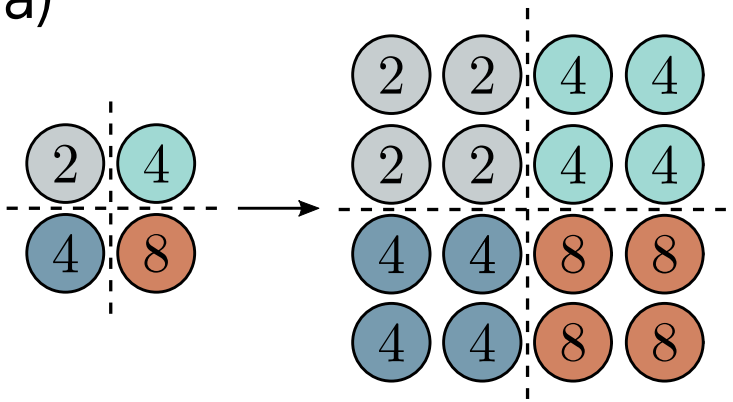
Jost Tobias Springenberg*, Alexey Dosovitskiy*, Thomas Brox, Martin Riedmiller
Department of Computer Science
University of Freiburg
Freiburg, 79110, Germany
{springj, dosovits, brox, riedmiller}@cs.uni-freiburg.de

"when pooling is replaced by an additional convolution layer with stride r = 2 performance stabilizes and even improves on the base model"

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Upsampling

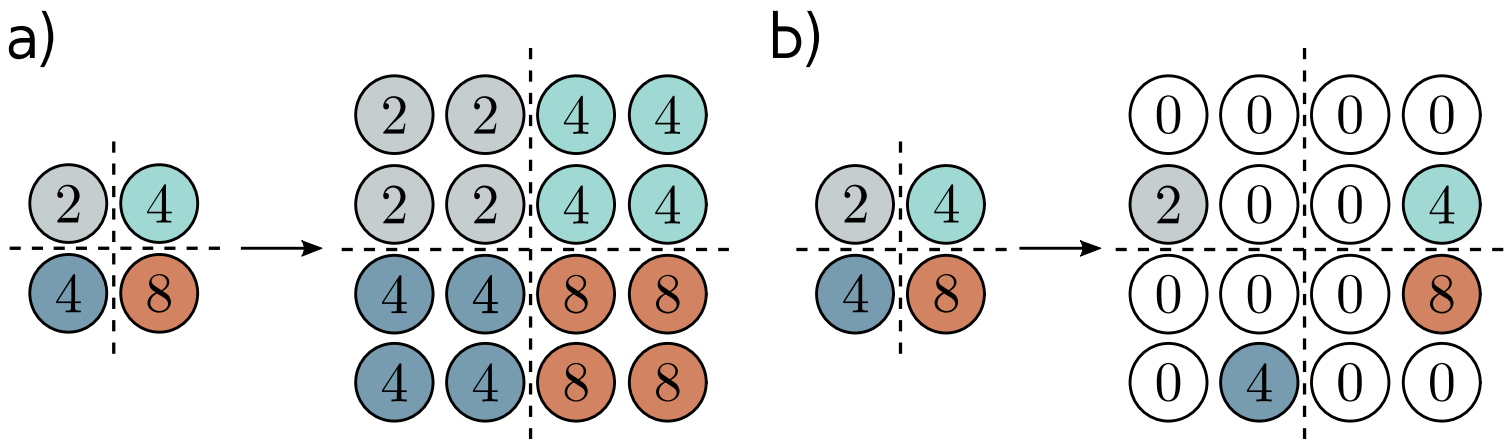
a)



Duplicate

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Upsampling

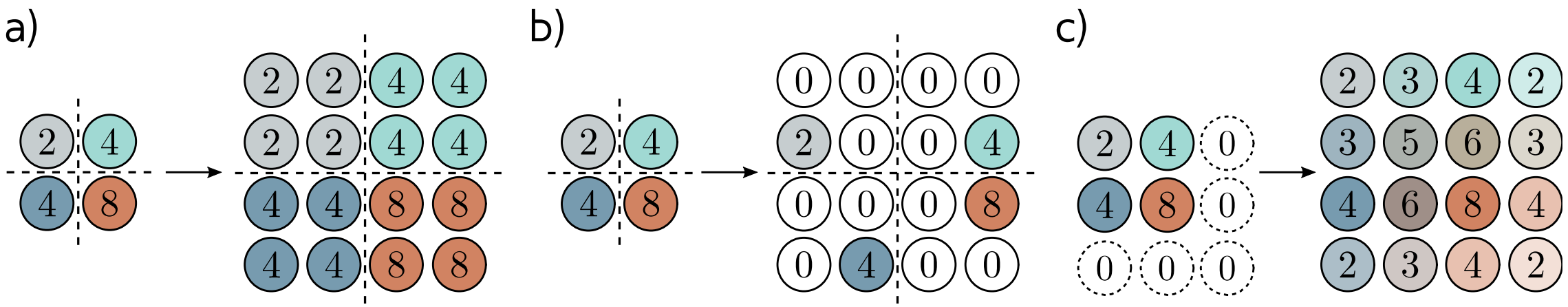


Duplicate

Max-upsampling

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Upsampling



Duplicate

Max-upsampling

Bilinear interpolation

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Transposed convolutions

- It can be thought of as a regular convolution, but in the reverse direction. Instead of mapping a 3×3 area to a single pixel, it maps a single pixel to a 3×3 area in the output. Mathematically, it is equivalent to the backward pass (gradient calculation) of a standard convolution.

To map a single pixel to a 3×3 area using a **Transposed Convolution**, you should use a **kernel size of 3**, a **stride of 3**, and **no padding**.

The relationship between the input size (H_{in}), output size (H_{out}), stride (s), padding (p), and kernel size (k) for a transposed convolution is:

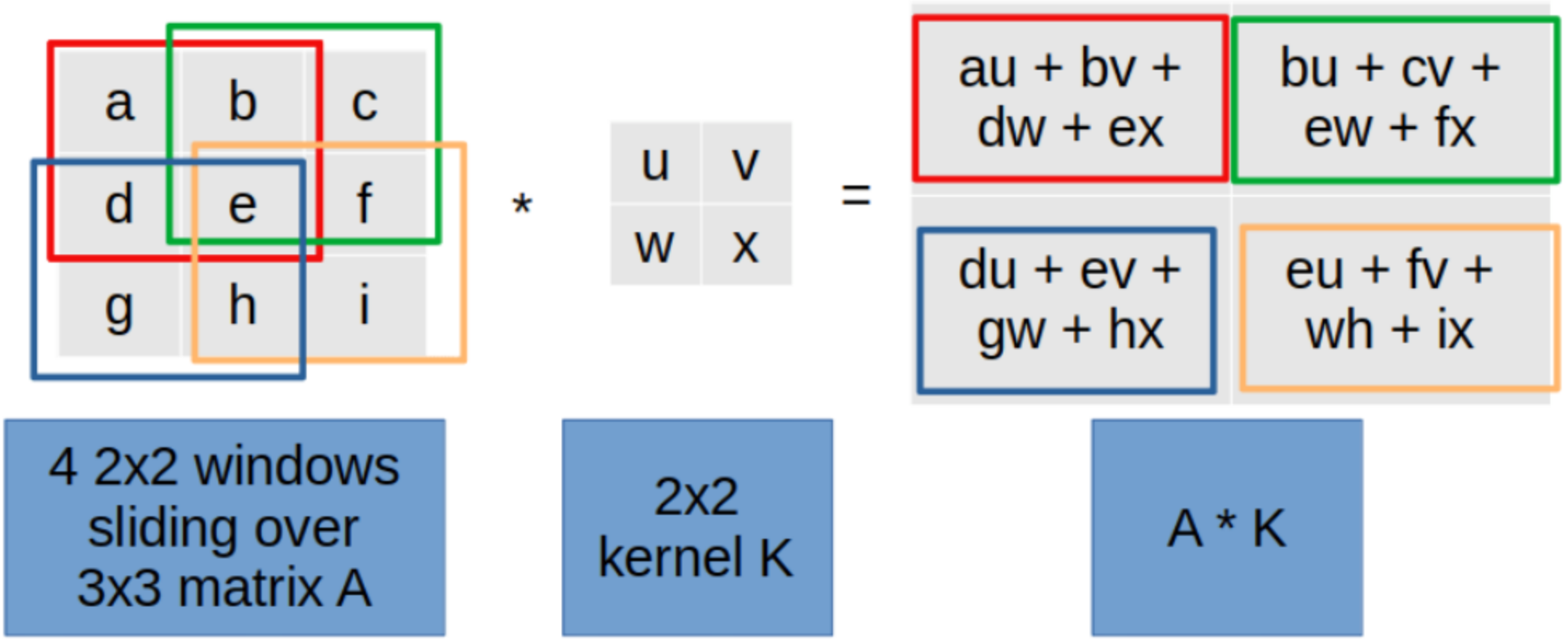
$$H_{out} = (H_{in} - 1) \times s - 2p + k$$

$$H_{out} = (1 - 1) \times 3 - 2(0) + 3$$

$$H_{out} = 0 - 0 + 3 = \mathbf{3}$$

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Convolution as matrix vector multiplication



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Let A be a matrix of size $n \times n$ and K a $k \times k$ kernel. For convenience, let $t = n - k + 1$.

To calculate the convolution $A * K$, we need to calculate the matrix-vector multiplication $Mv^T = v'^T$ where:

- M is a $t^2 \times n^2$ block matrix we get from the kernel K
- v is a row vector with the n^2 elements of A concatenated row by row
- v' is a row vector of t^2 elements which can be reshaped into a $t \times t$ matrix by splitting the row vector into t rows of t elements

$$\begin{pmatrix} x1 & x2 & x3 \\ x4 & x5 & x6 \\ x7 & x8 & x9 \end{pmatrix} * \begin{pmatrix} k1 & k2 \\ k3 & k4 \end{pmatrix}$$

Convolution as matrix-vector product

Here is a constructed matrix with a vector:

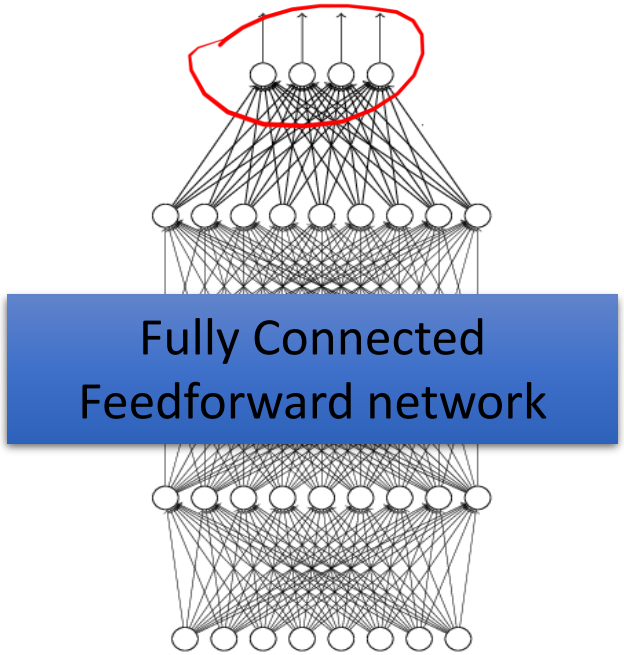
$$\begin{pmatrix} k1 & k2 & 0 & k3 & k4 & 0 & 0 & 0 & 0 \\ 0 & k1 & k2 & 0 & k3 & k4 & 0 & 0 & 0 \\ 0 & 0 & 0 & k1 & k2 & 0 & k3 & k4 & 0 \\ 0 & 0 & 0 & 0 & k1 & k2 & 0 & k3 & k4 \end{pmatrix} \cdot \begin{pmatrix} x1 \\ x2 \\ x3 \\ x4 \\ x5 \\ x6 \\ x7 \\ x8 \\ x9 \end{pmatrix}$$

which is equal to $\begin{pmatrix} k1\,x1 + k2\,x2 + k3\,x4 + k4\,x5 \\ k1\,x2 + k2\,x3 + k3\,x5 + k4\,x6 \\ k1\,x4 + k2\,x5 + k3\,x7 + k4\,x8 \\ k1\,x5 + k2\,x6 + k3\,x8 + k4\,x9 \end{pmatrix}$.

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

The whole CNN

cat dog ……



Fully Connected Feedforward network

Convolution

Max Pooling

Convolution

Max Pooling

Can repeat many times

Flattened

This file is meant for personal use by mepratikpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

The whole CNN

Convolution

Max Pooling

A new image

Convolution

Max Pooling

A new image

Flattened

Fully Connected Feedforward network

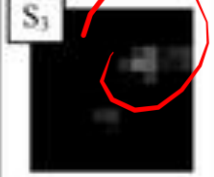
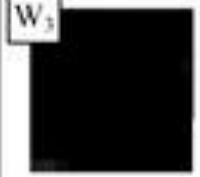
cat dog ……

size of the feature map is reduced

feature map

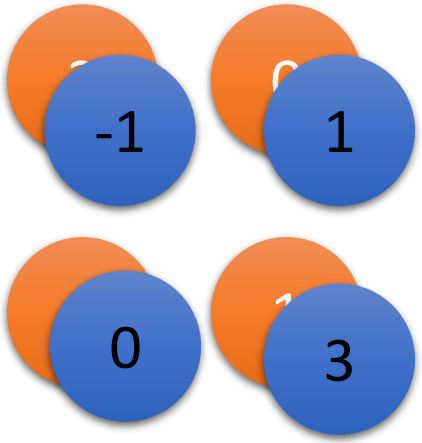
feature map

This is meant for personal use by [redacted]@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

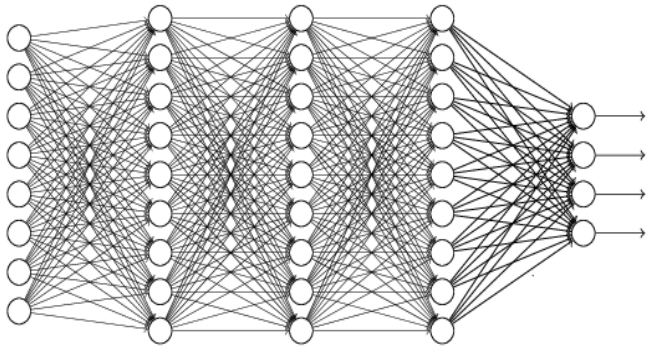
Raw Image	Feature Map	Top Activation Image Crops					
x	$v^{(5)}(x)$	*1st*	*2nd*	*3rd*	*4th*	*5th*	*6th*
 Strongly correlated convolutional kernels	S_1 						
	S_2 						
	S_3 						
 Weakly correlated convolutional kernels	W_1 						
	W_2 						
	W_3 						

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Flattening



Flattened



Fully Connected
Feedforward network

This file is meant for personal use by mayuravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Conv Net Topology

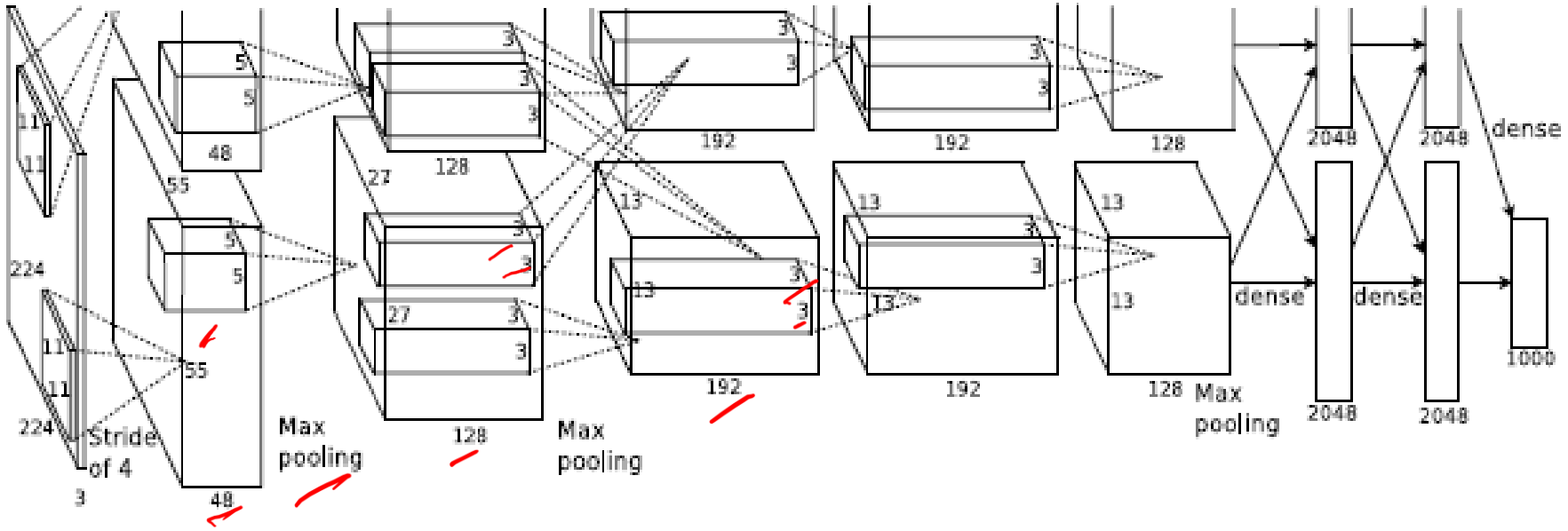
- 5 convolutional layers
- 3 fully connected layers + soft-max
- 650K neurons , 60 Mln weights

ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky University of Toronto kriz@cs.utoronto.ca

Ilya Sutskever University of Toronto ilya@cs.utoronto.ca

Geoffrey E. Hinton University of Toronto hinton@cs.utoronto.ca



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

track cycling
cycling
track cycling
road bicycle racing
marathon
ultramarathon

ultramarathon
ultramarathon
half marathon
running
marathon
inline speed skating

heptathlon
heptathlon
decathlon
hurdles
pentathlon
sprint (running)

bikejoring
mushing
bikejoring
harness racing
skijoring
carting

longboarding
longboarding
aggressive inline skating
freestyle scootering
freeboard (skateboard)
sandboarding

ultimate (sport)
ultimate (sport)
hurling
flag football
association football
rugby sevens

demolition derby
demolition derby
monster truck
mud bogging
motocross
grand prix motorcycle racing

telemark skiing
snowboarding
telemark skiing
nordic skiing
ski touring
skijoring

whitewater kayaking
whitewater kayaking
rafting
kayaking
canoeing
adventure racing

arena football
indoor american football
arena football
canadian football
american football
women's lacrosse

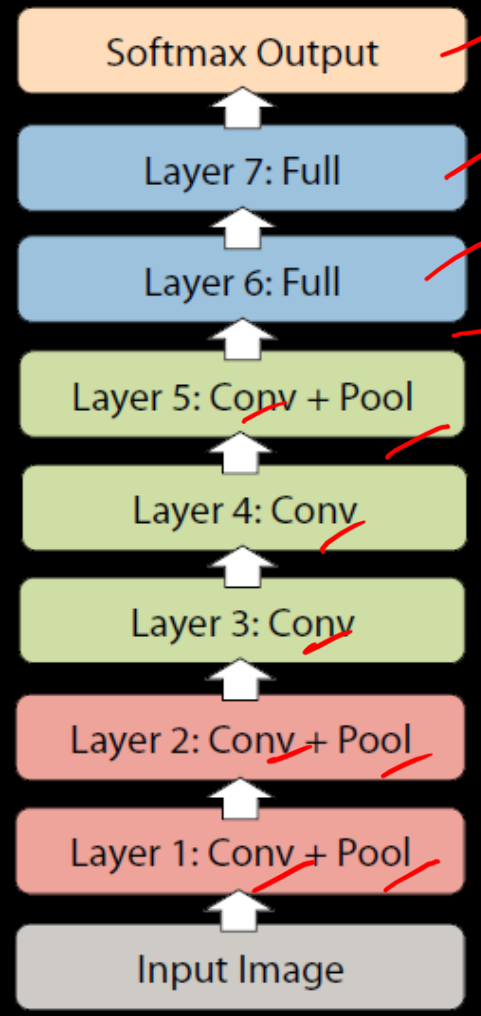
reining
barrel racing
rodeo
reining
cowboy action shooting
bull riding

eight-ball
nine-ball
blackball (pool)
trick shot
eight-ball
straight pool

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Why do we need a deep CNN?

- 8 layers total
- Trained on Imagenet dataset [Deng et al. CVPR'09]
- 18.2% top-5 error
- Our reimplementation:
  18.1% top-5 error

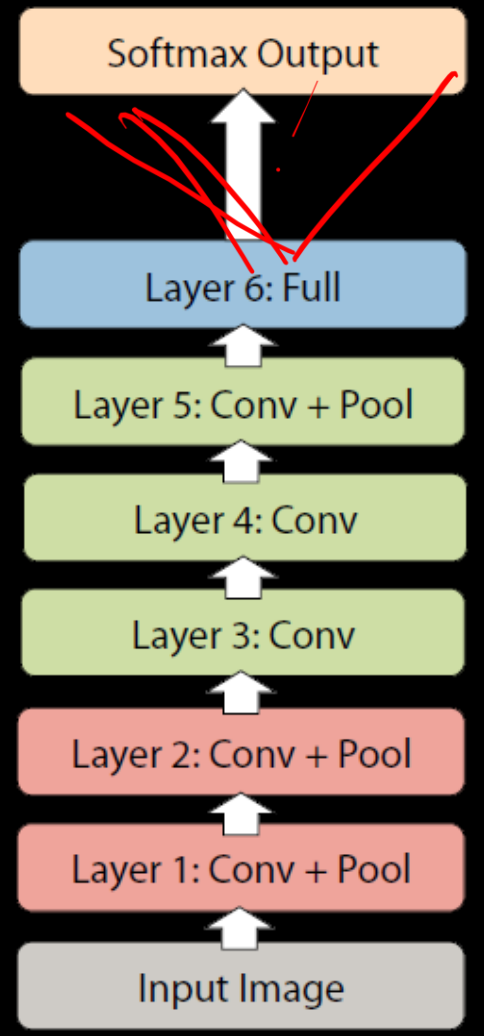


Courtsey: ICRI

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Why do we need a *deep* CNN?

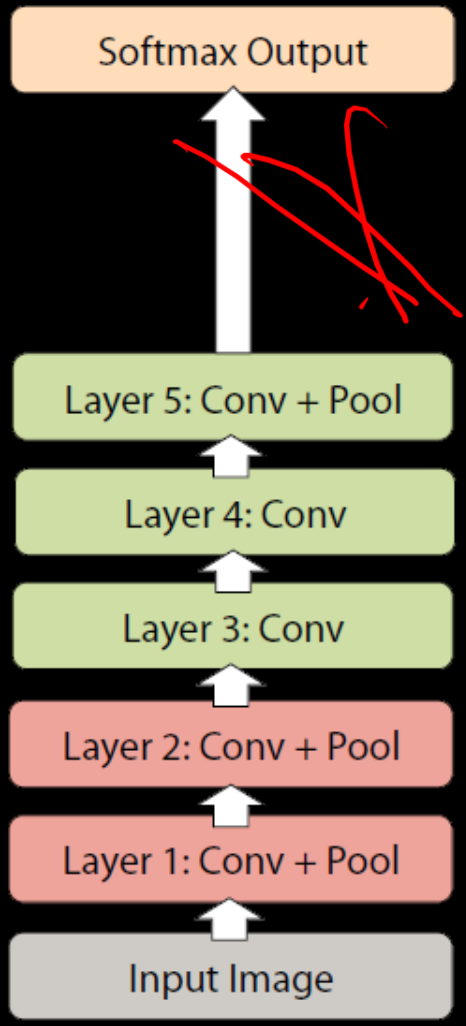
- Remove top fully connected layer
  - Layer 7
- Drop 16 million parameters
- Only 1.1% drop in performance!



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Why do we need a *deep* CNN?

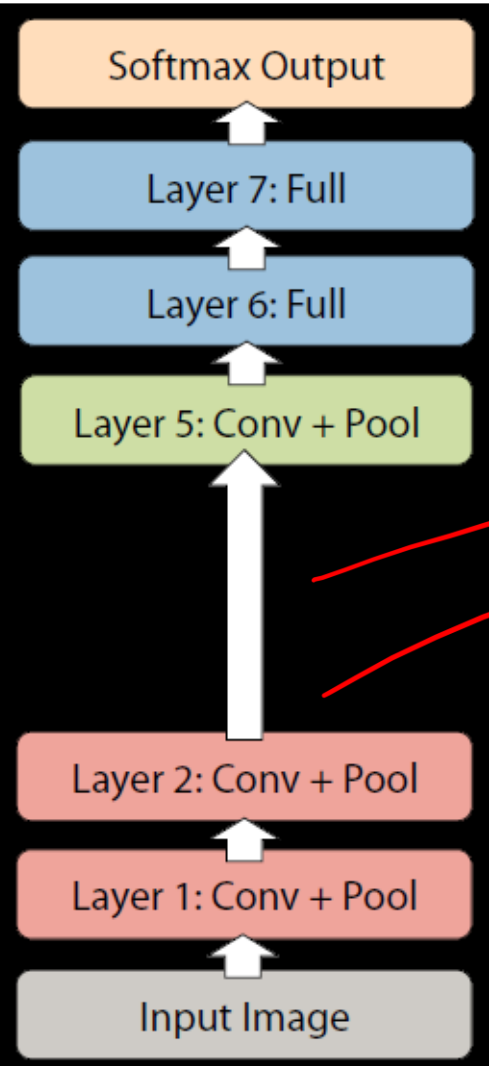
- Remove both fully connected layers
  - Layer 6 & 7
- Drop ~50 million parameters
- 5.7% drop in performance



This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Why do we need a *deep* CNN?

- Now try removing upper feature extractor layers:
  - Layers 3 & 4
- Drop ~1 million parameters
- 3.0% drop in performance

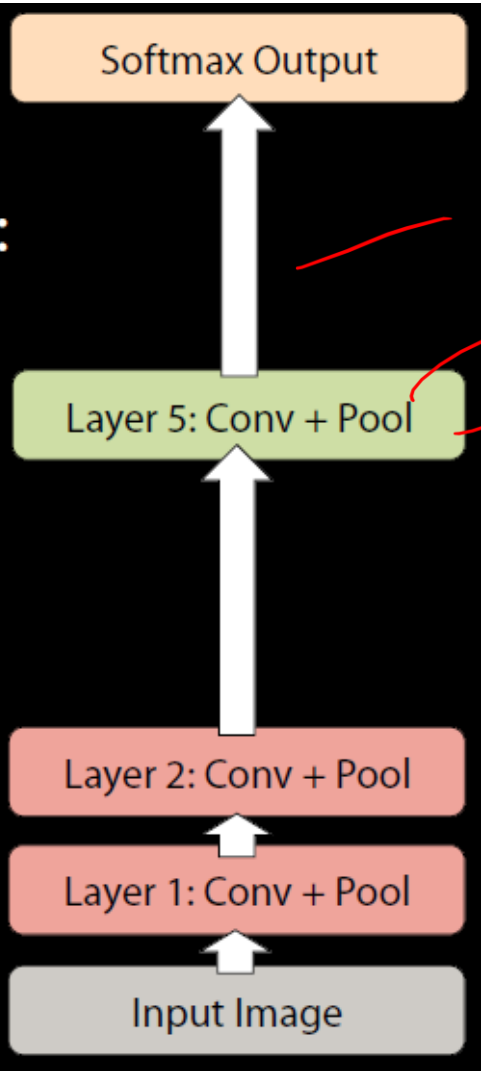


This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Why do we need a *deep* CNN?

- Now try removing upper feature extractor layers & fully connected:
  - Layers 3, 4, 6 ,7
- Now only 4 layers
- 33.5% drop in performance

→Depth of network is key



This file is meant for personal use by mepravinpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Suggested reading

A guide to convolution arithmetic for deep learning

Vincent Dumoulin[1]★ and Francesco Visin[2]★†

★MILA, Université de Montréal
†AIRLab, Politecnico di Milano

January 12, 2018

This file is meant for personal use by mepravintpatil@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.